



UNIVERSITÀ DEGLI STUDI DI CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA

UNIVERSITÉ DE ROUEN NORMANDIE
LABORATOIRE D'INFORMATIQUE, DU TRAITEMENT DE
L'INFORMATION ET DES SYSTÈMES

PH.D. THESIS

Text Sampling for Efficient Information Processing

Francesco Pio Marino

SUPERVISORS:
Prof. Simone Faro
Prof. Thierry Lecroq

Cycle XXXIX — Academic Year 2026

A mia Mamma, la mia forza.
A mio Papà, il mio esempio.
A Voi devo tutto!

Acknowledgements

I would like to thank my parents, who have always encouraged me and given me the freedom to make my own decisions about my studies, my career, and my life. They have supported me at every stage of this journey and have always been there to take care of the things I could not take care of myself, especially during my PhD and my many travels. Without them, this work would not have been possible.

A special thanks goes to Dominik Köppl for hosting me at the University of Yamanashi, for all the interesting discussions we had, and for the work we did together. Our lunches and dinners in Kofu, as well as what were probably the most enjoyable and productive three months of work of my career so far, made my time in Japan a truly special experience. I am very grateful for this collaboration and I hope we will have many more opportunities to work together, discuss ideas, and share good food in the future.

A warm thanks goes to Felipe A. Louza and Guilherme P. Telles for arranging and hosting my stay at Universidade Estadual de Campinas (Unicamp). They gave me the opportunity to spend two wonderful months in one of my favorite countries, Brazil. I am very grateful for all the conversations we had, both about research and everything else, and for the interest they showed in my work. I hope we can continue working together, meet again soon, and have many more discussions about research and new ideas.

A sincere thanks goes to Arianna Pavone, one of the kindest and friendliest colleagues I could have asked for, and the best person I could have had by my side when organizing my first workshop. She was always available whenever I needed help or support and guided me through many situations during these years. Our trips to the USA also became some of the most memorable moments of my PhD, and having her there made them even better. I hope we will organize many more events together and, most importantly, continue the friendship and mutual support that we have built over these years.

A heartfelt thanks goes to Thierry Lecroq, who agreed to become my co-supervisor from the very beginning, even though he did not know me personally at the time. By accepting me in France and giving me the opportunity to join the cotutelle program with the University of Rouen, he had a much bigger impact on my PhD than he probably realizes.

My stays in Rouen were the beginning of my interest in spending longer periods abroad and working in different research environments. Thanks to this experience, I became much more willing to look for other international opportunities, which eventually led me to spend time in Japan, Brazil, and elsewhere. These experiences were an important part of my PhD and, I believe, made me a better and more

independent researcher.

Throughout these years, Thierry has always been available whenever I needed advice or support. He listened to my many ideas, project proposals, funding applications, and possible research directions, and he always gave me the freedom to explore them. Even when some of my ideas were probably a little too ambitious, he encouraged me to try them rather than telling me to stop. This freedom was very important to me and allowed me to discover new topics and approaches that I would probably never have explored otherwise.

I am also very grateful for his kindness and hospitality during my many stays in Rouen. He always made me feel welcome and made my time there much more enjoyable. I truly hope that we will continue working together and having discussions, both about research and everything else, for many years to come.

Above all, I would like to thank Simone Faro, my supervisor in Catania and the most important academic figure in my life. He met me when I was sixteen years old, when I was still in high school and taking my first steps in competitive programming. From the very beginning, he saw something in me and believed in me, even at times when I was not so sure about myself or about the path I was taking.

He introduced me to research, guided my first steps, and taught me many of the things that form the basis of what I know today. More importantly, he was always there throughout these years. His patience, availability, and support helped me grow not only as a researcher, but also as a person.

He always gave me the freedom to follow the research directions that interested me, even when they were not necessarily the most obvious choices. He supported my many visits abroad, my constant travelling, and my tendency to work from wherever I happened to be. He never tried to force me into a particular way of doing research. Instead, he trusted me and allowed me to find my own way.

He also supported many of my ideas, requests, projects, and funding applications, including some that probably looked unrealistic at first. Yet he always encouraged me to try. Having someone who was willing to believe in me and support my ideas made a huge difference throughout these years.

His presence in my academic life started long before I became a university student and continued throughout my entire PhD. I owe him a great deal, and it is difficult to put into words how much his trust and support have meant to me.

Without his guidance, trust, and support, I would not be the researcher, or even the person, I am today, and this journey would have been very different.

Abstract

Efficient string matching is essential in various computational fields, including text retrieval, bioinformatics, and cybersecurity. While classical online and offline matching algorithms provide efficient solutions, they often suffer from limitations in either search time or memory consumption. Offline indexing techniques such as suffix trees and arrays enable fast queries but require excessive storage, making them impractical for large-scale datasets. To address these limitations, sampled string matching has emerged as a space-efficient alternative, reducing memory usage while preserving search efficiency.

This thesis explores advancements in sampled string matching, focusing on character distance sampling (CDS), a method that records relative character distances instead of absolute positions to Text Sampling for Efficient Information Processing space requirements. While CDS improves efficiency, it struggles with small alphabets and indexing overhead. To overcome these issues, this work introduces character context sampling (CCS), an enhancement that leverages local character distributions to refine sampling and improve search accuracy. Additionally, a set cover-based optimization is proposed to systematically select pivot characters, further reducing index size without compromising performance.

Experimental results demonstrate that these novel sampling techniques significantly outperform existing methods in both space efficiency and query performance. By bridging the gap between traditional indexing and online search, the proposed approaches provide a scalable and effective solution for pattern matching in large and memory-constrained environments. This work contributes to the development of more efficient text search systems, particularly in big data and bioinformatics applications.

Riassunto

L'efficiente ricerca di stringhe è fondamentale in diversi ambiti computazionali, tra cui il recupero di testo, la bioinformatica e la cybersicurezza. Sebbene gli algoritmi classici di ricerca online e offline offrano soluzioni efficienti, essi presentano spesso limitazioni in termini di tempo di ricerca o consumo di memoria. Le tecniche di indicizzazione offline, come gli alberi e gli array dei suffissi, consentono interrogazioni rapide ma richiedono un'elevata quantità di memoria, rendendole impraticabili per dataset su larga scala. Per superare queste limitazioni, la ricerca di stringhe campionata è emersa come un'alternativa efficiente in termini di spazio, riducendo l'uso della memoria senza compromettere le prestazioni di ricerca.

Questa tesi esplora i progressi nella ricerca di stringhe campionata, concentrandosi sul Character Distance Sampling (CDS), un metodo che registra le distanze relative tra i caratteri anziché le posizioni assolute per minimizzare i requisiti di spazio. Sebbene il CDS migliori l'efficienza, presenta difficoltà con alfabeti di piccole dimensioni e un'elevato sovraccarico nell'indicizzazione. Per affrontare questi problemi, questo lavoro introduce il Character Context Sampling (CCS), un miglioramento che sfrutta la distribuzione locale dei caratteri per affinare il campionamento e migliorare l'accuratezza della ricerca. Inoltre, viene proposta un'ottimizzazione basata sulla copertura degli insiemi (set cover) per selezionare in modo sistematico i caratteri pivot, riducendo ulteriormente la dimensione dell'indice senza compromettere le prestazioni. I risultati sperimentali dimostrano che queste tecniche di campionamento superano i metodi esistenti sia in termini di efficienza dello spazio che di prestazioni delle interrogazioni. Colmando il divario tra l'indicizzazione tradizionale e la ricerca online, gli approcci proposti offrono una soluzione scalabile ed efficace per il riconoscimento di pattern in ambienti su larga scala e con vincoli di memoria. Questo lavoro contribuisce allo sviluppo di sistemi di ricerca testuale più efficienti, con particolare applicazione nei contesti del big data e della bioinformatica.

Resumé

La recherche de chaînes est essentielle dans divers domaines informatiques, notamment la récupération de texte, la bioinformatique et la cybersécurité. Bien que les algorithmes classiques de recherche en ligne et hors ligne offrent des solutions performantes, ils présentent souvent des limitations en termes de temps de recherche ou de mémoire. Les techniques d'indexation hors ligne, permettent des requêtes rapides mais nécessitent un stockage considérable, les rendant impraticables pour des ensembles de données à grande échelle. Pour surmonter ces limites, la recherche de chaînes échantillonnée a émergé comme une alternative efficace en termes d'espace, réduisant l'utilisation de la mémoire tout en préservant l'efficacité de la recherche. Cette thèse explore les avancées en matière de recherche de chaînes échantillonnée, en se concentrant sur le Character Distance Sampling (CDS), une méthode qui enregistre les distances relatives entre les caractères plutôt que leurs positions absolues afin de minimiser les besoins en espace. Bien que le CDS améliore l'efficacité, il rencontre des difficultés avec les alphabets de petite taille et un coût élevé d'indexation. Pour remédier à ces problèmes, ce travail introduit le Character Context Sampling (CCS), une amélioration qui exploite la distribution locale des caractères pour affiner l'échantillonnage et améliorer la précision de la recherche. De plus, une optimisation basée sur la couverture d'ensemble est proposée afin de sélectionner de manière systématique les caractères pivot, réduisant ainsi la taille de l'index sans compromettre les performances. Les résultats expérimentaux montrent que ces techniques surpassent les méthodes existantes, tant en termes d'efficacité spatiale que de performance. En comblant l'écart entre l'indexation traditionnelle et la recherche en ligne, les approches proposées offrent une solution évolutive et efficace pour la recherche de motifs dans des environnements de grande taille et contraints en mémoire. Ce travail contribue au développement de systèmes de recherche textuelle plus performants, en particulier pour les applications en big data et en bioinformatique.

Table of Contents

1	Introduction	1
1.1	Contributions to this Thesis	3
1.1.1	Conference Proceedings	3
1.1.2	Journal Articles	5
1.2	Experimental Setting	6
2	Notions and Definitions	8
2.1	Suffix Arrays	9
2.2	Order Preserving Pattern Matching	10
2.3	Cartesian Tree Pattern Matching	11
3	Sampled String Matching	13
4	Minimizer Sampling	16
4.1	Fundamentals of Minimizer Sampling	16
4.2	Variants and Extensions of Minimizer Sampling	18
4.3	Applications	20
4.4	Summary and Perspectives	21
5	The Occurrence Text Sampling Algorithm	23
5.1	Online Searching	23
5.2	Offline Version of OTS	25
6	Character Distance Sampling	28
6.1	The Searching Phase	30
6.1.1	Case 1: $m_c = 0$	31
6.1.2	Case 2: $m_c = 1$	32
6.1.3	Case 3: $m_c \geq 2$	34
6.2	Offline Indexing with CDS	36
6.3	Condensed Alphabet Sampling	37
7	Online String Matching Algorithms	41
7.1	Horspool Algorithm	41
7.2	Quick-Search (QS) Algorithm	42
7.3	WFR q : Weak Factor Recognition with q -Grams	43
7.4	FJS	44
7.5	Experimental Results	45

8 Fake Sampling Representation	47
8.1 Spatial Considerations in Text Sampling	50
8.2 Search Phase in Sampling-Based Searching	53
8.3 Experimental Evaluation	55
9 Character Context Sampling	60
9.1 Impact on the number of verifications	63
9.2 Analysis of the space consumption.	64
10 Computing Periods and Covers Using Character Distance Sampling	65
10.1 Classical Computation of the Period of a String	65
10.2 Classical Computation of the Shortest Cover of a String	67
10.3 Computing the period of a string from its CDS representation	68
10.4 Computing the shortest cover of a string from its CDS representation	72
10.5 Experimental results	74
11 Optimizing Alphabet Sampling Using Set Cover	76
11.1 Mapping Pivot Selection to the Set Cover Problem	77
11.2 Optimizing Search Efficiency with Pivot-Free Patterns	79
11.3 Enhancing Space Efficiency through Distance Constraints	79
11.4 Extending Set Coverage for Condensed Alphabets	80
11.4.1 Complexity Issues	82
11.5 On the Space Complexity	82
11.6 On the Construction of the Sampled Alphabet	83
11.7 Experimental Results	84
12 Monotonic Run Length Scaling	88
12.1 On the Average Length of a Monotonic Run	91
12.2 Searching Using Monotonic Run Length Scaling	95
12.3 Monotonic Run Length Sampling	98
12.3.1 Searching Using Monotonic Run Length Sampling	100
12.4 Experimental Evaluation	101
12.4.1 Space Consumption	102
12.4.2 Running Times for the OPPM problem	102
12.4.3 Running Times for the ESM problem	106
12.4.4 Running Times for the CTPM problem	107
13 Parameterized String Matching through Sampling	110
13.1 A Text Sampling Solution to Parameterized Matching	113
13.2 Experimental Results	115
14 Conclusions	119
14.1 Future Directions	119

Bibliography

121

1

Introduction

String matching is a fundamental problem in computer science [1], playing a critical role in various domains such as natural language processing, information retrieval, bioinformatics, and data compression. The central challenge is to efficiently locate all occurrences of a pattern x of length m within a text y of length n , where both sequences are drawn from an alphabet Σ of size σ . Over the years, numerous algorithms have been developed to address this problem, each balancing computational efficiency, memory usage, and preprocessing requirements.

Traditionally, string matching algorithms are classified into two categories: counting and reporting. Counting algorithms focus solely on determining the number of times a pattern appears in a given text, making them well-suited for frequency analysis and statistical applications. Reporting algorithms, on the other hand, provide the exact positions of all occurrences, making them indispensable in text editors, search engines, and DNA sequence analysis. While counting algorithms tend to be more efficient in terms of both time and space complexity, reporting algorithms offer richer information at the cost of additional computational overhead.

Two fundamental paradigms have shaped the field of string matching: online matching and offline matching. Online algorithms, such as the Knuth-Morris-Pratt algorithm [2], process the text in real time with a worst-case complexity of $O(n)$, ensuring immediate responsiveness. More sophisticated heuristics like the Boyer-Moore algorithm [3] further optimize performance by allowing the search to skip portions of the text, leading to significant practical speedups. Offline approaches, in contrast, preprocess the text to construct an index, which allows for extremely fast queries at the cost of additional space requirements. Notable examples include suffix trees [4], suffix arrays [5] and FM-index [6], which facilitate rapid pattern retrieval by storing structured representations of the text. However, despite their efficiency, these methods suffer from high memory consumption, often requiring space several times larger than the original text itself.

To address the space constraints of full-index solutions, sampled string matching has emerged as a compelling alternative. Originally introduced by Vishkin [7], this approach selectively stores a subset of the text rather than constructing a complete index. By reducing the storage footprint while maintaining search efficiency, sampled string matching provides a balanced trade-off between space complexity and

computational performance. Once a sampled index is built, any standard online string matching algorithm can be applied directly to it, with only a minor verification phase required [8].

A significant advancement in this direction was the Optimal Text Sampling (OTS) method introduced by Claude et al. [9], which employs alphabet reduction techniques to create a compact sampled representation. Their approach incurs an additional space overhead of only 14% while achieving search speeds up to five times faster than traditional online algorithms on English texts. Furthermore, they extended their method by integrating an indexed version of the sampled text, adapting suffix arrays to store sampled positions instead of the full text.

Among the most successful innovations in sampled string matching is Character Distance Sampling (CDS) [10], a technique that records distances between selected pivot characters rather than absolute positions. This method was originally designed for online search applications and has been shown to achieve exceptional search speeds while keeping the index compact, often requiring less than 5% of the space needed for a full-text representation.

Beyond its initial application, CDS has been extended to offline string matching [11, 12], significantly reducing query times while maintaining a small memory footprint. In particular, experiments have shown that CDS-based approaches can achieve search time improvements of up to 91% over traditional index-based solutions, using only 15% of the storage required for full suffix arrays. Such improvements make CDS highly attractive for large-scale text processing in environments where computational resources are constrained.

Despite its advantages, sampled string matching methods such as OTS and CDS struggle when applied to texts with small alphabets, such as DNA sequences in bioinformatics. To address this limitation, Faro et al. [13] introduced a q-gram-based technique that artificially expands the alphabet size, making sampling techniques viable even in low-entropy domains. By leveraging this enhancement, sampled string matching has been successfully extended to genomic applications and other fields where traditional sampling approaches fail.

While CDS has proven effective, it relies on a distance representation that requires multiple bytes, making it challenging to store compactly. To overcome this limitation, a new technique called Fake Decomposition was recently introduced [14], enabling significant space reductions without sacrificing accuracy. This innovation further strengthens the feasibility of sampled string matching in resource-limited environments.

Recent developments in the field have also extended sampled string matching beyond classical exact matching. Techniques such as Monotonic Run-Length Scaling (MRLX) and Monotonic Run-Length Sampling (MRLS) [15] have been proposed for non-traditional string matching tasks, including Order Preserving Pattern Matching (OPPM) [16] and Cartesian Tree Pattern Matching (CTPM) [17]. These methods integrate run-length encoding principles into the sampling process, optimizing performance in structured and numerical data applications.

Another promising refinement is Character Context Sampling (CCS) [18], an extension of CDS that incorporates contextual information when selecting sampled characters. Unlike CDS, which records distances between isolated pivot characters, CCS enhances the process by considering local character distributions and their relationships within a predefined window. By capturing a richer set of features, CCS reduces false candidate occurrences and maintains a compact index, making it particularly effective in domains with skewed character distributions or repetitive structures. Empirical evaluations have demonstrated that CCS significantly enhances search efficiency while preserving a small index size, further solidifying its role as a cutting-edge sampled string matching technique.

Beyond pattern matching, the principles of sampled string matching have found utility in structural analysis of strings [19]. In particular, CDS has been successfully adapted for computing string periodicity and shortest covers, which are fundamental problems in text processing. The sampled representation enables significantly faster computations compared to classical approaches, making it a valuable tool for large-scale data analysis.

Recently, a new sampling framework has been proposed that leverages a Set Cover approach [20] to optimize the selection of pivot characters and improve the sampled text representation [21]. This innovation directly addresses a critical challenge in sampled matching: optimizing the trade-off between index size and search efficiency while ensuring robust performance across various pattern distributions. By refining the theoretical underpinnings of sampling techniques, this approach further cements the role of sampled string matching as a powerful and scalable solution for large-scale text retrieval.

This thesis explores these advancements in sampled string matching, building upon prior techniques while addressing their limitations. The contributions of this work seek to enhance the efficiency, adaptability, and practical applicability of sampled string matching algorithms, ultimately advancing the field of pattern matching in large-scale text analysis.

1.1 Contributions to this Thesis

Over the course of my PhD studies, I have contributed to several published works. While only those marked with ★ are directly included in this thesis, the remaining publications were an integral part of my doctoral training, reflecting the broader scope of my academic research and development.

1.1.1 Conference Proceedings

1. ★ Simone Faro and **Francesco Pio Marino**. Reducing time and space in indexed string matching by characters distance text sampling. *Prague Stringology Conference 2020*.

2. Simone Faro, **Francesco Pio Marino** and Stefano Scafiti: Fast-Insertion-Sort: a New Family of Efficient Variants of the Insertion-Sort Algorithm. *SOFSEM (Doctoral Student Research Forum) 2020*.
3. ★ Simone Faro, **Francesco Pio Marino**, and Arianna Pavone. Enhancing characters distance text sampling by condensed alphabets. *Italian Conference on Theoretical Computer Science 2021*.
4. ★ Simone Faro, **Francesco Pio Marino**, Arianna Pavone, and Antonio Scardace. Towards an efficient text sampling approach for exact and approximate matching. *Prague Stringology Conference 2021*.
5. ★ Simone Faro, **Francesco Pio Marino**, and Andrea Moschetto. Beyond Horspool: A comparative analysis in sampled matching. *Prague Stringology Conference 2024*.
6. ★ Simone Faro, **Francesco Pio Marino**, Andrea Moschetto, Arianna Pavone, and Antonio Scardace. The great textual hoax: Boosting sampled string matching with fake samples. *Fun with Algorithms 2024*.
7. ★ Simone Faro, Thierry Lecroq, **Francesco Pio Marino**, Arianna Pavone, and Stefano Scafiti. Improving sampled matching through character context sampling. *Italian Conference on Theoretical Computer Science, 2024*.
8. **Francesco Pio Marino**, Simone Faro and Antonio Scardace. Practical Implementation of a Quantum String Matching Algorithm. *Workshop on Quantum Search and Information Retrieval 2024*.
9. Domenico Cantone, Claudio Caudullo, Simone Faro, **Francesco Pio Marino**, Arianna Pavone and Caterina Viola. Practical Quantum Combinatorial String Matching. *Future Technologies Conference 2024*.
10. Simone Faro, **Francesco Pio Marino**. Scaling Grover's Search for Large Solution Spaces. *International Symposium on High-Performance Parallel and Distributed Computing, HPDC 2025*.
11. Simone Faro, **Francesco Pio Marino** and Gabriele Messina. Qutes: A High-Level Quantum Programming Language for Simplified Quantum Computing. *International Symposium on High-Performance Parallel and Distributed Computing, HPDC 2025*.
12. Simone Faro, Dominik Köppl and **Francesco Pio Marino**. Attractor Matching: A New Paradigm for Structural String Comparison. *Data Compression Conference 2026*.

13. Simone Faro, Dominik Köppl, Thierry Lecroq and **Francesco Pio Marino**. Enabling FM-Index for Elastic-Degenerate Strings via a new Min/Max Wavelet Tree. *Data Compression Conference 2026*.
14. Simone Faro, Dominik Köppl, Thierry Lecroq and **Francesco Pio Marino**. A Bitwise Approach to SCER Matching in Indeterminate Strings. *Combinatorial Pattern Matching 2026*.
15. ★ Thierry Lecroq, Arnaud Lefebvre and **Francesco Pio Marino**. Computation of Periods and Covers Using Sampling. *Prague Stringology Conference 2026*.
16. Simone Faro, Farida Farsian, **Francesco Pio Marino**, Gabriele Messina, Francesco Schillirò, Eva Sciacca and Fabio Vitello. Delta-Based Rare Pattern Discovery with Quantum Speedup *International Symposium on High-Performance Parallel and Distributed Computing 2026*.
17. Simone Faro, **Francesco Pio Marino** and Gabriele Messina. Structural Parallelism in Quantum Programs *International Symposium on High-Performance Parallel and Distributed Computing 2026*.
18. Simone Faro, **Francesco Pio Marino**, Arianna Pavone, Simone Spina and Caterina Viola. Quantum Bit-Parallel Swap Matching *International Symposium on High-Performance Parallel and Distributed Computing 2026*.
19. Simone Faro, **Francesco Pio Marino** and Gabriele Messina. Spells for Quantum Programmers: Expressive High-Level Commands in Qutes *Fun With Algorithms 2026*.
20. Simone Faro, **Francesco Pio Marino** and Arianna Pavone. Shallow Quantum Shuffling via Measurement-Driven Multi-Scale Rotations. *Proceedings of the 27th Italian Conference on Theoretical Computer Science, (ICTCS26)*.
21. Dominik Köppl and **Francesco Pio Marino**. Recompressing Compressed Data *Proceedings of the 27th Italian Conference on Theoretical Computer Science, (ICTCS26)*.

1.1.2 Journal Articles

22. ★ Simone Faro, **Francesco Pio Marino**, and Arianna Pavone. Efficient on-line string matching based on characters distance text sampling. *Algorithmica 2020*.
23. ★ Simone Faro, **Francesco Pio Marino**, and Arianna Pavone. Improved characters distance sampling for online and offline text searching. *Theoretical Computer Science 2023*.

24. Simone Faro, **Francesco Pio Marino**, and Gabriele Messina. Extending Qutes: A Practical High-Level Language for Quantum Computing. *The Computer Journal* 2026.
25. ★ Simone Faro, Thierry Lecroq and **Francesco Pio Marino**. Optimal Character Distance Sampling for Exact String Matching through Set Cover Reformulation. *IEEE Access* 2026.
26. **Francesco Pio Marino** and Jorma Tarhio. Improving String Matching through Enhanced Hashing *Software Practice and Experience* 2026
27. ★ Simone Faro, Thierry Lecroq, **Francesco Pio Marino**, Antonio Scardace and Arianna Pavone. Monotonic Run Length Text Sampling and Fake Sampled Positions. *The Computer Journal* 2026
28. ★ Simone Faro, Thierry Lecroq, **Francesco Pio Marino**, Arianna Pavone and Antonio Scardace. Parameterized String Matching by Text Sampling. *Mathematics* 2026

1.2 Experimental Setting

To ensure a rigorous and fair evaluation of the algorithms discussed in the following chapters, all implementations were developed in the C programming language and tested using a revised version of the SMART benchmarking tool [22], which doesn't make use of the shared memory. This tool provides a standardized framework for assessing the performance of string matching algorithms under various conditions, ensuring consistency across different implementations. The SMART tool is publicly available and widely used in the string matching research community, allowing for reproducibility and comparative analysis.¹

All experiments were conducted on a MacBook Pro equipped with a 2.7 GHz Intel Core i7 processor, featuring four physical cores and eight logical threads, 16 GB of LPDDR3 RAM clocked at 2133 MHz, 256 KB of L2 cache per core, and 8 MB of shared L3 cache. To maximize execution efficiency, all implementations were compiled using the `-O3` optimization flag, enabling aggressive compiler optimizations such as loop unrolling, inlining, and vectorization. These optimizations ensure that performance results are reflective of the best possible execution speeds under real-world conditions.

Performance evaluation was conducted based on search execution times, reported in milliseconds (ms). Each experiment was repeated multiple times to account for variability in execution due to system background processes and caching effects. The reported results correspond to the average execution time over multiple runs.

¹The SMART tool is available at <https://github.com/ostafen/smart/tree/develop>.

To provide a diverse and representative evaluation, the experiments utilized datasets sourced from the Pizza and Chili corpus [23], a widely recognized benchmark collection in the field of text indexing and compression. These datasets include a variety of natural language texts, DNA sequences, and other structured data, allowing for a comprehensive assessment of algorithm performance in real-world scenarios. Each dataset was loaded into memory in 100 MB chunks to ensure uniform testing conditions across different text sources.

In addition to real-world datasets, synthetic test cases were generated to analyze algorithm behavior under controlled conditions. Specifically, a set of $\text{Rand}\sigma$ synthetic text buffers was constructed, where each buffer consists of 100 MB of randomly generated characters drawn from a uniform distribution over an alphabet of size σ . Eight distinct $\text{Rand}\sigma$ buffers were created, corresponding to alphabet sizes $\sigma \in \{2, 4, 8, 16, 32, 64, 128, 256\}$. These synthetic datasets allow for an in-depth study of how alphabet size influences search efficiency, particularly in the context of sampled string matching methods, which rely on character distributions to optimize performance.

The evaluation methodology employed in this study follows standard practices in algorithm benchmarking, aiming to provide reproducible and interpretable results. By testing both real-world and synthetic datasets under controlled conditions, this experimental setting ensures that the conclusions drawn from the results accurately reflect the relative efficiency of the studied approaches.

2

Notions and Definitions

An *alphabet* is a finite set of elements called *characters*, *letters*, or *symbols*. A *string* is a sequence of zero or more symbols from an alphabet Σ of size σ . The string of zero symbols is denoted by ε . The set of all strings over the alphabet Σ is denoted by Σ^* . A string x of length n is represented by $x[0..n]$, where $x[i] \in \Sigma$ for $1 \leq i \leq n$. The character at position i is referred to as $x[i]$, while a substring spanning positions i to j is denoted by $x[i..j]$.

A string u is a *prefix* of x if there exists some $w \in \Sigma^*$ such that $x = uw$. Similarly, u is a *suffix* of x if $x = wu$ for some $w \in \Sigma^*$. A substring u of x is called a *border* if it is both a prefix and a suffix of x , meaning there exists non-empty strings v and w such that $x = uv = wu$.

A *finite state automaton* (FSA) is a tuple $A = (Q, q_0, F, \Sigma, \delta)$, where Q is the set of states, $q_0 \in Q$ is the initial state, $F \subseteq Q$ is the set of accepting states, Σ is the alphabet of characters labeling transitions, and $\delta : (Q \times \Sigma) \rightarrow Q$ is the transition function. If $\delta(q, c)$ is not defined for a state $q \in Q$ and a character $c \in \Sigma$, we say that $\delta(q, c) = \emptyset$. The transition function naturally extends to a final state function, $\delta^* : \Sigma^* \rightarrow Q$, defined recursively by $\delta^*(w) = \delta(\delta^*(w'), c)$, for each string $w = w'c$, where $w' \in \Sigma^*$ and $c \in \Sigma$.

The *exact string matching problem* consists of finding all occurrences of a pattern x of length m in a text y of length n . That is, finding all indices j such that $y[j + i] = x[i]$ holds for all $1 \leq i \leq m$. When the first character of the pattern, $x[1]$, is aligned with the character $y[s]$ in the text, such that the character $x[i]$ is aligned with $y[s + i]$ for $1 \leq i \leq m$, we say that the pattern x has shift s in y . In this case, the substring $y[s..s + m - 1]$ is called the *current window* of the text. If $y[s..s + m - 1] = x$, then the shift s is called a *valid shift*. Thus, the string matching problem can be rephrased as the problem of finding all valid shifts of a pattern x relative to a text y .

In string matching, algorithms can be broadly classified into *counting* and *reporting* types. Counting algorithms determine the total number of occurrences of a given pattern in a text, without explicitly providing their locations. These are useful in frequency analysis and applications where presence rather than position is of interest. Reporting algorithms, on the other hand, not only identify all occurrences but also return their exact positions. These algorithms are essential in applications such

as text editors, DNA sequence alignment, and plagiarism detection. While counting algorithms are typically more efficient in terms of time and space complexity, reporting algorithms provide richer information at the cost of additional computational overhead.

To facilitate efficient searching, a pattern x can be preprocessed to construct a *partial index* of the text y . In sampled string matching, rather than storing the entire text, a sampled version $\hat{y}$ is constructed based on a selected subset of characters. The sampled text $\hat{y}$ is typically much smaller than y , enabling faster searching while still preserving structural information necessary for pattern matching. The challenge in designing sampled string matching algorithms lies in selecting an appropriate sampling strategy that balances space efficiency and retrieval accuracy.

2.1 Suffix Arrays

In the following sections, we will also discuss various indexing solutions for string matching, frequently referring to suffix arrays as a fundamental baseline. To ensure a clear understanding, we first introduce the suffix array, a widely used data structure that enables efficient substring searching while maintaining a relatively compact representation compared to suffix trees.

A suffix array is a fundamental data structure in text indexing and pattern matching. It provides a space-efficient alternative to suffix trees while maintaining fast query performance, making it widely used in applications such as full-text search, bioinformatics, and data compression. Given a string T of length n , the suffix array SA is an array of integers that stores the starting positions of all suffixes of T in lexicographically sorted order. If $SA[i] = j$, it means that the suffix of T starting at position j is the i -th smallest suffix in lexicographical order.

For example, consider the string $T = \text{banana}$. Its suffixes and corresponding suffix array are:

Suffix	Index	Sorted Suffix	SA Index
banana	1	a	6
anana	2	ana	4
nana	3	anana	2
ana	4	banana	1
na	5	na	5
a	6	nana	3

Thus, the suffix array for T is:

$$SA = [5, 3, 1, 0, 4, 2]$$

Suffix arrays are typically constructed using efficient algorithms that improve upon the naive $O(n^2 \log n)$ sorting approach. The Manber-Myers algorithm [5] builds

suffix arrays in $O(n \log n)$ time using a prefix-doubling approach, while the SA-IS algorithm [24] achieves optimal $O(n)$ construction time, making it one of the most widely used methods in practice.

Once constructed, a suffix array allows for fast substring searches using binary search. Given a pattern P of length m , the suffix array enables a search for occurrences of P in $O(m \log n)$ time by locating the first and last occurrences of P within the lexicographically sorted suffixes. The efficiency of this approach makes suffix arrays a powerful tool for large-scale text searching, particularly when combined with the longest common prefix (LCP) array, which further optimizes query performances.

Despite their efficiency, suffix arrays require significant memory overhead, typically storing $4n$ bytes in standard implementations. This space requirement can be prohibitive for very large datasets, such as genomic sequences, where the text length can reach billions of characters. To address this limitation, compressed suffix arrays such as the FM-index [6] have been developed, significantly reducing storage while maintaining fast query times.

In this study, suffix arrays serve as a benchmark for evaluating space-efficient string matching techniques. While suffix arrays provide optimal query performance, their high space requirements motivate the exploration of alternative approaches, such as sampled string matching, which trades off some query speed for significantly lower memory consumption. By comparing the efficiency of sampled methods against suffix arrays, this work highlights the feasibility of space-conscious text indexing solutions in large-scale data retrieval applications.

2.2 Order Preserving Pattern Matching

Two strings u and v of the same length over Σ are said to be *order-equivalent*, written $u \approx v$, if

$$u[i] < u[j] \iff v[i] < v[j]$$

for all pairs of positions i, j on the strings. For example,

$$\langle 5, 2, 9, 4, 3 \rangle \approx \langle 6, 1, 7, 5, 2 \rangle,$$

which shows in particular their central value is the largest in both strings.

The order preserving pattern matching (OPPM) problem is naturally defined as follows: given a pattern $x \in \Sigma^*$ and a text $y \in \Sigma^*$, find all the substrings of y that are order-equivalent to x . For example, $\langle 5, 2, 9, 4, 3 \rangle$ occurs equivalently at position 1 on

$$\langle 4, \underline{6}, \underline{1}, \underline{7}, \underline{5}, 2, 9, 8, 3 \rangle$$

but nowhere else. For instance, it does not appear at position 5 because $5 < 8$, while in the pattern the corresponding values satisfy $5 > 4$.

We now introduce two functions that enable to check the order-equivalence between strings.

Definition 2.1. (Rank Function) Let x be a non-null sequence over a totally ordered alphabet $(\Sigma, \leq)$. For $i \in [1; |x|]$, let us set

$$\begin{aligned} x^<(i) &= \{k \mid x[k] < x[i]\}, \\ x^=(i) &= \{k \mid x[k] = x[i] \text{ and } k < i\}. \end{aligned}$$

Then, the rank function of x is the bijection $\text{rk}_x : [1; |x|] \rightarrow [1; |x|]$ defined by

$$\text{rk}_x(i) = |x^<(i)| + |x^=(i)|,$$

namely,

$$\text{rk}_x(i) = |\{k \mid x[k] < x[i] \text{ or } (x[k] = x[i] \text{ and } k < i)\}|,$$

for $0 \leq i < |x|$ and $0 \leq k < |x|$.

Thus, $\text{rk}_x(i)$ is the rank of $x[i]$ in the sequence x , namely the number of items in x strictly less than $x[i]$ plus the number of items in x equal to $x[i]$ and appearing in x before $x[i]$; hence the rank function rk_x is clearly bijective.

Definition 2.2. (Equality Function) Let x be a sequence of length $m \geq 2$ over a totally ordered alphabet $(\Sigma, \leq)$. The equality function of x is the binary map $\text{eq}_x : [1; m-1] \rightarrow \{0, 1\}$, where

$$\text{eq}_x(i) = \begin{cases} 1 & \text{if } x[\text{rk}_x^{-1}(i)] = x[\text{rk}_x^{-1}(i+1)] \\ 0 & \text{otherwise.} \end{cases}$$

for $1 \leq i \leq m-1$.

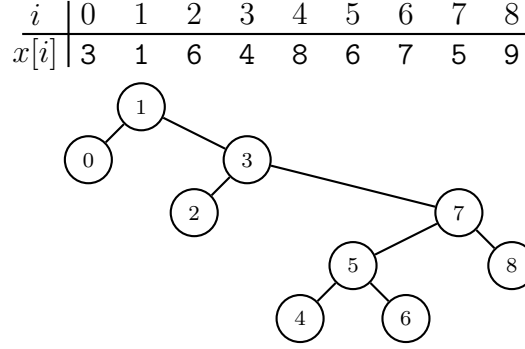
The rank and equality functions allow a complete characterization of order-equivalent [25].

2.3 Cartesian Tree Pattern Matching

Let x be a string of length m . The Cartesian Tree $\mathcal{CT}(x)$ of x is the binary tree where:

- the root corresponds to the index i of the minimal element of x (if there are several occurrences of the minimal element, the leftmost one is chosen);
- the left subtree of the root corresponds to the Cartesian Tree of $x[0..i-1]$;
- the right subtree of the root corresponds to the Cartesian Tree of $x[i+1..m-1]$.

In Fig. 2.1 the Cartesian tree of $x = \langle 3, 1, 6, 4, 8, 6, 7, 5, 9 \rangle$ is depicted.

Figure 2.1: Cartesian tree of string $x = \langle 3, 1, 6, 4, 8, 6, 7, 5, 9 \rangle$

The Cartesian tree pattern matching (CTPM) problem is naturally defined as follows: given a pattern $x \in \Sigma^*$ of length m and a text $y \in \Sigma^*$ of length n , find all the substrings of y that have the same Cartesian tree as x . More formally, find all j such that $0 \leq j \leq n - m$ and $\mathcal{CT}(x) = \mathcal{CT}(y[j..j + m - 1])$.

For example, if

$$x = \langle 3, 1, 6, 4, 8, 6, 7, 5, 9 \rangle$$

and

$$y = \langle 10, 12, 16, 15, 6, 14, 9, 12, 11, 14, 9, 17, 12, 10, 12 \rangle$$

then $\mathcal{CT}(x) = \mathcal{CT}(y[4..12])$ and this is the only position where it occurs.

The Parent-Distance representation of x is an array PD_x , which is defined as follows:

$$PD_x[i] = \begin{cases} i - \max_{0 \leq j < i} \{j \mid x[j] \leq x[i]\} & \text{if such } j \text{ exists} \\ 0 & \text{otherwise.} \end{cases}$$

The Parent-Distance representation has a one-to-one mapping to the Cartesian tree. [17]

Below is the Parent-Distance representation of $x = \langle 3, 1, 6, 4, 8, 6, 7, 5, 9 \rangle$.

i	0	1	2	3	4	5	6	7	8
$x[i]$	3	1	6	4	8	6	7	5	9
$PD_x[i]$	0	0	1	2	1	2	1	4	1

The Parent-Distance representation of a string x of length m can be computed in $\mathcal{O}(m)$ time using an algorithm similar to the one for building the Cartesian tree of x (see [26]).

Sampled String Matching

Sampled string matching aims to efficiently locate all occurrences of a given pattern x of length m within a text y of length n by leveraging a partial-index that enables fast and memory-efficient search operations. Unlike full-text indexes, which store exhaustive information about the text, sampled string matching constructs a compact representation by selectively sampling portions of the text. This approach significantly reduces storage overhead while maintaining efficient query performance, making it particularly valuable for large-scale text searching.

One of the key advantages of sampled string matching is its ability to balance space efficiency and search speed. By storing only the most relevant information, sampled indexes facilitate rapid lookups while requiring substantially less memory than traditional full-index structures such as suffix arrays and suffix trees. Furthermore, different sampling strategies can be employed depending on the nature of the text and the specific query requirements, allowing for adaptability across various applications. Given the exponential growth in data volume, optimizing both storage and query efficiency is critical for applications in natural language processing, bioinformatics, data mining, and large-scale information retrieval.

For a sampled string matching technique to be practically and theoretically effective, the partial-index should satisfy the following properties:

1. **Succinct representation:** Since the partial-index must be stored alongside the original text, it should be designed to occupy minimal additional space while maintaining its effectiveness. Efficient space usage is particularly important in large-scale applications where memory constraints are a limiting factor. By carefully selecting which portions of the text to sample, it is possible to construct an index that remains lightweight yet powerful enough to support efficient querying. This is crucial in constrained environments such as embedded systems, real-time processing units, and mobile computing, where excessive memory consumption can significantly degrade performance.
2. **Efficient construction:** The index should be built with minimal computational overhead, ensuring that preprocessing remains feasible even for large datasets. Constructing a full-text index, such as a suffix array or suffix tree, can be computationally expensive and impractical for massive datasets. A well-designed

sampled index should be created efficiently, allowing deployment in environments where rapid preprocessing is necessary, such as real-time search engines, dynamically updating databases, and large-scale genomic sequence analysis. The efficiency of the construction phase directly influences the usability of the method, as an overly expensive preprocessing step could make the technique impractical for real-world applications.

3. **Optimized search performance:** The primary goal of the partial-index is to significantly reduce search time, enhancing the efficiency of the underlying string matching algorithm. By selectively storing key portions of the text, sampled indexes allow queries to be resolved more quickly than naive search methods. The challenge lies in choosing sampling positions that maximize lookup efficiency while minimizing unnecessary comparisons. An optimized sampled index should achieve a near-optimal trade-off between sampling density and query time. The performance of the search depends not only on the effectiveness of the sampled index but also on the heuristics used for navigating the partial data structure efficiently.
4. **Robustness against worst-case scenarios:** The index should be designed to minimize the impact of cases where patterns lack pivots, ensuring consistent performance across different types of texts. Certain patterns may naturally align poorly with the sampled positions, leading to increased query times in worst-case scenarios. A well-designed sampled index incorporates strategies to mitigate such inefficiencies, such as adaptive sampling techniques that adjust based on observed textual distributions. This is especially relevant for diverse datasets where textual patterns vary significantly, such as multilingual corpora, noisy data sources, or highly repetitive sequences in DNA and protein databases.
5. **Adaptability to updates:** The index should allow for efficient modifications when the text undergoes changes, ideally supporting updates at a speed comparable to the modification process itself. Many real-world applications involve dynamic text that evolves over time, such as news articles, genetic sequences, or database records. A sampled index should be designed to support incremental updates efficiently, avoiding the need for complete re-indexing whenever minor modifications occur. Some modern sampling techniques incorporate structures that facilitate fast insertions and deletions, ensuring that updates do not degrade overall search performance.

Each of these properties plays a crucial role in ensuring that sampled string matching remains an effective solution for scalable and efficient text searching. The choice of sampling technique depends on the specific application and the nature of the text being processed. For example, applications requiring high-speed searches over vast datasets may benefit from distance-based sampling due to its predictable

structure, while those dealing with irregular textual distributions may achieve better results with alphabet-reduction sampling. The ongoing development of hybrid approaches, which combine multiple sampling techniques, continues to enhance the efficiency and adaptability of sampled string matching in various domains.

Overall, sampled string matching offers a compelling alternative to traditional full-text indexing techniques by striking a balance between storage efficiency and query performance. As research advances, further optimizations in sampling strategies and indexing structures will refine this approach, ensuring its applicability across an even wider range of text-searching scenarios. With continuous improvements in hardware and algorithmic techniques, sampled string matching is expected to play an increasingly vital role in large-scale information retrieval, automated data processing, and high-performance computing. Continued research in this field will likely lead to even more sophisticated and efficient sampling mechanisms, further expanding the practical applications of this powerful indexing technique.

Minimizer Sampling

In the era of large-scale sequence analysis and massive text collections, efficient sampling techniques are crucial to reduce redundancy while preserving essential matching information. Among these, *minimizer sampling* has emerged as a cornerstone method, originally proposed for biological sequence comparison [27], and subsequently extended to many domains, including pangenomic indexing [28] and compressed text indexing.

Minimizer sampling differs from traditional pivot-based or random sampling strategies by operating over *sliding windows* of substrings and selecting the smallest element according to a predefined order. This design ensures that any sufficiently long common subsequence between two texts will almost surely share at least one sampled element, providing strong guarantees for downstream applications.

4.1 Fundamentals of Minimizer Sampling

Minimizer sampling operates by selecting representative substrings from a larger text, ensuring that long common substrings between two texts are not missed despite aggressive sparsification. The key idea is that, by focusing on local minima within sliding windows, one can preserve meaningful matches while drastically reducing the amount of data that must be stored or indexed.

Let y be a text of length n over an alphabet Σ . Fix integers $k \geq 1$ (k-mer size) and $w \geq 1$ (window size). For each starting position i in $[1, n - (w + k - 1)]$, we consider the substring $y[i..i + w + k - 2]$ of length $\ell = w + k - 1$, which covers w overlapping k-mers. Among these w candidates, we select the lexicographically smallest k-mer as the *minimizer* for that window. In case of ties, the earliest occurrence is preferred.

Formally, if we denote by $\mathcal{K}_i = \{y[j..j + k - 1] \mid j \in [i, i + w - 1]\}$ the set of k-mers starting within the window, the minimizer at window i is defined as:

$$m(i) = \arg \min_{j \in [i, i + w - 1]} y[j..j + k - 1].$$

This approach ensures that substrings of sufficient length — namely, at least $w + k - 1$ characters — must share at least one sampled minimizer. This property

is crucial for applications such as read mapping and sequence comparison, where detecting long exact or near-exact matches is essential.

Example 4.1. Consider the text $y = \text{GATTACA}$, with parameters $k = 3$ and $w = 2$. The sequence of k -mers is:

GAT, ATT, TTA, TAC, ACA.

Sliding a window of two consecutive k -mers, we obtain:

- Window GAT, ATT: minimizer is ATT;
- Window ATT, TTA: minimizer remains ATT;
- Window TTA, TAC: minimizer becomes TAC;
- Window TAC, ACA: minimizer is ACA.

Thus, the selected minimizers are $\{\text{ATT}, \text{TAC}, \text{ACA}\}$.

We say that a k -mer is a (w, k) -minimizer for the string y if it is a minimizer for some window of w consecutive k -mers containing it. For simplicity we often refer to a (w, k) -minimizer simply as a minimizer. A more detailed illustration is provided in Table 4.1, where minimizers are explicitly highlighted for different settings of k and w .

Position	1	2	3	4	5	6	7	1	2	3	4	5	6	7	8	9	10	11	12
Sequence	2	3	1	0	3	4	3	4	2	6	4	7	2	8	1	4	7	5	1
k -mers	2	3	1					4	2	6	4	7	2	8					
			3	1	0					2	6	4	7	2	8	1			
				1	0	3					6	4	7	2	8	1	4		
					0	3	4					4	7	2	8	1	4	7	
						3	4	3					7	2	8	1	4	7	5
(a)								(b)					2	8	1	4	7	5	1

Table 4.1: Illustration of all k -mers in two windows of the sequence as well as their minimizers. The sequence positions and symbols within the window are listed in the first two rows. The adjacent k -mers are listed in the subsequent rows. The minimizer in each window is highlighted in bold. Note that w adjacent k -mers correspond to a window of length $l = w + k - 1$ letters. (a) Choosing the $(5, 3)$ -minimizer from 5 adjacent 3-mers ($w = 5$, $k = 3$, $l = 7$). (b) Choosing the $(6, 7)$ -minimizer from 6 adjacent 7-mers ($w = 6$, $k = 7$, $l = 12$).

From a theoretical perspective, the expected fraction of sampled minimizers can be estimated as approximately $2/(w + 1)$ [29]. This estimation derives from the

observation that a minimizer persists across roughly $w/2$ window shifts before being replaced by a new minimum. Therefore, larger window sizes result in sparser sampling, providing a tunable parameter to balance sensitivity and storage requirements.

Importantly, minimizer sampling guarantees *full coverage* of sufficiently long substrings. Specifically, if two sequences share an identical substring of length at least $w + k - 1$, they are guaranteed to have at least one matching minimizer. This property underpins their use in approximate matching algorithms, ensuring that no long matches are inadvertently missed due to sampling sparsity.

Efficient Computation. While the minimizer in each window could be determined by naïvely scanning all w k-mers, this approach would require $\mathcal{O}(nw)$ time overall, which becomes prohibitive for large-scale data.

An efficient method instead maintains a double-ended queue (deque) to track candidate minimizers as the window slides forward: - As the window shifts right by one position, k-mers that fall out of the window are dequeued from the front. - New k-mers are appended at the back, and larger candidates that can no longer become minimizers are removed. - At any point, the minimizer is the element at the front of the deque.

This technique ensures that each element is inserted and deleted at most once, leading to an overall $\mathcal{O}(n)$ amortized time complexity.

Example 4.2. Suppose we slide a window of $w = 4$ over a sequence of k-mers. Initially, the deque contains all w candidates. As the window advances: if the exiting k-mer was the minimizer, we recompute among remaining candidates. Otherwise, we only consider whether the new entering k-mer replaces the minimizer. This selective maintenance keeps operations per step constant on average.

Efficient minimizer extraction via sliding windows is fundamental for their practical use in large-scale applications such as compressed indexing, read overlap detection, and sequence search, where billions of k-mers must be processed rapidly.

Thus, the concept of minimizers, despite its apparent simplicity, encapsulates a powerful sampling mechanism capable of drastically reducing computational burden without sacrificing essential information needed for similarity detection and indexing.

4.2 Variants and Extensions of Minimizer Sampling

While classical minimizer sampling has proven highly effective, several variants have been proposed to address specific limitations such as bias in selection, non-uniform sampling density, and robustness to sequence variation. These extensions refine

the original idea by introducing randomized selection, structural constraints, and composite seeding strategies.

Randomized Minimizers. A natural limitation of lexicographic minimizers is their deterministic bias: in sequences with skewed nucleotide distributions (e.g., genomes rich in A/T content), certain k-mers are systematically favored, leading to uneven sampling. To overcome this, *randomized minimizers* apply a random hash function to each k-mer and select the one with the smallest hash value within the window. This randomized ordering spreads sampling more uniformly across the text, reducing redundancy and enhancing coverage in non-uniform or repetitive regions. Randomized minimizers maintain the essential property that substrings of sufficient length share sampled seeds with high probability, while improving the spatial regularity of sampled positions.

Weighted Minimizers. An alternative refinement is introduced by *weighted minimizers* [30], where each k-mer is assigned a dynamic weight based on its frequency or information content. Rare k-mers are given lower weights, making them more likely to be selected as minimizers. This approach mitigates the oversampling of common motifs — a typical issue in highly repetitive genomes — and leads to more diverse and informative sampling. Weighted minimizers can be especially beneficial in metagenomics, where detecting rare species or strain variants depends on distinguishing unique, low-frequency patterns.

Syncmers. Syncmers [31] further refine sampling regularity by imposing internal structural constraints on k-mers. Instead of minimizing over a window, a syncmer is defined by the position of a minimal nucleotide within the k-mer itself. Specifically, a syncmer selects a k-mer if a designated position (or subset of positions) satisfies a minimality criterion. This mechanism ensures that selected k-mers are spaced at more regular intervals compared to classical minimizers, reducing positional clustering and improving robustness to small insertions or deletions. Syncmers thus offer a compelling trade-off between sparsity and uniformity, particularly valuable for alignment-free sequence comparisons.

Strobemers. The concept of *strobemers* was introduced by Sahlin [32] as an alternative to classical k-mers and minimizers for sequence comparison tasks. Instead of selecting a single k-mer within a window, strobemers link together multiple k-mers sampled from different positions in the text, forming a composite seed that spans a larger genomic region. Two principal variants were proposed: *minstrobemes*, where secondary k-mers are chosen independently within a downstream window, and *randstrobemes*, where the choice of secondary k-mers depends on the identity of the first. This structure increases robustness against mutations, especially insertions and deletions, since disruption at a single location does not invalidate the entire strobemer. Additionally, strobemers provide better match distribution across the sequence compared to traditional k-mers, reducing redundancy and improving sensitivity in both highly similar and divergent datasets.

Mod-Minimizers. More recently, *mod-minimizers* [33] introduced a simple yet powerful two-phase sampling strategy. Instead of minimizing over all k -mers directly, mod-minimizers first identify the smallest t -mer (for small $t < k$) within a window, and then select a corresponding k -mer based on its relative position to this t -mer. This design achieves several advantages: it allows tuning the sampling density precisely, it reduces computational overhead by operating on smaller sub-units, and it approaches the theoretical limit of $1/w$ sampling density for large k . In particular, mod-minimizers demonstrate better uniformity and theoretical efficiency than standard minimizers when applied to random or low-complexity sequences.

Together, these variants enrich the conceptual landscape of minimizer-based sampling. They highlight the ongoing tension between sparsity, uniformity, robustness, and computational simplicity — key dimensions that researchers continue to optimize depending on application-specific requirements.

4.3 Applications

Minimizer sampling has become a fundamental building block in numerous large-scale applications, thanks to its ability to balance aggressive data reduction with strong guarantees of matchability. In bioinformatics, minimizers are widely employed to accelerate sequence mapping, genome comparison, and metagenomic classification.

A prominent example is Minimap2 [34], a state-of-the-art aligner for long sequencing reads. Minimap2 leverages minimizer sampling to efficiently identify candidate mapping regions before performing full alignments, dramatically reducing search space and runtime. The use of minimizers enables it to scale to gigabase-sized reference genomes and ultra-long Oxford Nanopore reads.

Similarly, Mash [35] uses a MinHash-based sketching method built upon minimizers to estimate genome and metagenome distances. By summarizing large genomes into compact sketches consisting of minimizers, Mash can quickly compute approximate similarity between entire genomes using only lightweight memory resources. This enables rapid comparison of thousands of genomes, a task otherwise computationally prohibitive.

Kraken [36] applies minimizer indexing for taxonomic classification of reads. Instead of aligning reads exhaustively, Kraken matches minimizers of reads to those of reference genomes, offering classification at high speed with reasonable memory usage. This methodology has proven particularly effective in metagenomics, where reads must be classified against vast and heterogeneous reference databases.

More recently, SPUMONI2 [28] has integrated minimizer digestion into pangenome indexing workflows. By digesting both the reference genomes and the input reads through a minimizer scheme, SPUMONI2 builds compressed indexes that scale sublinearly with respect to the total size of the data. This advancement enables

multi-class classification of both short and long reads across pangenomic references, offering significant improvements over traditional minimap2-based workflows.

Beyond bioinformatics, minimizer sampling has been successfully adopted in other domains involving large unstructured data. In document similarity detection, minimizers allow the construction of compact document fingerprints, enabling efficient near-duplicate detection across web-scale corpora [29]. In cybersecurity, malware detection systems employ minimizer-like sketches to represent executable files succinctly, supporting fast scanning and classification.

In the context of compressed text indexing, minimizers play a pivotal role by sparsifying input sequences before index construction. Techniques such as the r-index benefit from minimizer digestion by reducing alphabet complexity and total index size without losing essential matching information. This synergy enables scalable full-text indexing of massive sequence collections, including entire pangenomes and large-scale genomic databases.

Overall, the use of minimizer sampling across disciplines highlights its remarkable generality. By offering a scalable solution to the trade-off between memory, speed, and matching accuracy, minimizers have become indispensable in any application that requires fast and approximate matching over massive data collections.

4.4 Summary and Perspectives

Minimizer sampling represents a pivotal concept in the theory and practice of scalable sequence and text analysis. By selecting the locally minimal k-mer within a sliding window, it achieves an intrinsic trade-off between aggressive data reduction and guaranteed detectability of long common subsequences. Its operational simplicity belies its profound impact: minimizers enable sublinear-space indexing, fast approximate matching, and robust large-scale classification, across domains ranging from genome assembly to document fingerprinting.

The theoretical properties of minimizers — particularly their predictable sampling density and guaranteed coverage — provide strong analytical foundations for their deployment. Practical implementations, such as those in Minimap2, Mash, Kraken, and SPUMONI2, demonstrate that minimizer-based strategies can support real-world applications involving hundreds of gigabases or millions of documents, delivering speed and compression unattainable by exhaustive methods.

Nevertheless, classical minimizer sampling is not without weaknesses. The reliance on fixed ordering schemes induces sampling biases, leading to uneven coverage in non-uniform or highly repetitive sequences. Edge effects at the boundaries of strings can disrupt the matching guarantees unless additional sentinels are introduced. Furthermore, the stochastic persistence of minimizers across adjacent windows may lead to irregular spacing, undermining uniform seed distribution critical for some downstream algorithms.

The recent emergence of weighted minimizers, randomized schemes, syncmers,

strobemers, and mod-minimizers attests to the vitality of this research area. These methods aim to refine the spatial distribution of sampled positions, mitigate bias, and optimize robustness under sequence variation. In particular, strobemer-based constructions show promise for enhancing sensitivity in error-prone data, while mod-minimizers approach the information-theoretic lower bound for sparse sampling.

Future directions are rich and diverse. Adaptive minimizer strategies, wherein the window size or ordering dynamically adjusts to local sequence complexity, could further enhance performance, particularly in heterogeneous datasets. Hybrid models, combining pivot-based and window-based sampling, may yield schemes with both global control and local sensitivity. Moreover, formal theoretical analyses of minimizer behavior on structured and adversarial inputs — such as highly compressible or periodic strings — remain an open and compelling avenue of investigation.

In conclusion, minimizer sampling and its extensions stand at the heart of scalable data analysis. As the size and complexity of biological and textual datasets continue to grow, the development of smarter, more adaptive sampling strategies grounded in rigorous theory will be critical to sustaining progress across fields. It is expected that minimizers, and the ideas they have inspired, will remain central to the design of next-generation algorithms for large-scale sequence analysis.

5

The Occurrence Text Sampling Algorithm

Let y be the input text of length n , and let x be the input pattern of length m , both defined over an alphabet Σ of size σ . The *Occurrence Text Sampling* (OTS) algorithm selects a subset of the alphabet, denoted as $\hat{\Sigma} \subset \Sigma$ (the sampled alphabet), and constructs a partial index by extracting a sampled text $\hat{y}$ of length $\hat{n}$, which contains only characters from $\hat{\Sigma}$. Formally, each sampled character satisfies $\hat{y}[i] \in \hat{\Sigma}$ for all $1 \leq i \leq \hat{n}$.

a	b	r	a	c	a	d	a	b	r	a	m	a	g	i	c	a
	b	r		c		d		b	r		m		g	i	c	
	1							8						14		

Figure 5.1: A practical representation of OTS algorithm, omitting the only character a.

5.1 Online Searching

During the search phase, a sampled version of the pattern $\hat{x}$ of length $\hat{m}$ is constructed and searched in the sampled text $\hat{y}$. Since $\hat{y}$ contains only partial information about the original text, each candidate occurrence returned from the sampled search requires verification in the original text y . To efficiently map positions in $\hat{y}$ back to y , a position mapping table ρ is maintained. The mapping table ρ is constructed at regular intervals and stores corresponding positions in the original text.

The position mapping table ρ has size $\lfloor \hat{n}/q \rfloor$, where q is the interval factor, and is defined as:

$$\rho[i] = j \quad \text{if character } y[j] \text{ corresponds to } \hat{y}[q \times i].$$

The initial value is set as $\rho[0] = 0$. Authors suggest values of q in the set $\{8, 16, 32\}$ based on experimental evaluations [9].

For a candidate occurrence at position j , two cases arise. If j is stored in the mapping table, i.e., if $\rho[i] = j$ for some $1 \leq i \leq \lfloor \hat{n}/q \rfloor$, the algorithm directly verifies the full occurrence in y . Otherwise, if the sampled pattern is found at position r in $\hat{y}$, but r is not mapped in ρ , then verification is performed in the substring of the original text:

$$\rho[r/q] + (r \bmod q) - \alpha + 1$$

to

$$\rho[r/q + 1] - (q - (r \bmod q)) - \alpha + 1,$$

where α is the first position in x such that $x[\alpha] \in \hat{\Sigma}$.

If the input pattern does not contain any characters from the sampled alphabet, i.e., if $\hat{m} = 0$, the algorithm performs direct searching in the original text y .

Example 5.1. Consider the text $y = \text{"abaacabdaacaabcc"}$ of length 15, defined over the alphabet $\Sigma = \{a, b, c, d\}$. Suppose we select the sampled alphabet as $\hat{\Sigma} = \{b, c, d\}$, effectively omitting character a . The sampled text is:

$$\hat{y} = \text{"bcbdcbcc"}.$$

If we map every $q = 2$ positions in the sampled text, the position mapping table ρ is:

$$\langle 5, 8, 13, 15 \rangle.$$

To search for the pattern $x = \text{"acab"}$, the algorithm constructs the sampled pattern:

$$\hat{x} = \text{"cb"},$$

which is then searched in $\hat{y}$, yielding two occurrences at positions 2 and 5.

Since $\hat{y}[2]$ is mapped in ρ , verification starts at position 4 in y , confirming a match. However, $\hat{y}[5]$ is not mapped, so verification is required in the substring:

$$y[\rho(2) + 3 - 1 \dots \rho(3)].$$

No match is found in this case.

Since sampled patterns tend to be short, the authors implement the search phase using the Horspool algorithm, which performs well in this setting. Although experiments with alternative online algorithms have been conducted in other studies [8], Horspool remains a preferred choice due to its efficiency. A key challenge in OTS

is determining the optimal sampled alphabet. Based on analytical and experimental evaluations, the authors propose sampling the least frequent characters up to a certain threshold. Under this assumption, the algorithm achieves an extra space usage of only 14% of the text size and a search speedup of up to 5 times compared to standard online string matching on English texts.

5.2 Offline Version of OTS

An offline version of the Occurrence Text Sampling (OTS) algorithm has been developed to improve search efficiency by precomputing an indexed structure. In this variant, instead of performing direct online searches over the sampled text, the algorithm constructs an indexing structure, such as a suffix array or FM-index, on the sampled text $\hat{y}$, allowing for significantly faster retrieval of candidate positions.

This approach, similar to the sparse suffix array [37], indexes only a subset of suffixes, leading to different search algorithms and performance characteristics. More recently, Grabowski and Ranziewicz [38] introduced an improved suffix sampling approach, requiring only a minimal pattern length constraint for efficient searching.

By leveraging a sampled suffix array, which stores only a subset of suffixes corresponding to sampled positions, this approach enables efficient binary search operations, drastically reducing search time compared to direct scanning of $\hat{y}$. Additionally, this method supports range queries, allowing multiple occurrences of a pattern to be retrieved simultaneously.

A specific optimization proposed in previous work involves building a suffix array that stores only suffixes that start with a sampled character. This method ensures that the indexing process remains efficient while reducing unnecessary storage. However, as a consequence, this approach can only be used for patterns that contain at least one character from the sampled alphabet. The search phase is divided into two steps: first the algorithm searches for occurrences of the sampled suffix of the pattern using the sampled suffix array. Then each retrieved candidate occurrence is verified by checking the unsampled prefix of the pattern against the original text.

This two-step verification process helps maintain both efficiency and accuracy.

Example 5.2. Consider the text $y = \text{"abaacabdaacaabcc"}$ of length 15, defined over the alphabet $\Sigma = \{a, b, c, d\}$. Suppose we select the sampled alphabet as $\hat{\Sigma} = \{b, c, d\}$, effectively omitting character a . The sampled text, with the sentinel character "\$" appended, is:

$$\hat{y} = \text{"bcdbcbcc\$"}.$$

Instead of searching directly in $\hat{y}$, the offline version of OTS constructs a sampled suffix array while preserving the positions in the original text. The lexicographically sorted suffixes of $\hat{y}$, along with their original positions in y , are:

Position in y	Suffix
17	\$
2	bcbdcbcc\$
14	bcc\$
7	bdcbcc\$
16	c\$
11	cbcc\$
5	cbdcbcc\$
15	cc\$
8	dcbcc\$

The corresponding sampled suffix array $SA_{\hat{y}}$ stores only the positions of these suffixes in the original text:

$$SA_{\hat{y}} = \langle 17, 2, 14, 7, 16, 11, 5, 15, 8 \rangle.$$

To search for the pattern $x = \text{"acab"}$, we construct its sampled version:

$$\hat{x} = \text{"cb"}.$$

We also identify that the first sampled character in x appears at position $f = 1$ in the original pattern.

Using binary search in the sampled suffix array, we find two occurrences of "cb" in $SA_{\hat{y}}$ at positions 5 and 6 in $SA_{\hat{y}}$. The corresponding original text positions retrieved from $SA_{\hat{y}}$ are:

$$\langle SA_{\hat{y}}[6], SA_{\hat{y}}[7] \rangle = \langle 11, 5 \rangle.$$

Since we must verify the full occurrence in y , we need to adjust the positions by shifting left by f (discarded characters in x before the first sampled character). Thus, verification should be performed at positions:

$$SA_{\hat{y}}[i] - f.$$

Applying this adjustment:

$$\langle 11 - 1, 5 - 1 \rangle = \langle 10, 4 \rangle.$$

Thus, verification is performed at positions 4 and 10 in y . The occurrence at position 4 is confirmed as a match, while the occurrence at 10 does not yield a valid match.

This indexing-based approach is particularly effective for moderate to long patterns. Experimental evaluations on English texts suggest that optimal performance is achieved when the number of removed characters from the original alphabet ranges between 13 and 16. This highlights the trade-off between index size, search speed,

and pattern length, reinforcing the idea that the selection of sampled characters significantly impacts overall performance.

The offline version of OTS offers several advantages, including faster query times due to indexed lookups, reduced storage requirements compared to full-text suffix arrays and scalability for large datasets.

However, as with most offline indexing strategies, it comes with an increased pre-processing cost, making it more suitable for scenarios where the text remains static and multiple queries are performed. In contrast, frequent modifications to the text would require reconstructing the index, which can be computationally expensive.

6

Character Distance Sampling

Character Distance Sampling (CDS) is a text sampling technique that records the distances between consecutive occurrences of selected pivot characters in a string. This method provides an efficient trade-off between the high space complexity of full-text indexes and the slower performance of classical online string matching algorithms. By storing only the distances between selected characters rather than the characters themselves, CDS reduces storage requirements while maintaining structural information crucial for search efficiency.

As above, let y be the input text, of length n , and let x be the input pattern, of length m , both over the alphabet Σ of length σ . We assume that all strings can be treated as vectors starting at position 1. Thus we refer to $x[i]$ as the i -th character of the string x , for $1 \leq i \leq m$, where m is the length of x .

We first define the sampled alphabet $C \subseteq \Sigma$, which we call *set of pivot characters*. The set C could be very small, and in many practical cases could be reduced to a single character. if $|C| = 1$, the unique character of C is called the *pivot* character. For simplicity we will assume in some examples that $|C| = 1$, which can be trivially generalized to the case where $|C| > 1$.

The text sampling approach used in our solution is based on the following definition of *bounded position sampling*.

Definition 6.1 (The Bounded Position Sampling). Let y a text of length n , let $C \subseteq \Sigma$ be the set of pivot characters and let n_c be the number of occurrences of C in the input text y . First we define the *position function*, $\delta : \{1, \dots, n_c\} \rightarrow \{1, \dots, n\}$, where $\delta(i)$ denotes the position of the i -th occurrence of a pivot character in y . For convenience, equivalently, we write $\dot{y}[i] = \delta(i)$, so that $\dot{y}$ represents the sequence of positions of pivot characters within y . Formally, we have

- | | | |
|-------|--|----------------------------------|
| (i) | $1 \leq \dot{y}[i] < \delta(i+1) \leq n$ | for each $1 \leq i \leq n_c - 1$ |
| (ii) | $y[\dot{y}[i]] = c$ | for each $1 \leq i \leq n_c$ |
| (iii) | $y[\dot{y}[i] + 1.. \delta(i+1) - 1]$ does not contain C | for each $0 \leq i \leq n_c$ |

where, in (iii), we assume that $\delta(0) = 0$ and $\delta(n_c + 1) = n + 1$.

Example 6.2. Suppose $y = \text{"abaacbcabdada"}$ is a text of length 13, over the alphabet $\Sigma = \{a,b,c,d\}$ and let $C = \{a\}$ be the set of pivot characters. Thus

the position sampled version of y is $\dot{y} = \langle 1, 3, 4, 8, 11, 13 \rangle$. Specifically the first occurrence of character “a” is at position 1, its second occurrence is at position 3 and so on.

Definition 6.3 (The Characters Distance Sampling). Let $C \subseteq \Sigma$ be the set of pivot characters, let $n_c \leq n$ be the number of occurrences of the set of pivot characters in the text y and let δ be the position function of y . We define the *characters distance function* $\dot{y}[i] = \delta(i+1) - \dot{y}[i]$, for $1 \leq i \leq n_c - 1$, as the distance between two consecutive occurrences of any character $c \in C$ in y .

The *characters-distance sampled version* of the text y is a numeric sequence, indicated by $\bar{y}$, of length $n_c - 1$ defined as

$$\bar{y} = \langle \Delta(1), \Delta(2), \dots, \Delta(n_c - 1) \rangle. \quad (6.1)$$

Plainly we have

$$\sum_{i=1}^{n_c-1} \dot{y}[i] \leq n.$$

Example 6.4. As in Example 6.2, let $y = \text{“abaacbcabdada”}$ be a text of length 13, over the alphabet $\Sigma = \{a, b, c, d\}$. Let $C = \{a\}$ be the set of pivot characters. Thus the character distance sampled version of y is $\bar{y} = \langle 2, 1, 4, 3, 2 \rangle$. Specifically $\bar{y}[1] = \Delta(1) = \delta(1) - \delta(0) = 3 - 1 = 2$, while $\bar{y}[3] = \Delta(4) = \delta(4) - \delta(3) = 8 - 4 = 4$, and so on.

In theory, bounding all distances by 256 would allow storage within a single byte. However, this constraint is often impractical in real-world scenarios. Figure 6.1 presents the greatest and average distances for each character, sorted by rank, in an English text alphabet.

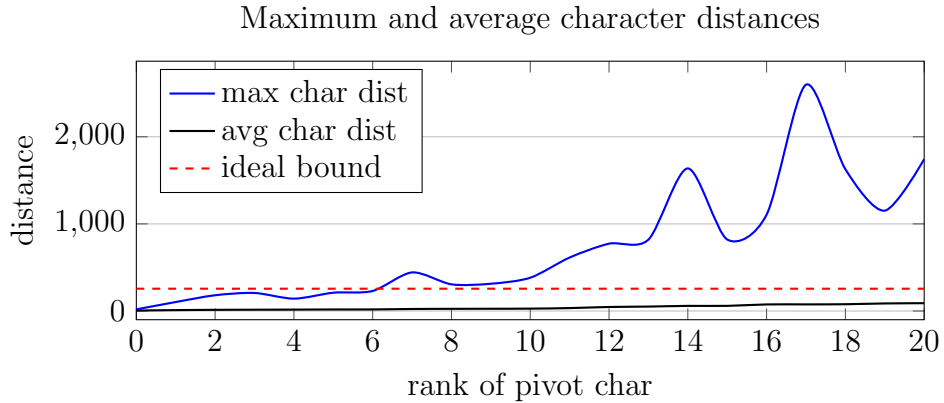


Figure 6.1: Maximum and average distances between two consecutive occurrences, computed for the most frequent characters in a natural language text. On the x axis characters are ordered on the base of their rank value, in a non decreasing order. The red line represents the ideal bound 256.

Thus, considering that also storing distances, four different bytes are required, from now on we will consider applications where the position-sampled version of the text $\dot{y}$ is stored, and the distances between consecutive occurrences are computed online.

We are now ready to describe the searching phase of our new proposed algorithm.

COMPUTE-DISTANCE-SAMPLING(y, n, C)	COMPUTE-POSITION-SAMPLING(y, n, C, k)
1. $\bar{y} \leftarrow \langle \rangle$	1. $\dot{y} \leftarrow \langle \rangle$
2. $j \leftarrow 0$	4. $j \leftarrow 0$
3. $p \leftarrow 0$	5. for $i \leftarrow 1$ to n do
4. for $i \leftarrow 1$ to n do	8. if $y[i] \in C$ then
5. if $y[i] \in C$ then	10. $\dot{y}[j] \leftarrow i$
7. $\bar{y}[j] \leftarrow i - p$	9. $j \leftarrow j + 1$
6. $j \leftarrow j + 1$	12. return $(\dot{y}, j)$
8. $p \leftarrow i$	
9. return $(\bar{y}, j)$	

Figure 6.2: (On the left) The pseudocode of procedure COMPUTE-DISTANCE-SAMPLING for the construction of the *character distance sampling* version of a text y . (On the right) The pseudocode of procedure COMPUTE-POSITION-SAMPLING for the construction of the *character position sampling* version of a text y .

6.1 The Searching Phase

Let x be an input pattern of length m and let $C \subseteq \Sigma$ be the set of pivot characters. Let m_c be the number of occurrences of the set of pivot characters in x . The searching phase can be then divided in three different subroutines, depending on the value of m_c . All searching procedures work using a filtering approach. They take advantage of the partial-index precomputed during the preprocessing phase in order to quickly locate any candidate substring of the text which may include an occurrence of the pattern.

If such candidate substring has length m the algorithm simply performs a character-by-character comparison between the pattern and the substring. Figure 6.3 shows the pseudocode of procedure VERIFY used by the algorithm for testing the occurrence of a pattern x , of length m , in a text y , of length n , starting at position s . Its complexity is plainly $O(m)$ in the worst case.

```

VERIFY( $x, m, y, s$ )
1.    $i \leftarrow 1$ 
2.   while  $i \leq m$  and  $x[i] = y[s + i]$  do  $i \leftarrow i + 1$ 
3.   if  $i > m$  then return TRUE
4.   return FALSE

```

Figure 6.3: The pseudocode of procedure VERIFY for testing the occurrence of a pattern x , of length m , in a text y , of length n , starting at position s .

Otherwise if the candidate substring has length greater than m , then a searching procedure is called, based on any exact online string matching algorithm. As we will discuss later, if we suppose that the underlying algorithm has a linear worst case time complexity, as in the case of the KMP algorithm [2], then our solution achieves the same complexity in the worst case. Similarly if we suppose to implement the searching procedure using an optimal average string matching algorithm, like the BDM algorithm [39], the resulting solution achieves an optimal $O(n \log_{\sigma} m/m)$ average time complexity.

In what follows we describe in details the three different searching procedures which are applied when $m_c = 0$, $m_c = 1$ and $m_c > 1$, respectively.

6.1.1 Case 1: $m_c = 0$

Consider firstly the case where the pattern does not contain any occurrence of the set of pivot characters C , so that m_c is equal to zero. Under this assumption the algorithm searches for the pattern x in all substrings of the original text which do not contain the set of pivot characters. Specifically such substrings are identified in the original text by the intervals $[\dot{y}[i] + 1.. \delta(i + 1) - 1]$, for each $0 \leq i \leq n_c$, assuming $\delta(0) = 0$ and $\delta(n_c + 1) = n + 1$.

Figure 6.4 shows the pseudocode of the algorithm which searches for all occurrences of a pattern x , when the set of pivot characters C does not occur in it. Specifically, for each $1 \leq i \leq n_c + 1$, the algorithm checks if the interval $\dot{y}[i] - \dot{y}[i - 1]$ is greater than m (lines 5-7). In such a case the algorithm searches for x in the substring of the text $y[\dot{y}[i - 1] + 1.. \dot{y}[i] - 1]$ (line 8) using any standard string matching algorithm. Otherwise the substring is skipped, since no occurrence of the pattern could be found at such position.

```

SEARCH-0( $x, \dot{y}, y$ )
 $\Delta$     We assume  $C$  does not occur in  $x$ 
1.       $m \leftarrow \text{len}(x)$ 
2.       $n_c \leftarrow \text{len}(\dot{y})$ 
3.       $b \leftarrow 1$ 
5.      for  $i \leftarrow 2$  to  $n_c$  do
7.          if  $(\dot{y}[i] - \dot{y}[i-1] > m)$  then
8.              search for  $x$  in  $y[\delta_{i-1} + 1.. \delta_i - 1]$ 
9.          if  $(n + 1 - \dot{y}[n_c] > m)$  then
10.             search for  $x$  in  $y[\dot{y}[n_c] + 1..n]$ 

```

Figure 6.4: The pseudocode of procedure SEARCH-0 for the sampled string matching problem, when the set of pivot characters does not occur in the input pattern x .

The following theorem proves that procedure SEARCH-0 achieves optimal worst-case and average-case time complexity.

Theorem 1. *Let x and y be two strings of size m and n , respectively, over an alphabet Σ of size $\sigma > 1$. Let $C \subseteq \Sigma$ be the set of pivot characters and let $\dot{y}$ be the position sampled version of the text y . Under the assumption of equiprobability and independence of characters in Σ , the worst-case and average time complexity of the SEARCH-0 algorithm are $O(n)$ and $O(n \log_\sigma m/m)$, respectively.*

Proof. In order to evaluate the worst-case time complexity of procedure SEARCH-0, we can notice that each substring of the text is scanned at least ones in line 10, with no overlap. Thus if we use a linear algorithm to perform the standard search then it is trivial to prove that the whole searching procedure requires

$$T_{\text{wst}}^0(n) = O(m) + \sum_{i=1}^{n_c-1} O(\dot{y}[i]) = O(n).$$

Assuming that the underlying algorithm has an $O(n \log m/m)$ average time complexity, on a text of length n and a pattern of length m , we can express the expected average time complexity as

$$T_{\text{avg}}^0(n) = \sum_{i=1}^{n_c-1} O\left(\frac{\dot{y}[i] \log_\sigma m}{m}\right) = O\left(\frac{n \log_\sigma m}{m}\right).$$

□

6.1.2 Case 2: $m_c = 1$

Suppose now the case where the pattern x contains a single occurrence of the set of pivot characters C , so that the sampled pattern is still empty. The algorithm

```

SEARCH-1( $x, \dot{y}, y$ )
1.    $m \leftarrow \text{len}(x)$ 
2.    $n_c \leftarrow \text{len}(\dot{y})$ 
3.    $\alpha \leftarrow \min\{i : x[i] = c\}$ 
4.    $b \leftarrow 1$ 
5.   for  $i \leftarrow 2$  to  $n_c$  do
6.       if  $(\dot{y}[i-1] - \dot{y}[i-2]) > \alpha - 1$  and  $\dot{y}[i] - \dot{y}[i-1] > m - \alpha$  then
7.           VERIFY( $x, m, y, \dot{y}[i-1] - \alpha + 1$ )

```

Figure 6.5: The pseudocode of procedure SEARCH-1 for the sampled string matching problem, when the set of pivot characters occurs ones in the input pattern x .

efficiently takes advantage of the informations precomputed in $\dot{y}$ using the positions of the set of pivot characters C in y as an anchor to locate all candidate occurrences of x .

Figure 6.5 shows the pseudocode of the algorithm which searches for all occurrences of a pattern x , when the set of pivot characters C occurs only once in it. Specifically, let α be the unique position in x which contains the set of pivot characters (line 3), i.e. we assume that $x[\alpha] = c$ and $x[1..\alpha-1]$ does not contain any characters in C . Then, for each $0 \leq i \leq n_c - 1$, the algorithm checks if the interval $\dot{y}[i-1] - \dot{y}[i-2]$ is greater than $\alpha - 1$ and if the interval $\dot{y}[i] - \dot{y}[i-1]$ is greater than $m - \alpha$ (lines 7-9). In such a case the algorithm merely checks if the substring of the text $y[\dot{y}[i] - \alpha + 1..\dot{y}[i] + m - \alpha]$ is equal to the pattern (line 10). Otherwise the substring is skipped. As before we assume that $\delta(0) = 0$ (line 5) and $\delta(n_c + 1) = n + 1$ (line 11). The last alignment of the pattern in the text is verified separately at the end of the main cycle (lines 11-12).

The following theorem proves that procedure SEARCH-1 achieves optimal worst-case and average-case time complexity.

Theorem 2. *Let x and y be two strings of size m and n , respectively, over an alphabet Σ of size $\sigma > 1$. Let $C \subseteq \Sigma$ be the set of pivot characters and let $\dot{y}$ be the position sampled version of the text y . Under the assumption of equiprobability and independence of characters in Σ , the worst-case and average time complexity of the SEARCH-1 algorithm are $O(n)$ and $O(n \log_\sigma m/m)$, respectively.*

Proof. In order to evaluate the worst-case time complexity of the algorithm notice that each character could be involved in, at most, two consecutive checks in line 10. Specifically any text position in the interval $[\dot{y}[i-1] + 1..\dot{y}[i] - 1]$ could be involved in the verification of the substrings $y[\dot{y}[i-1] - \alpha + 1..\dot{y}[i-1] + m - \alpha]$ and $y[\dot{y}[i] - \alpha + 1..\dot{y}[i] + m - \alpha]$. Thus the overall worst case time complexity of the searching phase is $T_{\text{wst}}^1(n) = O(n)$.

In order to evaluate the average-case time complexity of the algorithm notice that the expected number of occurrences in y of the set of pivot characters is given

by $\mathbf{E}(n_c) = n/\sigma$. Moreover, for any candidate occurrence of x in y , the number $\mathbf{E}(\text{insp})$ of expected character inspections performed by procedure `VERIFY`, when called on a pattern of length m , is given by

$$\mathbf{E}(\text{insp}) = 1 + \sum_{i=1}^{m-1} \left(\frac{1}{\sigma}\right)^i \leq \frac{\sigma}{\sigma-1}$$

Thus the average time complexity of the algorithm can be expressed by

$$\begin{aligned} T_{\text{avg}}^1(n) &= \mathbf{E}(n_c) \cdot \mathbf{E}(\text{insp}) = \\ &O\left(\frac{n}{\sigma}\right) \cdot O\left(\frac{\sigma}{\sigma-1}\right) = \\ &O\left(\frac{n}{\sigma-1}\right) \end{aligned}$$

obtaining the optimal average time complexity $O(n \log_{\sigma} m/m)$ for great enough alphabets of size $\sigma > (m/\log_{\sigma} m) + 1$. $\square$

6.1.3 Case 3: $m_c \geq 2$

If the occurrences of the set of pivot characters in x are more than 1 (i.e. if $m_c \geq 2$ and $\bar{m} \geq 1$) then the algorithm uses the sampled text $\bar{y}$ to compute on the fly the character-distance sampled version of y and use it to searching for any occurrence of $\bar{x}$. This is used as a filtering phase for locating in y any candidate occurrence of x .

Figure 6.6 shows the pseudocode of the algorithm which searches for all occurrences of a pattern x , when the set of pivot characters C occurs more than once in it. First the character distance sampled version of the pattern $\bar{x}$ is computed (line 5). Then the algorithm searches for $\bar{x}$ in the $\bar{y}$ using any exact online string matching algorithm (line 6).

For each occurrence position j of $\bar{x}$ in $\bar{y}$ an additional procedure must be run to check if such occurrence corresponds to a match of the whole pattern x in y (lines 7-9). For this purpose the algorithm checks if the substring of the text $y[\delta(j) - \alpha \dots \delta(j) - \alpha + m - 1]$ is equal to x , where α , as before, is the first position in x where the set of pivot characters occurs. Notice that the value of $\delta(j)$ can be obtained in constant time from $\bar{y}[j]$ (line 8).

```

SEARCH-2+( $x, \dot{y}, y$ )
 $\Delta$    We assume  $C$  occurs more than once in  $x$ 
1.     $m \leftarrow \text{len}(x)$ 
2.     $\dot{n} \leftarrow \text{len}(\dot{y})$ 
3.     $\alpha \leftarrow \min\{i : x[i] = c\}$ 
4.     $b \leftarrow 1$ 
5.     $(\bar{x}, \bar{m}) \leftarrow \text{COMPUTE-DISTANCE-SAMPLING}(x, m, \{c\})$ 
6.    search for  $\bar{x}$  in  $\bar{y}$  :
7.        for each  $i$  such that  $\bar{x} = \bar{y}[i..i + \bar{m} - 1]$  do
8.             $(\delta_i, b) \leftarrow \text{GET-POSITION}(\tau, b, i)$ 
9.             $\text{VERIFY}(x, m, y, \delta_i - \alpha)$ 

```

Figure 6.6: The pseudocode of procedure SEARCH-2 for the sampled string matching problem, when the set of pivot characters occurs more than ones in the input pattern x .

The following theorem proves that procedure SEARCH-2 achieves optimal worst-case and average-case time complexity.

Theorem 3. *Let x and y be two strings of size m and n , respectively, over an alphabet Σ of size $\sigma > 1$. Let $C \subseteq \Sigma$ be the set of pivot characters and let $\dot{y}$ be the position sampled version of the text y . Under the assumption of equiprobability and independence of characters in Σ , the worst-case and average time complexity of the SEARCH-2 algorithm are $O(n)$ and $O(n \log_\sigma m/m)$, respectively.*

Proof. In order to evaluate the worst-case time complexity of the algorithm in this last case notice that, if we use a linear algorithm to search $\bar{y}$ for $\bar{x}$, the overall time complexity of the searching phase is $O(n_c + n_x m)$, where n_x is the number of occurrences of $\bar{x}$ in $\bar{y}$. In the worst case it translates in $O(n_c m)$ worst case time complexity. However it is not difficult to suppose to implement procedure VERIFY based on a linear algorithm, as KMP, in order to remember all positions of the text which have been already verified, allowing the algorithm to run in overall $T_{\text{wst}}^{2+}(n) = O(n)$ worst-case time complexity.

In order to evaluate the average-case time complexity of the algorithm notice that time required for searching $\bar{x}$ in $\bar{y}$ is $O(n_c \log m_c / m_c)$. Moreover, observe that the number of verification is bounded by the expected number of occurrences of the set of pivot characters in y , thus, following the same line of Theorem 2, the overall average time complexity of the verifications phase is $O(n/(\sigma - 1))$. Thus the average time complexity of the algorithm can be expressed by

$$T_{\text{avg}}^{2+}(n) = O\left(\frac{n_c \log m_c}{m_c}\right) + O\left(\frac{n}{\sigma - 1}\right) = \frac{n \log_\sigma m}{m}$$

obtaining the optimal average time complexity $O(n \log_\sigma m/m)$. □

6.2 Offline Indexing with CDS

Character Distance Sampling (CDS) provides an efficient mechanism for text indexing by constructing a compact, distance-based representation of a given text. The offline variant of CDS enhances search efficiency by precomputing an index over this sampled text. Instead of executing direct searches in $\bar{y}$, this approach builds an index structure—such as a suffix array, FM-index, or wavelet tree—over the character-distance sampled representation. By precomputing an index, queries can be processed significantly faster compared to online searching.

The primary goal of offline CDS indexing is to enable rapid lookup of candidate positions in the text while maintaining storage efficiency. This is achieved through selecting a set of pivot characters C to construct the sampled representation, encoding the distances between consecutive occurrences of pivot characters, and indexing the resulting sequence using efficient data structures. A key challenge lies in ensuring that the sampled representation retains sufficient discriminative power for retrieval. The selection of pivot characters is crucial: it must balance reducing storage costs while maintaining search effectiveness. The distance-based encoding allows the index to remain compact, preserving the essential structure of the text while reducing redundancy.

To support efficient searches, different indexing techniques can be employed. A suffix array is a lexicographically sorted array of all suffixes of the sampled representation, allowing binary search for efficient pattern matching. The FM-index is a compressed suffix array combined with the Burrows-Wheeler Transform, enabling sublinear search time while using reduced memory. Wavelet trees are hierarchical data structures that compress sequences while supporting rank and select operations, useful for efficient character-based searches. Each of these indexing structures has trade-offs in terms of preprocessing time, query efficiency, and memory consumption.

To illustrate offline CDS indexing, consider the text $y = \text{"agaacgcagtata"}$ with pivot character set $C = \{a\}$. Its character-distance sampled representation is $\bar{y} = \langle 2, 1, 4, 3, 2 \rangle$. Instead of performing an online search, we construct a suffix array over $\bar{y}$, sorting all suffixes of the numeric sequence:

Position	Suffix
2	1, 4, 3, 2
5	2
1	2, 1, 4, 3, 2
3	3, 2
4	4, 3, 2

The corresponding sampled suffix array is $SA_{\bar{y}} = \langle 5, 3, 4, 1, 2 \rangle$. To search for the pattern $x = \text{"gaa"}$, its sampled form is computed as $\bar{x} = \langle 1 \rangle$. Using binary search on $SA_{\bar{y}}$, we locate an occurrence at position 1. This position is then mapped back to the original text, and verification is performed at $y[2 : 4]$, confirming the match.

The efficiency of offline CDS indexing depends on multiple factors, including the size of the sampled representation and the choice of indexing structure. Constructing the sampled representation takes $O(n)$ time, where n is the length of the text. Building a suffix array requires linear time using efficient sorting algorithms. FM-index construction can be done in $O(n)$ time using wavelet trees and the Burrows-Wheeler Transform. Querying the suffix array is performed in $O(m \log n)$ time, where m is the length of the pattern. FM-index search can be performed in $O(m)$ time, making it more efficient for large-scale text indexing. This analysis highlights the trade-offs: suffix arrays provide a simple but effective indexing strategy, while FM-index offers better query performance at the cost of a more complex construction.

Offline CDS indexing is particularly useful in scenarios requiring repeated searches over a static dataset. Notable applications include biological sequence analysis, where DNA and RNA sequence alignment often require efficient substring matching; plagiarism detection, where large-scale text comparisons benefit from indexed sampling techniques to identify similar passages quickly; information retrieval, where search engines and database indexing can leverage CDS for compressed yet efficient text searches; and log analysis, where large-scale log monitoring and anomaly detection often involve searching for patterns within massive text corpora.

The offline indexing approach significantly reduces search time by leveraging precomputed data structures. However, it comes with certain trade-offs. Advantages include faster search queries due to precomputed indices, compact storage with reduced redundancy, and suitability for static datasets where repeated queries are performed. Limitations involve high preprocessing time, making it unsuitable for dynamic texts; additional memory overhead for storing indexing structures; and reduced flexibility when handling text updates.

Character Distance Sampling provides a powerful mechanism for efficient text searching by leveraging a compact representation that preserves essential distance relationships. The offline variant of CDS indexing further enhances performance by enabling rapid indexed queries through data structures such as suffix arrays and FM-indexes. While the preprocessing overhead may not be suitable for dynamic texts, the approach offers significant performance gains for static datasets, making it an attractive choice for large-scale text processing applications. Future work may explore hybrid approaches that balance preprocessing costs with update efficiency, enabling CDS indexing to be applied in more dynamic environments.

6.3 Condensed Alphabet Sampling

Condensed Alphabet Sampling is a technique designed to improve the efficiency of sampled string matching in scenarios where the input alphabet is small. Traditional sampling methods, such as Character Distance Sampling, achieve substantial improvements in search performance and space efficiency but may not yield optimal results when applied to texts with small alphabets, such as genomic or protein se-

quences. The fundamental limitation arises from the low number of distinct symbols in the input, which reduces the filtering effectiveness of sampling approaches. To overcome this issue, condensed alphabets are introduced as a way to artificially expand the effective alphabet size, thereby enhancing the discrimination power of the sampled representation.

The core idea behind condensed alphabets is to transform the input sequence into a representation where each symbol encodes a combination of consecutive characters from the original alphabet. By grouping symbols, the number of distinct elements in the transformed text increases, allowing for more effective filtering in the search phase. This transformation preserves the underlying structure of the original text, ensuring that the sampled representation remains accurate and meaningful for pattern matching. The process of constructing a condensed alphabet involves defining a mapping function that assigns each substring of length q , known as a q -gram, to a unique symbol in the new alphabet through a hashing function. The choice of q depends on the statistical properties of the input text and the desired trade-off between space efficiency and search performance.

Given an input text y of length n over an alphabet Σ , the condensed representation $\hat{y}$ is obtained by partitioning the text into overlapping or non-overlapping q -grams and applying a hashing function to map each q -gram to a unique symbol from an expanded alphabet $\hat{\Sigma}$. This process effectively increases the alphabet size from $\sigma = |\Sigma|$ to approximately σ^q , leading to a more informative sampled representation.

Example 6.5. Consider a genomic sequence $y = \text{"actgacgtac"}$ of length 10, defined over the alphabet $\Sigma = \{a, c, g, t\}$. Using $q = 2$, the set of possible q -grams is:

$$\{\text{"ac"}, \text{"ct"}, \text{"tg"}, \text{"ga"}, \text{"ac"}, \text{"cg"}, \text{"gt"}, \text{"ta"}, \text{"ac"}\}.$$

Each unique q -gram is assigned a symbol from the condensed alphabet $\hat{\Sigma}$, for example:

q -gram	Condensed symbol
ac	X
ct	Y
tg	Z
ga	W
cg	V
gt	U
ta	T

Replacing the original sequence with its condensed representation, we obtain:

$$\hat{y} = \text{"XYZWXVTUX"}.$$

By increasing the effective alphabet size, the condensed representation reduces redundancy and improves search efficiency. A similar approach is widely used in bioinformatics, where the IUPAC nucleotide code is employed to represent groups of nucleotides using a condensed alphabet [40]. In this representation, ambiguous nucleotides are mapped to a reduced alphabet, allowing for more efficient processing of genetic sequences while preserving biologically relevant information.

Using condensed alphabets in sampled string matching introduces several advantages. First, the increased alphabet size leads to a more diverse distribution of symbols in the sampled representation, reducing the probability of false positive matches during the filtering phase. This directly translates to fewer verification steps in the original text, accelerating the overall search process. Second, the condensed representation significantly reduces space consumption, as multiple characters are encoded into a single symbol. In practical applications, this results in a compact index that requires a fraction of the storage needed for the original text. Third, the technique remains compatible with existing sampled string matching frameworks, allowing seamless integration with methods such as Character Distance Sampling.

The choice of q has a significant impact on both space efficiency and search performance. A small q results in a less informative representation, reducing the filtering effectiveness of sampling methods. Conversely, a large q increases the number of distinct symbols, potentially leading to sparsity issues in the condensed representation. The optimal value of q depends on the statistical properties of the input text and the expected pattern lengths in the search phase.

Experimental results demonstrate that condensed alphabets provide substantial improvements in both space efficiency and search speed. When applied to DNA sequences, the method achieves a space reduction of up to 80% while accelerating search times by up to 96% compared to previous sampled string matching techniques. In comparison to standard online string matching algorithms, the performance gains reach up to 99.6%, making condensed alphabets a powerful tool for processing biological data. The combination of space efficiency and speed improvements makes this approach particularly suited for large-scale datasets, where reducing storage and computational costs is crucial.

The effectiveness of condensed alphabet sampling is influenced by the choice of grouping strategy. A well-designed condensation function ensures that symbol mappings retain meaningful relationships between characters while maximizing the distinction between different substrings. Adaptive methods that dynamically adjust the condensation rules based on the statistical distribution of the input can further enhance performance. In applications such as genomic research, where specific character patterns carry biological significance, domain-specific condensation strategies may yield additional benefits.

The integration of condensed alphabets into sampled string matching frameworks also enables compatibility with offline indexing methods. Since the condensed representation preserves structural relationships in the text, suffix arrays or FM-index

structures can be built efficiently, allowing for fast query times with significantly reduced storage overhead. The approach remains robust across various application domains, from natural language processing to biological sequence analysis.

In conclusion, condensed alphabets provide an effective means of addressing the limitations of sampled string matching in small-alphabet texts. By increasing the effective alphabet size, the method enhances the filtering capability of sampled representations while simultaneously reducing storage requirements. The experimental results confirm that this technique significantly outperforms traditional methods in terms of both speed and space efficiency, making it a viable solution for large-scale string matching applications, particularly in bioinformatics and computational biology.

Online String Matching Algorithms

In this section, we delve into the algorithms utilized to execute the searching phase within the context of the sampling methodologies. Our selection of algorithms aims to present a comparative analysis against the original sampling implemented through Horspool algorithm, considering their suitability across varied pattern and alphabet sizes. We have curated three distinct algorithms:

- Horspool (HOR) [41]: widely used algorithm, it has been the inspiration for much works.
- Quick Search (QS) [42]: specifically tailored for large patterns and extensive alphabets.
- Weak Factor Recognition (WFR) [43]: for normal-sized patterns and smaller alphabets.
- Franek-Jennings-Smyth Algorithm (FJS) [44]: for very small patterns and large alphabets.

The subsequent sections provide an in-depth exploration of each algorithm, elucidating their underlying mechanisms and performance characteristics.

7.1 Horspool Algorithm

The Horspool algorithm is a simplified variant of the Boyer-Moore string matching technique, designed to efficiently locate occurrences of a pattern within a given text. It retains the core principle of skipping sections of the text when mismatches occur but simplifies the shift mechanism to improve ease of implementation while maintaining good average-case performance.

Given a pattern p of length m and a text t of length n , the algorithm begins by aligning the pattern at position i in the text and performing a character-by-character comparison from right to left. If a mismatch occurs or the entire pattern is not matched, a shift is determined using the character in the text that was aligned with the last character of the pattern. The shift function is computed using the bad

character heuristic, which records the rightmost occurrence of each character within the pattern. Formally, the shift table is defined as

$$\text{badchar}_p(c) = \min\{1 \leq k < m \mid p[m - k] = c\} \cup \{m\}$$

for every character c in the alphabet Σ . This function ensures that if the mismatched character appears somewhere earlier in the pattern, the pattern is shifted such that its last occurrence aligns with the same character in the text. If the character does not appear in the pattern, the default shift is equal to the pattern length m .

The algorithm consists of two phases. In the preprocessing phase, the shift table is constructed by scanning the pattern from left to right and recording the rightmost position of each character. The search phase then proceeds by aligning the pattern with the text and performing a character comparison from right to left. If a full match is found, the occurrence is recorded and the pattern shifts forward. If a mismatch occurs, the shift function determines the next alignment based on the rightmost occurrence of the mismatched character in the pattern.

The time complexity of the Horspool algorithm varies depending on the nature of the input. In the best case, the complexity is $O(n/m)$ when large shifts allow the pattern to rapidly advance through the text. In the worst case, when every character in the text forces minimal shifts, the complexity is $O(nm)$, though this scenario is rare. The average-case complexity is close to $O(n)$, making the algorithm efficient in many practical scenarios. Compared to the full Boyer-Moore algorithm, Horspool's approach is easier to implement while still leveraging heuristics to achieve significant performance improvements over naïve methods.

7.2 Quick-Search (QS) Algorithm

The Quick-Search algorithm, introduced by Sunday, is a variant of the Horspool algorithm and belongs to the family of pattern matching algorithms derived from Boyer-Moore. Its main distinguishing feature is the way it determines the shift distance after a mismatch. Instead of relying solely on the rightmost occurrence of a mismatched character within the pattern, the Quick-Search algorithm considers the character immediately following the current search window in the text.

Given a pattern p of length m and a text t of length n , the algorithm aligns the pattern at position i in the text and attempts a left-to-right comparison of characters. If a mismatch occurs or the pattern is completely scanned without finding a match, the shift is determined based on the character at position $t[s + m]$. The shift function is defined as

$$qbc_p(c) = \min\{1 \leq k \leq m \mid p[m - k] = c\} \cup \{m + 1\}$$

for every character c in the alphabet Σ . This function returns the smallest shift that aligns the rightmost occurrence of c in the pattern with its position in the text.

If the character does not appear in the pattern, the default shift is $m + 1$, allowing the search window to move beyond the current pattern alignment.

The algorithm consists of two main phases. In the preprocessing phase, a table is constructed to store shift values for every character in the alphabet based on its rightmost occurrence in the pattern. This table enables efficient computation of shifts during the search phase. The search process then proceeds by aligning the pattern with the text, performing a character-by-character comparison from left to right. If a complete match is found, the occurrence is recorded, and the pattern shifts forward. Otherwise, the shift function determines the next alignment based on the next character in the text.

The Quick-Search algorithm has a best-case time complexity of $O(n/m)$, which occurs when the shifts allow rapid advancement through the text. In the worst case, the complexity is $O(nm)$, similar to the naïve string matching algorithm, though this rarely occurs in practice. On average, the performance approaches $O(n)$, making it an efficient choice for many practical applications. Compared to Boyer-Moore and Horspool, the Quick-Search algorithm is simpler to implement and particularly effective when searching for short patterns in long texts.

7.3 WFR $_q$: Weak Factor Recognition with q -Grams

The Weak Factor Recognition (WFR) algorithm, introduced by Cantone *et al.* [43], indexes all factors of a pattern during preprocessing. The WFR $_q$ variation extends this approach by indexing all q -grams of the pattern. A hash function, **HASH**, maps each q -gram to a 16-bit integer. During preprocessing, a Boolean table F is constructed, where $F[v] = \text{True}$ if there exists a q -gram z in the pattern such that $\text{HASH}(z) = v$. The actual search pattern, denoted as P_0 , is a prefix of the pattern with a length constrained by $|P_0| = m_0 = m - (m \bmod q)$.

During the search phase, the algorithm slides a window of size m_0 along the text. At each alignment, the hash value of the last q -gram within the window is computed. If this value is not present in the factor table F , the algorithm skips to the next alignment. Otherwise, it proceeds to verify the preceding non-overlapping q -grams from right to left until either a non-factor is encountered or the start of the window is reached. In the latter case, a potential occurrence of the original pattern is identified and subsequently verified.

WFR $_q$ is both efficient and compact in implementation, making it an excellent platform for evaluating various hash functions in string matching. However, one minor drawback is its bias toward pattern lengths that are multiples of q , due to its prefix-based search mechanism. Additionally, the implementation of WFR $_q$ in the Smart repository differs slightly from the pseudocode presented in [43]. The variation of the algorithm used in our experiments is shown in Fig. 7.1.

```

SEARCH( $x, m, y, n$ )
1   $m_0 \leftarrow m - (m \bmod q)$ 
2   $j \leftarrow m_0 - 1$ 
3  while  $j < n$  do
4       $h \leftarrow \text{HASH}(j)$ 
5       $i \leftarrow j - m_0 + 1$ 
6      while  $F[h]$  and  $j > i + q - 1$  do
7           $j \leftarrow j - q$ 
8           $h \leftarrow \text{HASH}(j)$ 
9      if  $j = i + q - 1$  and  $F[h]$  then
10         if  $\text{memcmp}(y + i, x, m) = 0$  and  $i \leq n - m$  then
11             OUTPUT( $i$ )
12          $j \leftarrow j + m_0 - q + 1$ 

```

Figure 7.1: Implementation of the WFR q algorithm, a variation of Weak Factor Recognition (WFR) that indexes q -grams of the pattern for efficient string matching. This version, used in our experiments, differs slightly from the original implementation in [43] because, among other things, incremental hash calculation has been omitted. The condition $i \leq n - m$ in line 11 could be omitted by changing the upper limit in line 4, but the modified version is slightly slower.

7.4 FJS

The Franek-Jennings-Smyth string matching algorithm (FJS for short) is a simple hybrid algorithm which mixes the linear worst-case time complexity of Knuth-Morris-Pratt algorithm and the sublinear average behavior of Quick-Search algorithm. Specifically the FJS algorithm searches for matches of p in t by shifting a window of size m from left to right along t . Each attempt of the algorithm is divided into two steps. During the first step, in accordance with the Quick-Search approach, the FJS algorithm first compares the rightmost character of the pattern, $p[m - 1]$, with its corresponding character in the text, that is, $t[s + m - 1]$. If a mismatch occurs, a Quick-Search shift is implemented, moving p along t until the rightmost occurrence in p of the character $t[s + m]$ is aligned with position $s + m$ in the text. At this new location, the rightmost character of p is again compared with the corresponding text position. Only when a match is found the FJS algorithm invokes the second step. Otherwise another Quick-Search shift occurs. The second step of the algorithm consists in a Knuth-Morris-Pratt pattern matching starting from the leftmost character $p[0]$ and, if no mismatch occurs, extending as far as $p[m - 2]$. Then whether or not a match of p is found, a Knuth-Morris-Pratt shift is eventually performed followed by a return to the first step. The preprocessing time of the algorithm takes $O(m)$ time for computing the failure function of KMP,

and $O(m + \sigma)$ time in order to compute the Quick-Search bad character rule. The authors showed that the worst-case number of character comparisons is bounded by $3n - 2m$, so that the corresponding searching phase requires $O(n)$ -time. The space complexity of the algorithm is $O(m + \sigma)$.

7.5 Experimental Results

In this section, we present experimental results to evaluate the performance of the sampled string matching approaches outlined in this paper.

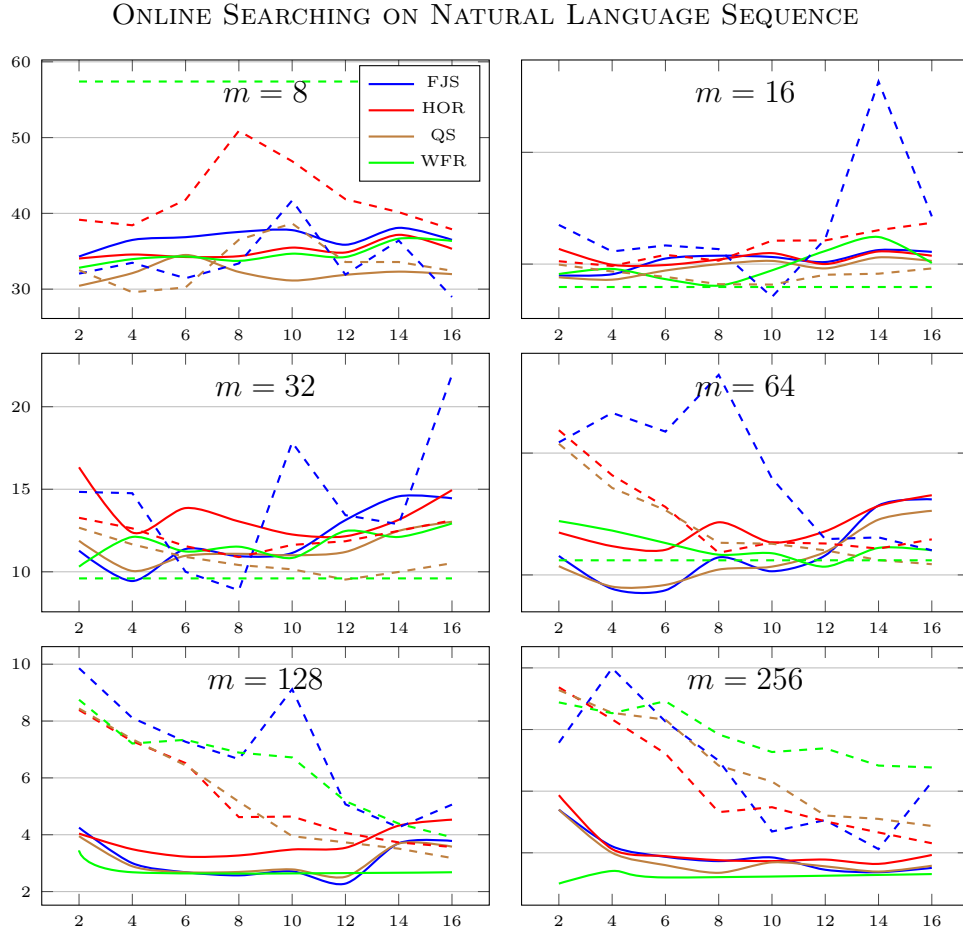


Figure 7.2: Experimental results of the sampling algorithms implemented through different online string matching algorithms, using English Texts. Straight lines represent the CDS algorithms while dashed lines represent OTS ones. Times are represented in the y axes in milliseconds (ms), while the x axes represents the rank of the pivot character $2 \leq K \leq 16$.

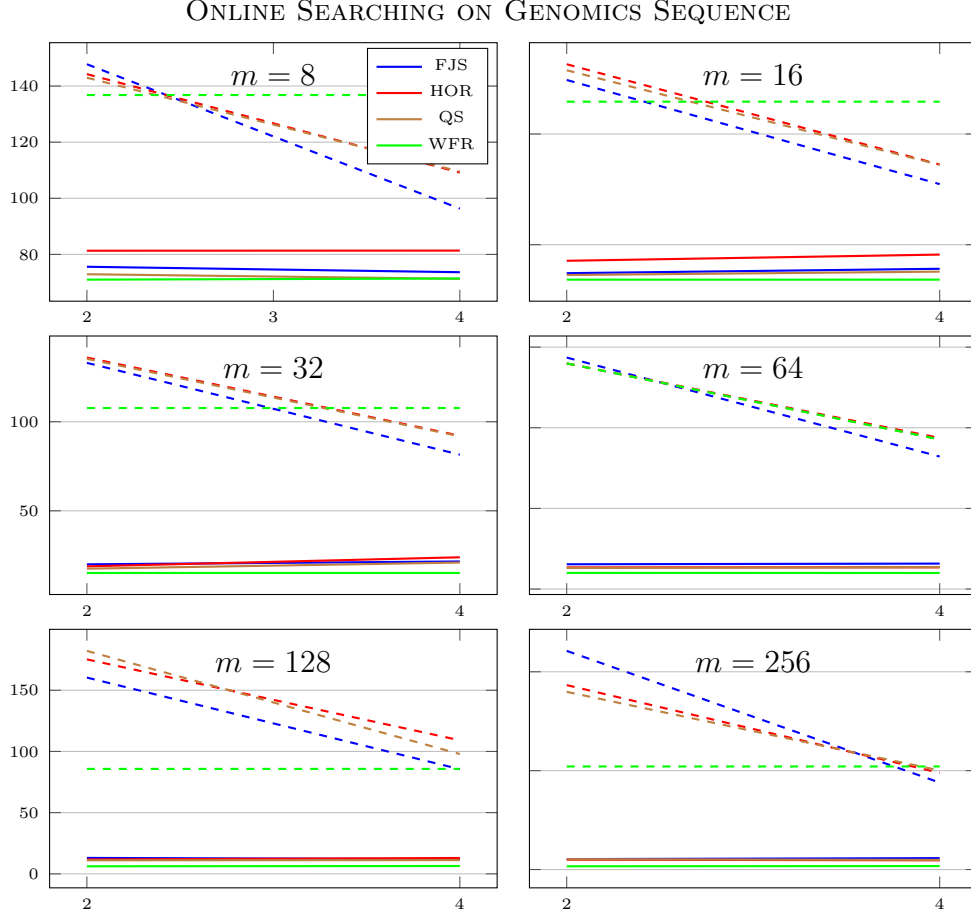


Figure 7.3: Experimental results of the sampling algorithms implemented through different online string matching algorithms, using Genomics Sequences. Straight lines represent the CDS algorithms while dashed lines represent OTS ones. Times are represented in the y axes in milliseconds (ms), while the x axes represents the rank of the pivot character $2 \leq K \leq 4$.

For both the natural language dataset and the genomics sequences, experiments were conducted for normal-sized patterns ($8 \leq m \leq 256$). Furthermore, for algorithms that can be implemented with different values of the parameter q , tests were conducted for multiple values, generally $1 \leq q \leq 8$, and only the best result among the different values of q was considered for each pattern size. Let K be the rank of a character in a totally ordered alphabet, we conducted our tests for different values of K , indeed for the natural language dataset we used $2 \leq K \leq 16$, while for the genomics sequences given $|\Sigma| = 4$ we could use only $2 \leq K \leq 4$. In both Figure 7.2 and 7.3 the legend show the color of the algorithm used in the searching phase, while the smooth line represent the CDS algorithm has been used, and the dashed lines are used to represent the OTS algorithm.

Fake Sampling Representation

Character distance sampling is a method designed to optimize storage and retrieval efficiency by selecting representative elements from a sequence. Its primary objective is to reduce computational costs associated with full-text indexing while maintaining search effectiveness. However, this approach introduces a substantial increase in per-element storage requirements. Specifically, while a character is conventionally represented using 8 bits, it is instead stored as a 32-bit integer under this scheme, leading to a fourfold increase in memory consumption.

In scenarios where the sampled positions extend beyond a quarter of the original sequence length, an unintended consequence emerges: the sampled representation may require more memory than the original text. This issue is particularly evident in genomic data processing, where sequences consist of a fixed alphabet of four nucleotides (A, C, G, and T). Given the uniform distribution of these nucleotides, character distance sampling may generate a partial index that, while improving search performance, closely matches the original text in terms of memory footprint. As a result, despite its benefits for query efficiency, this method risks negating its intended advantages by introducing excessive memory overhead.

In this section, we introduce an effective strategy to mitigate the excessive memory overhead associated with distance-based text sampling methods. The proposed technique offers a low-cost solution applicable in almost all practical scenarios. Although our discussion primarily focuses on the Character Distance Sampling (CDS) method, this approach is generalizable to any distance-based text sampling method where a constraint is imposed on the representation space of each sampled distance.

To formalize this approach, let us consider two consecutive sampled positions, denoted as p_1 and p_2 . The distance between these positions is given by $d = p_2 - p_1$. We define γ as the *distance representation bound* (DRB), a parameter that determines the number of bits allocated for storing each distance value. In this setting, every distance in the sequence is represented using exactly γ bits, restricting each stored value to the range $0 \leq d < 2^\gamma$.

As previously discussed, this constraint poses a significant limitation: distances satisfying $d \geq 2^\gamma$ cannot be directly represented, rendering the method impractical in many real-world applications.

To address this issue, we propose a solution based on the insertion of additional,

arbitrarily chosen sampled positions. This technique enables the decomposition of any distance $d \geq 2^\gamma$ into a sequence of smaller distances that conform to the imposed representation limit. Since these additional samples are not part of the original sampling process, we refer to them as *fake samples*. This terminology emphasizes that these positions are not genuine sampled locations but rather artificial placeholders introduced solely to ensure that all distances remain within the 2^γ constraint imposed

To avoid ambiguity in the sequence representation, the decomposition should be non-ambiguous. For this reason, among the many possible alternatives, this work proposes a decomposition that involves the introduction of a false sample, if necessary, every $2^\gamma - 1$ characters of the original text. In other words, if two samples are at a distance $d = p_2 - p_1 \geq 2^\gamma$, we introduce a false sample at position $p'_1 = p_1 + 2^\gamma - 1$ and we operate recursively on the residual interval $p_2 - p'_1$, provided that it is greater than or equal to 2^γ . An interval of d positions between two real samples will then be decomposed, as needed, into a sequence of $\lceil d/2^\gamma \rceil$ intervals of size $2^\gamma - 1$, made exception for the last interval whose size will be equal to $d \bmod (2^\gamma)$. The above is achieved through the insertion of fake samples. An example of the Sampling procedure is shown on **Algorithm 1**.

More formally we introduce the following definition of *Fake Distance Decomposition*.

Definition 8.1 (Fake Distance Decomposition). Let d be an integer value representing the distance between two sampled text positions. The fake distance decomposition $[d]_\gamma$ for a given DRB γ is a numeric sequence

$$[d]_\gamma = \langle d_0, d_1, \dots, d_{k-1} \rangle$$

such that:

- (a.) $k = \lceil d/2^\gamma \rceil$;
- (b.) $d_i = 2^\gamma - 1$, for $0 \leq i < k - 1$;
- (c.) $d_{k-1} = d \bmod (2^\gamma)$.

By Definition 8.1 it turns out that the sum of the distances obtained from a fake distance decomposition $[d]_\gamma$ results in the value of the original distance. Formally we have

$$\begin{aligned} \sum_{u \in [d]_8} u &= \sum_{i=0}^{k-1} d_i \\ &= \sum_{i=0}^{k-2} (2^\gamma - 1) + (d \bmod (2^\gamma)) \\ &= (2^\gamma - 1) \times \left\lfloor \frac{d}{2^\gamma} \right\rfloor + (d \bmod (2^\gamma)) \\ &= d \end{aligned} \tag{8.1}$$

Observe also that the fake distance decomposition of a distance value $d < 2^\gamma$ leaves the original sequence unchanged, since $k = \lceil d/2^\gamma \rceil = 1$. This can be expressed by the relationship $[d]_\gamma = \langle d \rangle$, if $d < 2^\gamma$.

Example 8.2. Let $\gamma = 8$ be the distance representation bound which assumes a binary representation of any distance value by using 8 bits, and therefore a maximum representable value equal to $2^\gamma - 1 = 255$. Then, if we take into account the distance values 841, 134, 256 and 255, we have the following fake distance decompositions:

$$\begin{aligned} [841]_8 &= \langle 255, 255, 255, 76 \rangle \\ [134]_8 &= \langle 134 \rangle \\ [265]_8 &= \langle 255, 10 \rangle \\ [255]_8 &= \langle 255 \rangle \end{aligned}$$

The definition of Fake Distance Decomposition given above can be easily extended in order to decompose a distance sequence and make it representable by using γ bits for each individual sample value. We then introduce the following definition of *Faked Distance Sampling*.

Definition 8.3 (Fake Distance Sampling). Let $y = \langle d_0, d_1, \dots, d_{n-1} \rangle$ be a sequence of distances, of length n , obtained from any kind of distance sampling performed on an input text. The fake distance sampled version of y , with DRB γ , is a numeric sequence, $[y]_\gamma$, obtained by the concatenation of the fake distance decompositions of its values. Formally

$$[y]_\gamma = [d_0]_\gamma + [d_1]_\gamma + \dots + [d_{n-1}]_\gamma$$

Example 8.4. Let $y = \langle 3, 124, 15, 255, 7, 9, 15, 262, 9, 841, 3 \rangle$ be a sequence of distances obtained from any kind of sampling performed on the text y . According to Definition 8.3 the fake distance sampling of y , using a DRB value equal to 8, is given by

$$[y]_8 = \langle 3, 124, 15, 255, 7, 9, 15, \underline{255, 7, 9, 255, 255, 255, 76}, 3 \rangle$$

where we underlined the sub-sequences obtained by a fake distance decomposition.

Example 8.5. Let $y = \langle 3, 124, 15, 255, 7, 9, 32, 15, 262, 9, 841, 3 \rangle$ be a sequence of distances obtained from any kind of sampling performed on the text y and let $x = \langle 15, 262, 9 \rangle$ be a sequence of distances obtained from the same kind of sampling performed on the pattern x . According to Definition 8.1 the fake sample sequences are as follows:

$$\begin{aligned} [y]_8 &= \langle 3, 124, \underline{15, 255, 7, 9}, 32, \underline{15, 255, 7, 9}, 255, 255, 255, 76, 3 \rangle \\ [x]_8 &= \langle 15, 255, 7, 9 \rangle \end{aligned}$$

Algorithm 1: Fake-Position-Distance-Sampling($y, n, pivot$)

Data: a string y , its length n and the selected pivot character
Result: The faked character distance representation of y

```

1  $\bar{y} \leftarrow \langle \rangle$ 
2  $j \leftarrow 0$ 
3  $prev \leftarrow 0$ 
4 for  $i \leftarrow 0$  to  $n - 1$  do
5   if ( $y[i] = pivot$ ) then
6      $a \leftarrow i - prev$ 
7     while ( $a > 255$ ) do
8        $\bar{y}[j] \leftarrow 255$ 
9        $j \leftarrow j + 1$ 
10       $a \leftarrow a - 255$ 
11   end
12    $\bar{y}[j] \leftarrow a$ 
13    $j \leftarrow j + 1$ 
14 end
15 return  $\bar{y}$ 
16 end

```

Therefore with this kind of representation, if we intend to search $[x]_8$ into $[y]_8$, we will have two different *verify*.

The first verify will be at position 7, will give negative response, due for the different original distances in fact, the 255, 7 of the pattern was originally a 262 differently from the one in the text. The second one at the position 12 which can also give positive response depends on the original text and pattern. It's noteworthy to highlight that employing solely the *CDS* representation without employing the *Fake Decomposition*, the number of verification steps would be reduced to just one at position 12. This consideration arises from the fact that the occurrence at position 7 would not match the sampled pattern if it were not decomposed.

8.1 Spatial Considerations in Text Sampling

In the realm of text sampling, the spatial demand for string matching emerges as a crucial consideration. It quantifies the additional space required, relative to the text size, that a given approach utilizes to efficiently address the matching problem.

Let us examine the partial index derived from our proposed approach, which incorporates an innovative mechanism. Despite employing a greater number of characters than the original technique (as illustrated in Figure 8.1), its efficiency is attributed to the inclusion of a controlled number of fake samples. Our exper-

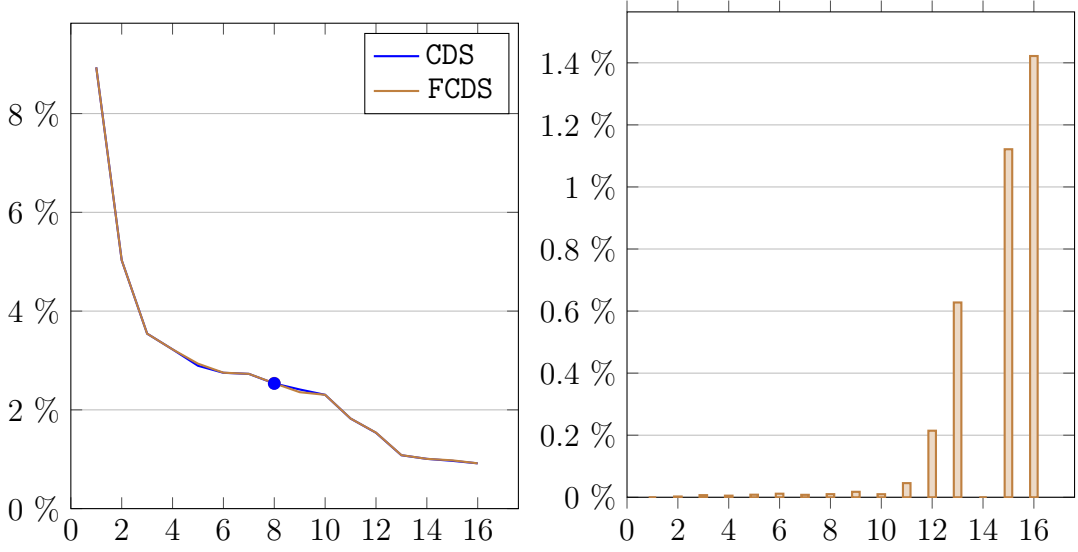


Figure 8.1: CDS and Faked-CDS approaches compared. On the left, the number of elements contained in the two indexes, calculated as a percentage of the number of characters of the text on which the indexes are built. On the right, the percentage of fake samples added in the construction of the partial index in the Faked-CDS approach, calculated relative to the number of characters in the text. In both cases, the x-axis identifies the rank of the pivot character used to construct each index.

imental analysis, conducted on an English text¹, indicates that the sizes of each index remain comparable, exhibiting only marginal variations in character count.

The explanation for this observation lies in the fact that only a small fraction of the distances between occurrences of the pivot character exceed the imposed threshold of 2^γ . This conclusion is further supported by the right-hand side of Figure 8.1, which illustrates the percentage of fake samples introduced during the construction of the partial index under the Faked-CDS approach. The proportion of fake samples required increases with the rank of the pivot character; however, it remains negligible overall.

A particularly noteworthy observation arises for the pivot of rank 14, corresponding to the character `\s` (representing a blank space) in our dataset. In this case, the number of fake samples is reduced to zero. This phenomenon can be attributed to the more uniform distribution of the space character within the text, ensuring that two occurrences of the same character never appear more than 256 characters apart.

A crucial distinction emerges upon closer examination of the memory requirements associated with these data structures. While the standard index allocates 32 bits per element, the proposed method exhibits a significant improvement in effi-

¹In our experiments, we utilized the 100MB dataset of English texts sourced from *Pizza and Chili* [23].

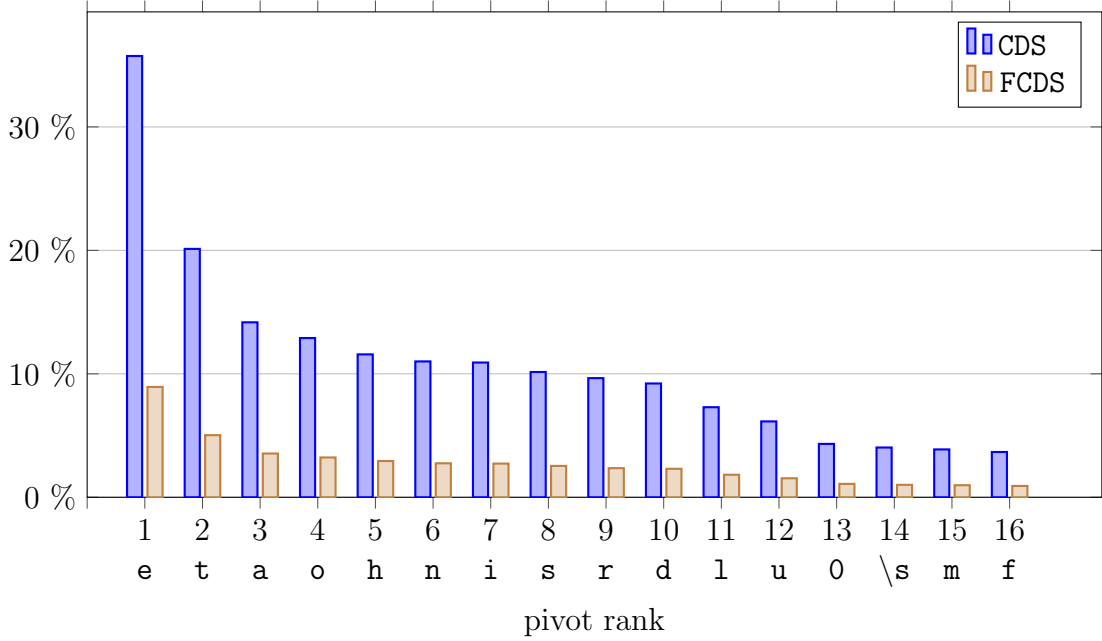


Figure 8.2: CDS and Faked-CDS approaches compared in terms of additional amount of space required for storing the two data structures. The percentage is computed as a relative to the space required for storing the text on which the indexes are built. The x-axis identifies the rank of the character used as a pivot in the construction of the index. The corresponding character for each rank is showed below each rank (\s is the space character).

ciency by utilizing only 8 bits for distance representation. This refinement results in a nearly fourfold reduction in the effective space consumption compared to its predecessor.

The proposed method effectively reduces the space required for storing the partial index by a factor ranging from 69% to 73% relative to the standard CDS approach, despite incorporating a greater number of samples in its representation. This outcome is particularly significant given that it also enhances search performance considerably (see Section 8.3 for further details). Specifically, when employing the pivot of rank 8, which has been demonstrated to yield optimal search time performance [10], the resulting index occupies only 2.5% of the text size, in contrast to the 10% required by the standard CDS method.

A comprehensive evaluation of the performance of the proposed method, considering both time and space complexity, is provided in Section 8.3.

8.2 Search Phase in Sampling-Based Searching

The search phase in a sampling-based searching approach involves the application of an exact string matching algorithm to identify all occurrences of the sampled pattern within the sampled text. Each detected candidate occurrence is subsequently verified against the original text to determine whether it constitutes a genuine match or a false positive.

In the conventional implementation of the Character Distance Sampling (CDS) approach, additional space is allocated to establish a direct mapping between each pivot character's position in the index and its corresponding position in the original text. Specifically, each index element $\dot{y}[i]$ represents the position of the i -th pivot character within the text, enabling efficient navigation from the sampled text to the original text. As a result, once a candidate occurrence is identified, its corresponding position in the text can be retrieved in $O(m)$ time, where m denotes the pattern length.

The introduction of the Fake Decomposition approach modifies this representation. Instead of storing absolute positions, this method records the distances $\bar{y}[i]$ between consecutive occurrences of pivot characters, thereby eliminating the explicit mapping of pivot character positions. Consequently, this change necessitates an alternative mechanism for retrieving the corresponding text position required for candidate occurrence verification.

By working with the distances between consecutive positions of pivot characters, we can derive the mapping $\dot{y}[i]$ of the i -th character of the index using the following formula:

$$\dot{y}[i] = \sum_{j=1}^i \bar{y}[j]$$

While this approach theoretically allows for direct application through a linear index scan algorithm, such as the Knuth-Morris-Pratt (KMP) algorithm [2], its practical efficiency requires further consideration. Given that the mapping for all positions less than i has already been computed, accessing the i -th position of the index enables the direct computation of the mapping for the i -th pivot character using the formula:

$$\dot{y}[i] = \dot{y}[i-1] + \bar{y}[i]$$

Although this method maintains simplicity and executes in linear time, its practical performance implications must be carefully evaluated. The most efficient string matching algorithms leverage forward jumps [1], allowing large portions of the text to

Algorithm 2: Search-FCDS($x, m, y, n, \bar{x}, \bar{m}, \bar{y}, \bar{n}$)

Data: a pattern x of length m , a text y of length n and their faked decomposition

Result: the number of occurrences of x in y

```

1  $hbc \leftarrow []$ 
2 for  $i \leftarrow 1$  to  $\Sigma$  do
3    $hbc[i] \leftarrow m$ 
4 end
5 for  $i \leftarrow 1$  to  $\bar{m}$  do
6    $hbc[\bar{x}[i]] \leftarrow \bar{m} - i - 1$ 
7 end
8  $s \leftarrow 1$ 
9  $count \leftarrow 0$ 
10  $last\_position \leftarrow 1$ 
11  $last\_position\_index \leftarrow 1$ 
12 while ( $s \leq \bar{n} - \bar{m}$ ) do
13    $i \leftarrow 1$ 
14   while  $i < \bar{m}$  and  $\bar{x}[i - 1] = \bar{y}[s + i - 1]$  do
15      $i \leftarrow i + 1$ 
16   end
17   if  $i = \bar{m}$  then
18      $position \leftarrow s$ 
19     while  $position > last\_position\_index$  do
20        $last\_position \leftarrow last\_position + \bar{y}[position]$ 
21        $position \leftarrow position - 1$ 
22     end
23      $count \leftarrow count + verify(x, m, y, last\_position)$ 
24   end
25    $s \leftarrow s + hbc[\bar{y}[s + \bar{m} - 1]]$ 
26   return  $count$ 
27 end

```

be skipped during scanning. Notable examples of this category include the Boyer-Moore [3] and Horspool [41] algorithms. However, such methods are unable to update the mapping on the text in constant time, thereby introducing a trade-off between theoretical efficiency and practical performance.

In this study, we examine two potential solutions to address this challenge:

The first approach, illustrated in Figure 2, involves retracing the mapping steps starting from the last verified position. Specifically, we maintain the position v of the last index position from which a text verification was performed. Given that the direct mapping for this position has already been established, we sequentially scan

all text positions from v to i in order to determine the direct mapping for the new verification position i .

This method offers efficient index construction during the search phase, as the mapping is computed only when a candidate occurrence requires verification. However, its primary drawback arises in the verification phase, where the computational burden may become significant if the last verified position is located far earlier in the text. We refer to this method as *Faked-CDS (FCDS)*.

The second approach, outlined in Algorithm 3, incorporates a technique inspired by the *OTS* approach [9]. Similar to that method, we introduce periodic links between elements in the sampled text and their corresponding positions in the original text. This design choice facilitates backtracking by eliminating the need to compute positions through cumulative summation of distances. Moreover, it allows for rapid retrieval of a character's position, provided that it has already been mapped, thereby accelerating the computation of unmapped positions in practical scenarios.

Formally, we define a parameter $k \geq 1$ and allocate an auxiliary array ρ of size $\lceil n/k \rceil$, which establishes a link between the sampled elements in the partial index and their actual positions in the text. Specifically, within the partial index, one element is sampled every k positions, resulting in a total of $\lceil n/k \rceil$ entries. Consequently, the additional space required by this structure amounts to $4 \times \lceil n/k \rceil$ bytes. When verifying position i , given that v is the index of the last verified position, the mapping process begins from the larger of the two values: either the mapping of v or $\rho[\lceil i/k \rceil]$.

The principal advantage of this approach is its ability to mitigate the need for extensive backtracking, thereby reducing verification time. However, this improvement comes at the cost of additional memory consumption to store the mapping array. We refer to this method as *Faked-CDS + Mapping (FCDS+)*.

8.3 Experimental Evaluation

In this section, we present experimental results in order to evaluate the performances of the sampling approaches presented in this paper.

In particular, we tested the original CDS approach (CDS), the Faked-CDS approach (FCDS) and the Faked-CDS enhanced with mapping (FCDS+) approach. The latter has been implemented using values of k in the set $\{8, 16, 32, 64\}$. However, the experimental results for these 4 variants are essentially identical, so we will simply unify the values of the 4 variants. Moreover we choose to set $\gamma = 8$ to conduct our experiments, where we remember that γ is the number of bits we use to represent each single element of the sampled text.

In all three instances, we opted for the pivot character ranked 8 within the text, identified as the letter **s**, as depicted in Figure 8.2. This particular choice was made

Algorithm 3: Search-FCDS+($x, m, y, n, \bar{x}, \bar{m}, \bar{y}, \bar{n}, \rho, k$)

Data: a pattern x of length m , a text y of length n , their sampled versions and the mapping array

Result: the number of matches

```

1  $hbc \leftarrow []$ 
2 for  $i \leftarrow 1$  to  $\Sigma$  do
3   |  $hbc[i] \leftarrow m$ 
4 end
5 for  $i \leftarrow 1$  to  $\bar{m}$  do
6   |  $hbc[\bar{x}[i]] \leftarrow \bar{m} - i - 1$ 
7 end
8  $s \leftarrow 1$ 
9  $count \leftarrow 0$ 
10 while ( $s \leq \bar{n} - \bar{m}$ ) do
11   |  $i \leftarrow 1$ 
12   while  $i < \bar{m}$  and  $\bar{x}[i - 1] = \bar{y}[s + i - 1]$  do
13     |  $i = i + 1$ 
14   end
15   if  $i = \bar{m}$  then
16     |  $position \leftarrow \rho[s/k]$ 
17     |  $idx \leftarrow s - (s \% k)$ 
18     while  $idx < s$  do
19       |  $position \leftarrow position + \bar{y}[idx]$ 
20       |  $idx \leftarrow idx + 1$ 
21     end
22     |  $count \leftarrow count + verify(x, m, y, position)$ 
23   end
24    $s \leftarrow s + hbc[\bar{y}[s + \bar{m} - 1]]$ 
25   return  $count$ 
26 end

```

based on its observed superior performance in terms of execution times, as detailed in [10].

We employed various pattern sizes ranging from 2^7 to 2^{12} . The space used to maintain the partial indexes and any mapping arrays is shown in the following table.

For each sequence in the dataset, we randomly selected 1000 patterns extracted from the text and computed the average running time over these 1000 runs.

As illustrated in Figure 8.3, despite utilizing less space, as demonstrated in previous sections, the *Fake Decomposition* method achieves superior performance in practice, particularly in terms of searching time.

Experimental results indicate that the newly proposed methods, FCDS and FCDS+,

Table 8.1: Memory requirements of sampling methods using a 100MB dataset of English text

Name	Index Size	Mapping Size	Samples	Fake Samples
CDS	10.14 MB	-	2536790	-
FCDS	2.53 MB	-	2537048	258
FCDS+	2.53 MB	1.26 MB	2537048	258

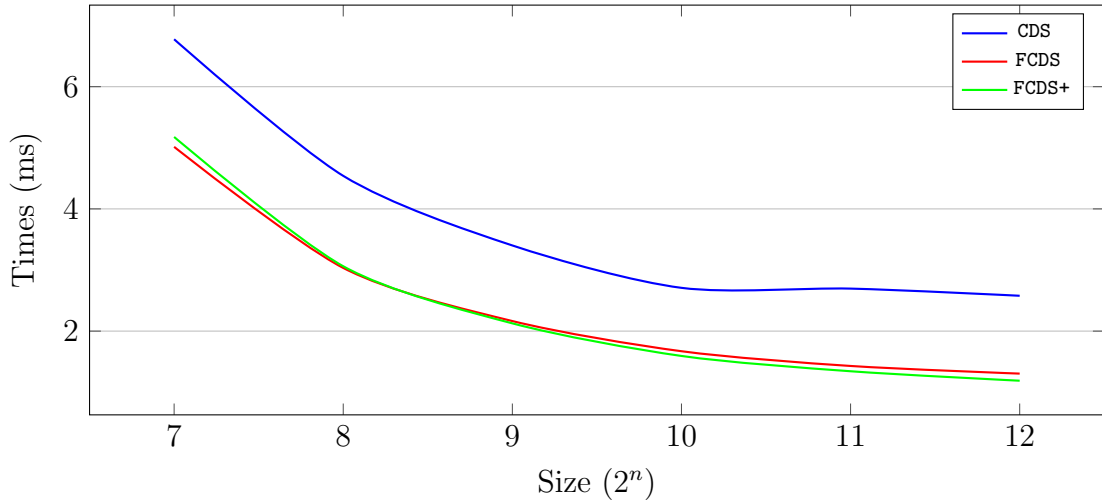


Figure 8.3: Running times of three sampled string matching algorithms for searching a 100 MB English text and averaging over 1000 different runs.

achieve a speed-up ranging from 33.7% to 55.1%. Despite the increased size of the partial index in these approaches, several factors contribute to the observed improvement in performance.

First, the expansion of the sampled pattern size—a well-documented factor in accelerating classical string matching algorithms—enhances performance, as such algorithms typically exhibit improved efficiency with larger pattern lengths. Second, while Character Distance Sampling (CDS) algorithms generally suffer from performance degradation when fewer than two distances are represented within the sampled pattern, the increase in the number of characters required for representation mitigates this issue, thereby preventing worst-case performance scenarios. Lastly, a notable speed-up is achieved in the search phase due to the elimination of distance computations via subtraction of consecutive positions; instead, each entry in the sampled text directly encodes a distance value.

While FCDS consistently achieves a space reduction of approximately 75%, the FCDS+ approach requires additional memory for position mapping, resulting in a space reduction ranging from 62.5% (when position mapping is stored every 8 characters) to 72% (when mapping occurs every 32 characters).

Although both FCDS and FCDS+ demonstrate comparable overall performance, subtle differences emerge based on pattern length. Specifically, FCDS exhibits slightly superior efficiency for shorter patterns, whereas FCDS+ performs better as pattern length increases. This discrepancy is attributed to differences in candidate occurrence verification within the partial index.

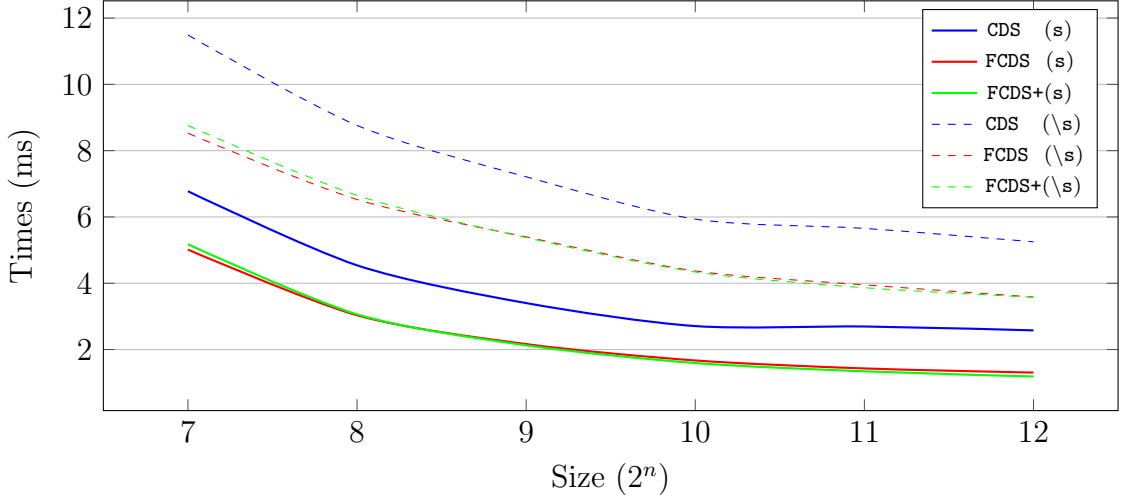


Figure 8.4: Performances of the algorithms with a 100 MB English text averaging over 1000 different executions. Solid lines indicate running times obtained with a partial index constructed using the 8-th ranked pivot character, specifically representing the character 's'. Dashed lines indicate running times obtained with a partial index constructed using the the 14-th pivot character, representing the blank space character '\s', which generates no fake samples.

For short patterns, the frequency of candidate occurrences is high, which justifies the verification time in FCDS. The close proximity of consecutive verifications reduces the impact of backtracking on search efficiency. Consequently, the additional overhead introduced by the mapping table in FCDS+ results in a slight performance penalty.

Conversely, for longer patterns, the number of candidate occurrences decreases significantly, exacerbating the performance degradation caused by backtracking in FCDS. In such cases, reconstructing the text position of an occurrence from distant locations incurs higher computational overhead. In contrast, FCDS+ benefits from the mapping table, which effectively reduces backtracking distances and improves overall efficiency.

Finally, we present a comparative analysis of the two proposed methodologies, focusing on scenarios in which different pivot characters are utilized. A common observation emerges regarding the potential advantage of selecting the **blank space** character (ranked 14 in the text) as a pivot. Specifically, within the FCDS approach, this choice eliminates the necessity for generating fake samples. One might hypoth-

esize that adopting this character as a pivot could be beneficial when performance metrics remain comparable, as it minimizes the impact on partial index construction.

However, empirical results reveal that the execution times are not comparable, as illustrated in Figure 8.4. Notably, selecting the rank 8 character as the pivot results in a substantial reduction in processing times, ranging from 40% to 50% when compared to the `blank space` character. This discrepancy underscores the importance of pivot character selection in optimizing both search efficiency and overall performance.

9

Character Context Sampling

In this section, we introduce the *Character Context Sampling* (CCS) approach, an enhanced variant of the CDS method which stores information computed on the context around each pivot character instead of the distances stored by the CDS method. At the core of this idea is the necessity to incorporate contextual information about the position of each pivot character within the partial index used for the search. This approach aims to reduce the number of false positives, thereby minimizing the number of verifications required during the search phase.

When we refer to a *context*, we mean a fixed-size substring, of fixed length q , within the vicinity of the pivot character. Figure 9.1 schematically shows the idea on which the context-based sampling model is based. The Figure compares the CDS and CCS methods if the same pivot character is used. Although the context size is set to $q = 2$, when two occurrences of the pivot character closer than 2 characters apart then the context is reduced accordingly.

However, we immediately notice that memorizing entire substrings is extremely more expensive than memorizing individual positions. For this reason, due to the potentially high memory cost of storing the entire set of substrings representing the

a	b	r	a	c	a	d	a	b	r	a	m	a	g	i	c	a
1			4		6		8			11		13				17
3			2		2		3			2		4				
a			a		a		a			a		a				a

Figure 9.1: Text sampling of the string $x = \text{abracadabramagica}$. On top, the Character Distance Sampling representation of x using the character a as the sole pivot. The exact positions of each occurrence of the pivot are stored, with the distances between consecutive occurrences indicated below. On the bottom, an example of Character Context Sampling of x , using the same pivot character and $q = 2$. Note that the last context has a length of 3, which is greater than our q ; therefore, the context is reduced to $q = 2$.

set of contexts, our method stores the context in a compact and approximate form by computing a fingerprint of the substrings, specifically a hash value.

More formally, let $hash : \Sigma^* \times \{0, 1, \dots, q-1\} \times \{0, 1, \dots, q-1\}$ be a function which, taking as input a string $w \in \Sigma^*$ and two indices i and j such that $1 \leq i < j \leq |w|$, computes an hash value of the substring $w[i..j]$. In addition, let q be an integer parameter such that $q \geq 2$, and let $\dot{y}$ be the position sampled version of the text y . Then, the CCS version of a string y is defined as $\tilde{y} = \langle CCS(1), CCS(2), \dots, CCS(n_c) \rangle$, where $CCS(i)$ corresponds to the context value of the i -th pivot character, which is defined by

$$CCS(i) = \begin{cases} hash(y, \dot{y}[i] + 1, \dot{y}[i+1] - 1) & \text{if } \dot{y}[i+1] - (\dot{y}[i] + 1) \leq q \\ hash(y, \dot{y}[i] + 1, \dot{y}[i] + q) & \text{otherwise.} \end{cases}$$

In other words, the CCS version, $\tilde{y}$, of the text y is the sequence of hash values computed on the substring starting at the positions just after each pivot character in the text, whose length is at most q and which does not include the next pivot character. These hash values encapsulate contextual information that can be used during the search phase to reduce the number of verifications.

$HASH_2(x)$	$HASH_4(x)$	$HASH_6(x)$
1 $h \leftarrow 0$	1 $h \leftarrow 0$	1 $h \leftarrow 0$
2 $h \leftarrow h + (x[1] \ll 4)$	2 $h \leftarrow h + (x[1] \ll 6)$	2 for $i \leftarrow 1$ to 6 do
3 $h \leftarrow h + x[2]$	3 $h \leftarrow h + (x[2] \ll 4)$	3 $s \leftarrow 6 - i$
4 return h	4 $h \leftarrow h + (x[3] \ll 2)$	4 $h \leftarrow h + (x[i] \ll s)$
	5 $h \leftarrow h + x[4]$	5 return h
	6 return h	

Figure 9.2: The implementations of three hash functions that are used in our implementation of the CCS sampling method. The implementations expect a hash value of 1 byte in size as output. The q values vary in the set $\{2, 4, 6\}$, while the shift values vary in the interval $\{4, 2, 1\}$, respectively.

We also notice that, in order to locate the candidate occurrences in the original text for the verification phase, the CCS approach necessitates storing both the hash values and the exact positions of each pivot character. In the worst-case scenario, storing the complete set of positions may lead to an hybrid method consuming a significant amount of additional space. To address this issue, an alternative approach involves using a mapping table ρ , as demonstrated in the OTS algorithm [9]. In their method, a mapping position is stored at regular intervals; specifically, every t pivots, the exact position of the pivot is recorded. When a sampled match is found at position i , the verification phase commences at the position $\rho[\lfloor i/k \rfloor]$ and continues until the next pivot character (see [9] for more details).

<pre> SEARCH($\tilde{y}, \tilde{x}, \dot{m}, \dot{n}, x, m, y, \rho, f$) 1 $res \leftarrow 0$ 2 $s \leftarrow 1$ 3 while $s \leq \dot{n} - \dot{m}$ do 4 $i \leftarrow 1$ 5 while $i \leq \dot{m}$ and $\tilde{y}[s + i] = \tilde{x}[i]$ do 6 $i \leftarrow i + 1$ 7 if $i = \dot{m}$ then 8 $p \leftarrow \text{COMPUTE-POSITION}(\rho, s) - f$ 9 $i \leftarrow 1$ 10 while $i \leq m$ and $y[p + i] = x[i]$ do 11 $i \leftarrow i + 1$ 12 if $i = m$ then 13 $res \leftarrow res + 1$ 14 return res </pre>	<pre> CHARACTER-CONTEXT-SAMPLING(x, m, q, p) 1 $i \leftarrow 1$ 2 $\dot{m} \leftarrow 0$ 3 while $i \leq n$ do 4 if $x[i] = p$ then 5 if $\dot{m} = 0$ then 6 $f \leftarrow i$ 7 $j \leftarrow 1$ 8 while $j \leq q$ and $x[i + j] \neq p$ do 9 $j \leftarrow j + 1$ 10 $\tilde{x}[\dot{m}] \leftarrow \text{hash}(x, i, j)$ 11 $i \leftarrow j$ 12 $\dot{m} \leftarrow \dot{m} + 1$ 13 else $i \leftarrow i + 1$ </pre>
---	---

Figure 9.3: On the left the pseudo-code for the brute-force searching procedure for the Character Context Sampling Matching. On the right the pseudo-code of the procedure to compute the Character Context Sampling version of a given string.

The searching procedure is divided into two distinct phases: the pre-matching phase, also known as sample-matching, and the verification phase. The pseudo-code of the searching procedure is described in Figure 9 (on the right).

Let y be a text of length n , let x be a pattern of length m , both strings over an alphabet Σ , and let $C \subseteq \Sigma$ be the set of pivot characters. In order to retrieve the exact position in the text, an additional position f must be stored. This position f corresponds to the distance between the first pivot and the initial position of the text. Specifically $f = \min(i : y[i] \in C)$. Thus $f = 0$ if the text starts with a pivot character, otherwise $f > 0$.

Both phases are straightforward and similar in nature. The first phase checks if all the contexts of the sampled pattern $\tilde{x}$ occurs in the sampled text $\tilde{y}$. If a candidate occurrence is found at position i of $\tilde{y}$, then the second verification phase is initiated to check the presence of a real match.

During the verification phase, the starting position is computed. This requires $\dot{y}[s]$, which is the exact position of the first pivot that was compared, and the distance f between the first pivot in the pattern and the initial position of the text. Consequently, the position p can be computed as $p = \dot{y}[s] - f$. However, we point out that, in the procedure shown in Figure 9, the exact position $\dot{y}[s]$ is computed by sub-routine COMPUTE-POSITION which uses the mapping table ρ to identify the value of such position. Once the initial position p in the original text is computed, a verification check is performed to ensure that all characters in the pattern $x[0..m-1]$ match the corresponding substring $y[p..p+m-1]$.

While its worst-case complexity can reach $O(nm)$, necessitating verification of all positions in the text, the described algorithm shows competitive, and often superior, performance compared to other established methods in practical scenarios. The subsequent section will provide supporting evidence in this regard.

9.1 Impact on the number of verifications

In order to provide a more comprehensive understanding of the improvements brought by the proposed new technique, we analyzed the average number of verifications executed by each algorithm under various settings. These settings include different pattern lengths, ranks of the chosen pivot character, and the number of characters used for hashing. The results of this analysis are illustrated in Table 9.1.

	j	$m = 16$	$m = 32$	$m = 64$	$m = 128$
Matches	-	147	18	13	11
FCDS+	2	19571	18412	5473	27
	4	30928	75715	2228	173
	6	22778	7702	6470	1040
	8	20235	4986	4487	25215
	10	19571	4099	5007	31603
CCS ₂	2	11175	12956	25	13
	4	28514	3390	476	35
	6	3330	7271	461	192
	8	2874	4520	707	350
	10	1957	2467	377	347
CCS ₄	2	9292	9832	25	13
	4	28403	6487	575	31
	6	758	1895	482	191
	8	1548	2990	891	163
	10	1957	780	51	168
CCS ₆	2	8215	9069	25	13
	4	28385	1749	277	11
	6	1814	6050	443	189
	8	1379	1377	174	194
	10	1957	512	31	168

Table 9.1: Number of verifications performed by the sampled matching algorithms during the search phase. The values shown represent the average results obtained over 1000 runs. The first row displays the exact number of pattern occurrences within the text. The subsequent rows show the number of false positives identified by each sampled matching algorithm during the search phase.

The experimental results show that for small patterns, the CDS algorithm re-

quires, in the worst case, more than 30,000 verifications, which is significantly more than the actual number of matches (147). In contrast, under optimal conditions, the CCS algorithm requires only 758 verifications, which is less than 2.5% of the original algorithm's verification count.

In the most favorable cases, the new CCS variants significantly reduce the number of verifications to values remarkably close to the actual number of pattern occurrences within the text. Specifically, for $m = 64$, the CCS_4 variant reduces the number of false positives to nearly match the number of actual occurrences (25 versus 13). Furthermore, for $m = 128$, the CCS_6 variant completely eliminates false positives. This demonstrates that the context-based approach manages to drastically reduce the number of verifications required during the search phase, justifying the superior performance in terms of search times discussed previously.

9.2 Analysis of the space consumption.

Given that the memory required to store the hashing is equivalent to the memory needed to store all the distances using the fake-representation, and that the mapping tables are identical, the newly proposed algorithm eliminates the need for additional fake-samples. Consequently, the CCS algorithm demands fewer memory resources while providing a speed-up in the searching time. Specifically, Figure 9.4 shows the space consumption of both the FCDS+ and CCS algorithm for different rank of the pivot character, ranging from 2 to 10. We observe that the size of the partial index is always below 10% (corresponding to 10 MB) of the text size and reaches 5% (5 MB) for the rank 8 pivot.

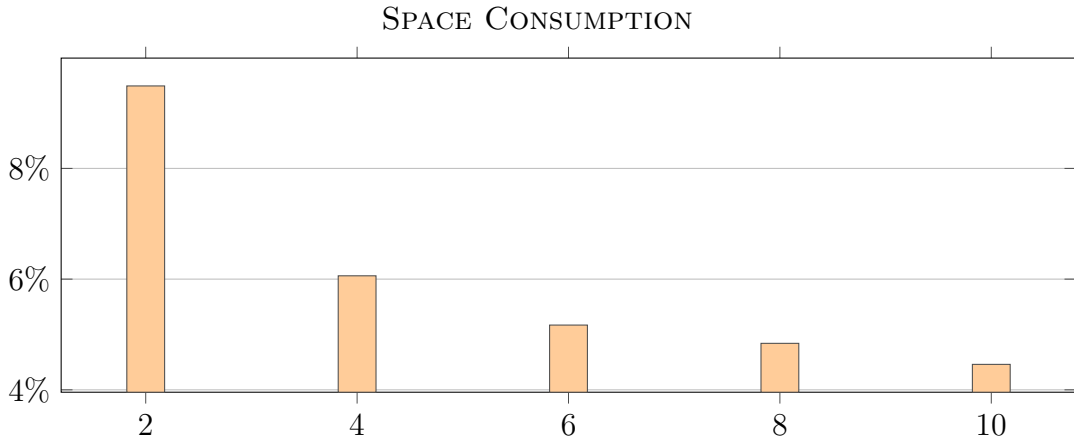


Figure 9.4: Space consumption of both the FCDS+ algorithm and the CCS algorithm for different rank of the pivot character.

Computing Periods and Covers Using Character Distance Sampling

String analysis is fundamental in various domains, including literature, linguistics, computer science, and computational biology, where DNA, RNA, and protein sequences are represented as character chains. Identifying structural regularities in strings is essential for efficient data processing, pattern recognition, and compression. Among these, *periods* [45] and *quasiperiodicities (covers)* [46, 47, 48] are widely studied in combinatorial pattern matching, with applications in text indexing, bioinformatics, and data compression.

In recent years, *Characters-Distance-Sampling* (CDS) has been successfully applied in string processing tasks such as exact pattern matching [11, 13, 12]. CDS represents a string by storing the distances between occurrences of selected *pivot characters*, typically leading to a more compact representation of the string. However, despite its effectiveness in searching and indexing, CDS has not yet been leveraged for computing string regularities.

We present the first application of CDS to the computation of fundamental string regularities, specifically the *period* and the *shortest cover* of a string. We develop efficient algorithms that operate directly on the CDS representation of a string, significantly improving computational efficiency compared to classical methods. Our experimental results demonstrate that these methods achieve substantial speedups over traditional approaches.

10.1 Classical Computation of the Period of a String

A fundamental approach for computing the period of a string x relies on determining the length of the border of all its prefixes. The key observation is that the length of the longest border of x directly determines its period.

The *border array* of a non-empty string x of length m is an array of size $m + 1$ defined as follows:

$$\text{border}_x[1] = -1, \quad \text{border}_x[i] = |\text{Border}(x[1 \dots i])| \quad \text{for } 2 \leq i \leq |x| + 1.$$

```

BORDERS( $x, m$ )
1   $border_x[1] \leftarrow -1$ 
2   $i \leftarrow 1$ 
3  for  $j \leftarrow 2$  to  $m$  do
4     $border_x[j] \leftarrow i$ 
5    while  $i > -1$  and  $x[i] \neq x[j]$  do
6       $i \leftarrow border_x[i]$ 
7     $i \leftarrow i + 1$ 
8   $border_x[m + 1] \leftarrow i$ 
9  return  $border_x$ 

```

Figure 10.1: Classical computation of the border array of string x of length m .

The border array can be computed in linear time using the classical algorithm presented in [49]. This structure allows the period of the string to be determined efficiently using the relation $per(x) = m - border_x[m]$, where m is the length of x . Figure 10.1 shows the pseudo-code for the classical computation of the border.

Figure 10.2 illustrates the computation of the border array for the string $x = \text{abaababaaba}$. The border array $border_x$ encodes structural information about the string by tracking the longest proper border of each prefix of x .

Each entry $border_x[i]$ represents the length of the longest border for the prefix $x[0 \dots i-1]$. This means that at position i , the prefix $x[0 \dots i-1]$ contains a repeated pattern of length $border_x[i]$.

For instance, the prefix $x[0 \dots 4] = \text{abaab}$ has a border of length 2, meaning that its first two characters match its last two characters. Similarly, the prefix $x[0 \dots 10] = \text{abaababaaba}$ has a border of length 6, indicating that the first six characters match the last six.

i	1	2	3	4	5	6	7	8	9	10	11	12
$x[i]$	a	b	a	a	b	a	b	a	a	b	a	
$border_x[i]$	-1	0	0	1	1	2	3	2	3	4	5	6

Figure 10.2: Border array of $x = \text{abaababaaba}$ of length 11.

For the full string of length $m = 11$, the border array value at index m is given by $border_x[12] = 6$. Since the period of a string is calculated as $m - border_x[m]$, we derive: $per(x) = 11 - border_x[12] = 11 - 6 = 5$.

This result confirms that x repeats itself every 5 characters, meaning the shortest repeating unit in x has length 5.

10.2 Classical Computation of the Shortest Cover of a String

Since a cover is inherently a border, computing the shortest cover of a string requires first identifying all its non-empty borders. For a string x of length m let us define the sequence of borders recursively as follows:

$$\text{Border}^0(x) = x, \quad \text{Border}^i(x) = \text{Border}(\text{Border}^{i-1}(x)) \text{ for } i > 0.$$

The sequence of non-empty borders is then:

$$\text{Border}^1(x), \text{Border}^2(x), \dots, \text{Border}^k(x) \neq \varepsilon, \quad \text{such that } \text{Border}^{k+1}(x) = \varepsilon.$$

The shortest cover of x is the smallest non-empty border $\text{Border}^j(x)$ that covers all the borders $\text{Border}^g(x)$ for $0 \leq g < j$.

The algorithm starts by computing the shortest border and iteratively checks whether each border covers longer ones. If a border $\text{Border}^j(x)$ fails to cover some $\text{Border}^h(x)$ with $h < j$, the process resumes with $j = h$. To ensure a linear-time complexity, when a border w of a prefix $x[0 \dots n-1]$ has length $\ell > n/2$, its length is adjusted as: $\ell = n - \ell + n \bmod (m - \ell)$. This effectively reduces w to uu' , where $w = u^r u'$ and u' is a prefix of u .

For instance, to compute the shortest cover, we first extract all the non-empty borders of x . The decreasing sequence of the non-empty border lengths for $x[0 \dots 11] = \text{abaababaaba}$ is:

$$(\text{border}_x^1[|x|] = 6, \text{border}_x^2[|x|] = 3, \text{border}_x^3[|x|] = 1).$$

This means that sequence of borders in increasing length of x is: (**a**, **aba**, **abaaba**).

For a string to be a cover, every occurrence of the border must occur in a way that ensures complete coverage of x . Here, the smallest border, **a**, is not sufficient to cover **aba**. However, the border **aba** does cover both **abaaba** and the full string x . Thus, the shortest cover of x is: **aba**. This result confirms that the minimal repeating unit that effectively covers x is of length 3.

The classical algorithm for computing the shortest cover of a string is given in Figure 10.3. It first computes the border array and then processes the sequence of borders using a stack S (Lines 6-12). The algorithm employs the function `ISCOVERING(listPos, b)` at lines 15 and 18 to verify whether a border b covers the list of positions listPos . The function `pos` (lines 14 and 17) can be implemented using any linear-time string matching algorithm.

```

SHORTESTCOVER( $x, m$ )
1   $border_x \leftarrow \text{BORDERS}(x, m)$ 
2   $S \leftarrow \text{EMPTYSTACK}()$ 
3   $\text{PUSH}(S, m)$ 
4   $n \leftarrow m$ 
5   $i \leftarrow border_x[m]$ 
6  while  $i \neq 0$  do
7      if  $i > m/2$  then
8           $\text{PUSH}(S, n - i + m \bmod (n - i))$ 
9      else  $\text{PUSH}(S, i)$ 
10      $n \leftarrow i$ 
11      $i \leftarrow border_x[i]$ 
12  $b \leftarrow \text{POP}(S)$ 
13 while  $|S| > 0$  do
14      $listPos \leftarrow pos(x[1 \dots b], x[1 \dots \text{TOP}(S)]) \cup \{m\}$ 
15     while  $|S| > 1$  and  $\text{ISCOVERING}(listPos, b)$  do
16          $\text{POP}(S)$ 
17          $listPos \leftarrow pos(x[1 \dots b], x[1 \dots \text{TOP}(S)]) \cup \{m\}$ 
18     if  $\text{ISCOVERING}(listPos, b)$  then
19         return  $x[1 \dots b]$ 
20     else  $b \leftarrow \text{POP}(S)$ 
21 return  $x[1 \dots b]$ 

```

Figure 10.3: Classical computation of the shortest cover of string x of length m .

10.3 Computing the period of a string from its CDS representation

This section presents an efficient method for computing the period of a string using its CDS representation. We first consider the case where the alphabet consists of only two distinct characters, meaning $\sigma = |\Sigma| = 2$, thus $\Sigma = \{a, b\}$. Let a be the only pivot character. Let x be a string of length m . The function δ denotes the position of occurrences of a designated pivot character in x . The CDS representation of x is denoted by $\bar{x}$, and its length is given by $\bar{m} = |\bar{x}|$.

The key idea is that the periodic structure of $\bar{x}$ provides enough information to determine the period of x . Before proving this, we present two auxiliary results that establish the fundamental relationships between x and its CDS representation.

From any position $\delta(i)$ of the pivot character a , all positions in δ greater than $\delta(i)$ correspond to $\delta(i)$ plus a sum of elements from $\bar{x}$. These positions also contain a .

Lemma 10.1. Let $x[\delta(i)] = a$ for $1 \leq i \leq \bar{m}$. Then:

$$x[\delta(i) + \sum_{j=i}^k \bar{x}[j]] = a, \quad \text{for } i \leq k \leq \bar{m} - 1.$$

Proof. Expanding the sum of distances between consecutive pivot occurrences, we obtain:

$$x[\delta(i) + \sum_{j=i}^k \bar{x}[j]] = x[\delta(i) + (\delta(i+1) - \delta(i)) + \cdots + (\delta(k+1) - \delta(k))].$$

Since $\delta(k+1)$ corresponds to the position of the pivot character, it follows that:

$$x[\delta(k+1)] = a.$$

□

A similar result holds for positions preceding a pivot occurrence.

Lemma 10.2. Let $x[\delta(i)] = a$ for $1 \leq i \leq \bar{m}$. Then:

$$x[\delta(i) - \sum_{j=k}^{i-1} \bar{x}[j]] = a, \quad \text{for } 0 \leq k \leq i - 1.$$

Proof. By summing the distances between previous occurrences of the pivot character, we obtain:

$$x[\delta(i) - (\delta(k+1) - \delta(k) + \cdots + \delta(i) - \delta(i+1))] = x[\delta(k)].$$

Since $\delta(k)$ corresponds to an occurrence of the pivot character, we conclude that: $x[\delta(k)] = a$. □

Observation 1. Let $k = \sum_{i=1}^{per(\bar{x})} \bar{x}[i]$. By definition of $per(\bar{x})$, it follows that:

$$\sum_{i=1}^{per(\bar{x})} \bar{x}[j+i] = k, \quad \text{for } 1 \leq j \leq \bar{m} - per(\bar{x}).$$

Let $x[1]$ be the pivot character. If the first and last character of x are the same ($x[1] = x[m]$), the period is determined directly from the sum of the first $per(\bar{x})$ elements of $\bar{x}$. If they are different ($x[1] \neq x[m]$), additional conditions must be checked.

i	1	2	3	4	5	6	7	8	9	10	11
$x[i]$	a	b	a	a	b	a	b	a	a	b	a
$\bar{x}[i]$	2	1	2	2	1	2					
$border_{\bar{x}}[i]$	-1	0	0	1	1	2	3				

Figure 10.4: Border array of the CDS representation of $x = \text{abaababaaba}$ of length 11 with pivot **a**. Then $per(\bar{x}) = 6 - border_{\bar{x}}[6] = 3$, thus $per(x) = \bar{x}[0] + \bar{x}[1] + \bar{x}[2] = 2 + 1 + 2 = 5$.

The table in Figure 10.4 illustrates the computation of the period for $x = \text{abaababaaba}$. The corresponding CDS representation, denoted by $\bar{x}$, consists of the distances between consecutive occurrences of the pivot character **a**. The border array of $\bar{x}$ is computed, and from it, the period of $\bar{x}$ is determined as $per(\bar{x}) = 3$. Using this result, we obtain the period of x by summing the first three elements of $\bar{x}$, yielding $per(x) = 2 + 1 + 2 = 5$. This example demonstrates how the periodic properties of $\bar{x}$ provide a direct way to compute the period of x efficiently.

If $x[0] \neq x[m-1]$, the structure of the borders must be analyzed further. Let $x = auav$, where $u \in \Sigma^*$ and $v = b^k$ does not contain any occurrences of a . The border of x can be computed by examining the borders of aua .

Lemma 10.3. The border of $auav$ is given by $u'v'$, where $u' = \text{Border}(aua)$ is the longest border of aua followed by $v' = b^{k'}$ such that $k \leq k'$.

Proof. Assume by contradiction that w is a border of $x = auav$ and that $|w| > |u'v'|$. In this case, w must be of the form $aw'av'$, where $aw'a$ is a border of aua that is longer than $\text{Border}(aua)$. This contradicts the assumption that $\text{Border}(aua)$ is the longest border of aua , proving the claim. $\square$

Figure 10.5 illustrates the CDS representation of the string $x = \text{abbababbabb}$ and the steps involved in computing its period. The figure shows the border array of the CDS representation, which allows us to determine the period efficiently. By analyzing the values in the border array and summing the appropriate distances from the CDS representation, we obtain $per(x) = \bar{x}[0] + \bar{x}[1] + \bar{x}[2] = 3 + 2 + 3 = 8$. This confirms that the period of x is 8, as derived from its compact CDS encoding.

When x is of the form $auab^k$, checking if a border of aua is followed by $b^{k'}$ with $k \leq k'$ can be determined in constant time by examining values in $\bar{x}$. Thus the worst-case complexity of the algorithm implementing this method is the same as the worst-case time complexity of the algorithm implementing the classical method. Thus it is linear in the length of $\bar{x}$, which is at most m .

Let us now consider the case where the alphabet contains more than two distinct characters. Suppose that x is of the form $x = auav$, where $u \in \Sigma^*$ and v is a substring that contains no occurrences of the pivot character a . In this scenario, the longest candidate border of x computed by the previous algorithm must be explicitly verified to determine whether it is a valid border of x .

i	1	2	3	4	5	6	7	8	9	10	11
$x[i]$	a	b	b	a	b	a	b	b	a	b	b
$\bar{x}[i]$	3	2	3								
$border_{\bar{x}}[i]$	-1	0	0	1							

Figure 10.5: Border array of the CDS representation of $x = \text{abbababbabb} = \text{abbababbab}^2$ of length 11 with pivot **a** thus $k = 2$. $border_{\bar{x}}[3] = 1$ but $\bar{x}[border_{\bar{x}}[3] = 1] = 2 \leq k = 2$. $border_{\bar{x}}[1] = 0$ and $\bar{x}[border_{\bar{x}}[1] = 0] = 3 > k = 2$. Since we use the border of $\bar{x}$ of length $border_{\bar{x}}[1] = 0$, we use the period of $\bar{x}$ which is equal to $|\bar{x}| - border_{\bar{x}}[1] = 3$, then we sum the first 3 elements of $\bar{x}$ to get $per(x) = \bar{x}[0] + \bar{x}[1] + \bar{x}[2] = 3 + 2 + 3 = 8$.

PERIODCDS(x, m)

```

1  $\bar{x} \leftarrow$  CDS version of  $x$ 
2  $\bar{m} \leftarrow |\bar{x}|$ 
3  $k \leftarrow$  length of the longest suffix of  $x$ 
   that does not contain any  $a$ 
4  $bor \leftarrow border_{\bar{x}}$ 
5  $\dot{x}[1] \leftarrow \bar{x}[1]$ 
6 for  $i \leftarrow 2$  to  $\bar{m}$  do
7    $\dot{x}[i] \leftarrow \dot{x}[i-1] + \bar{x}[i]$ 
8  $b \leftarrow \text{BORDERCDS}(x, m, \bar{x}, \bar{m}, k, \dot{x}, bor)$ 
9  $period \leftarrow m - b$ 
10 return  $period$ 
```

BORDERCDS($x, m, \bar{x}, \bar{m}, k, \dot{x}, bor$)

```

1  $i \leftarrow \bar{m}$ 
2 while  $i > -1$  do
3    $i \leftarrow bor[i]$ 
4   while  $i > -1$  and  $\bar{x}[i] \leq k$  do
5      $i \leftarrow bor[i]$ 
6   if  $i > -1$  then
7      $\bar{p} \leftarrow \bar{m} - i$ 
8      $p \leftarrow \dot{x}[\bar{p} - 1]$ 
9      $b \leftarrow m - p$ 
10    if  $x[1 \dots b] =$ 
11       $x[m - b \dots m]$  then
12      return  $(b, i)$ 
12 return  $(0, -1)$ 
```

Figure 10.6: Computation of the period of a string x of length m with its CDS representation $\bar{x}$ of length $\bar{m}$ (left) computed using the border array of $\bar{x}$ and its cumulative sum array $\dot{x}$ (right). The variable k corresponds to the number of non-pivot symbols after the rightmost pivot symbol in x . The algorithm borderCDS returns the length of the border of x and the length of the border of $\bar{x}$ that corresponds to the period of $\bar{x}$ that can enable to compute the period of x .

This candidate border should be followed by a substring v' , which, like v , contains no occurrences of a and is at least as long as v , i.e., $|v'| \geq |v|$. The algorithm then checks whether the candidate border occurs both as a prefix and a suffix of x . If this condition holds, the corresponding border length enables to compute a valid period for x . If not, the process continues iteratively, examining the next longest candidate border and its associated suffix v' , until either a valid period is found or no further non-empty borders exist.

If no valid period is identified through this process, it follows that x is borderless, and its period is simply m , the length of the entire string.

The algorithm presented in Figure 10.6 efficiently implements this method for computing the length of the longest border of x . Since several prefix sums of $\bar{x}$ may be required in the computation, the algorithm uses a precomputed cumulative sum array over $\bar{x}$. This preprocessing step ensures that the necessary sums can be retrieved in constant time, optimizing the overall efficiency of the method. As in the case of exact string matching with sampling, the method only computes border candidates that need to be naively checked (line 10) leading to an overall quadratic worst case time complexity. However, experimental results show that this method is faster than the classical one.

Theorem 4. *Algorithm PERCDS(x, m) computes the period of a string x of length m in time $O(\bar{m})$ and in extra space $O(m)$.*

Proof. The correctness of the algorithm comes from Lemmas 10.1 to 10.3. The time complexity comes from the fact that the inner loop of algorithm PERBOR can only be executed $O(\bar{m})$ time and that the comparison in line 10 can be performed in constant time using Range Minimum Queries on a Longest Common Prefix array on the suffixes of x . The extra space complexity comes from some constants and the arrays $border_{\bar{x}}$ and s that require $O(\bar{m})$ space and the $O(m)$ space needed to compute the RMQs in constant time. $\square$

A slight modification of the approach (see Figure 10.7) allows it to be extended to compute not only the length of the longest border of x but also all the length of its non-empty non-periodic border. The algorithm can enable to systematically compute and store these border lengths in decreasing order.

10.4 Computing the shortest cover of a string from its CDS representation

The classical computation of the shortest cover of a string x relies on its border array. Since borders play a fundamental role in determining covers, the approach used to compute borders in the CDS representation can be adapted to efficiently determine the shortest cover as well.

The key idea remains the same: covers, like periods, are closely tied to the border structure of a string. The shortest cover of x must be a border that also satisfies the covering condition, ensuring that every position in x is part of at least one occurrence of the cover.

The algorithm consists of merely identifying borders and verifying which of these borders serve as valid cover for x . This is accomplished by leveraging the border array of the CDS representation of x , efficiently determining the shortest cover without needing to explicitly reconstruct the full string.

```

BORDERSCDS( $x, m$ )
1   $B \leftarrow \emptyset$ 
2   $\bar{x} \leftarrow$  CDS version of  $x$ 
3   $\bar{m} \leftarrow |\bar{x}|$ 
4   $k \leftarrow$  length of the longest suffix of  $x$ 
   that does not contain any  $a$ 
5   $\dot{x}[1] \leftarrow \bar{x}[1]$ 
6  for  $i \leftarrow 1$  to  $\bar{m}$  do
7     $\dot{x}[i] \leftarrow \dot{x}[i-1] + \bar{x}[i]$ 
8   $bor \leftarrow border_{\bar{x}}$ 
9   $i \leftarrow \bar{m}$ 
10  $M \leftarrow m$ 
11 while  $i > -1$  do
12    $(b, i) \leftarrow \text{BORDERCDS}(x, m, \bar{x}, \bar{m}, k, \dot{x}, border_{\bar{x}})$ 
13   if  $b > M/2$  then
14      $b \leftarrow M - b + M \bmod M - i$ 
15   while  $i > -1$  and  $\dot{x}[i] \geq b$  do
16      $i \leftarrow i - 1$ 
17    $\bar{m} \leftarrow i + 1$ 
18    $m \leftarrow i$ 
19   if  $b > 0$  then
20      $B \leftarrow B \cup \{b\}$ 
21 return  $B$ 

```

Figure 10.7: Computation of the non-empty non-periodic borders of string x of length m using its CDS representation. This implementation uses the algorithm BORDERCDS described in Fig. 10.6.

By utilizing the properties of the CDS representation, the adapted algorithm ensures that the computation of the shortest cover remains efficient. The method minimizes redundant checks and avoids unnecessary recomputation by systematically examining the borders derived from the CDS representation. In case of binary alphabets this can be done in linear worst-case time complexity, while for larger alphabets this achieves quadratic worst-case time complexity, however experimental results demonstrate that this solution is faster in practice compared to the classical computation.

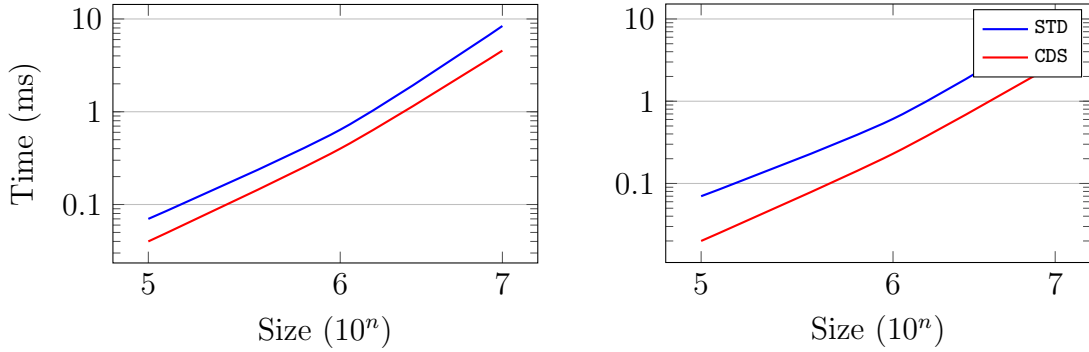


Figure 10.8: (Left) Average execution times of the standard shortest cover computation algorithm and its optimized version using the CDS representation over 10,000 runs. (Right) Average execution times of the standard period computation algorithm and its optimized version using the CDS representation over 10,000 runs.

10.5 Experimental results

In this section, we present experimental results to evaluate the performance of the algorithms proposed in this chapter. Our objective is to assess the computational efficiency of the methods that leverage the CDS representation for computing the period and shortest cover of a string.

To obtain statistically significant results, each algorithm was executed over 10,000 different runs, and the average execution time was recorded. The dataset used in our experiments consists of 100MB of English text extracted from the *Pizza and Chili* corpus [23], which is widely adopted in the field of string processing research. The experiments were performed on input strings of varying lengths, ranging from 10^5 to 10^7 characters. For consistency, the pivot character selected for constructing the CDS representation was always the first character of the input string. The additional memory required to store the CDS representation varied between 3% and 15% of the size of the original string, depending on the structure of the input data.

The results in Figure 10.8 indicate that the method for computing the period using the CDS representation achieves a speedup between 38% and 43% compared to the standard approach. This improvement is attributed to the more compact nature of the CDS representation, which reduces the number of operations required to determine the periodicity of the string.

Similarly, Figure 10.8 also presents the execution times for computing the shortest cover of a string using both the standard method and the optimized CDS-based approach. The results show that the CDS representation leads to a significant performance gain, achieving speedups between 63% and 72%. This improvement is due to the ability of the CDS representation to efficiently encode the positions of relevant characters, allowing the algorithm to determine covers with fewer operations.

These experimental results demonstrate the efficiency of the CDS-based approaches

for computing both the period and shortest cover of a string. By leveraging the compactness and structure of the CDS representation, these methods significantly reduce computation time while maintaining correctness. The proposed techniques are particularly well-suited for applications involving large-scale text processing, where reducing computational overhead is critical.

Optimizing Alphabet Sampling Using Set Cover

Despite recent advances, constructing an optimal sampling representation remains a complex and largely unexplored problem. The main challenge lies in the strategic selection of pivot characters or a subset of the alphabet, which directly impacts both the size of the sampled text and search performance. Existing approaches in the literature have primarily tackled this issue from an experimental perspective, lacking a theoretical foundation to guide the effective selection of pivot characters.

In this section, we present a novel perspective by framing the selection of pivot characters in CDS as a variant of the classical *Set Cover Problem*, although the presented method is valid for all the different sampling presented in this paper. Specifically, we observe that text substrings can be treated as elements to be covered, while pivot characters correspond to sets capable of covering multiple substrings. This theoretical insight bridges CDS with a well-studied optimization problem and provides a foundation for designing efficient algorithms that optimize memory usage in CDS representations.

Beyond the innovative aspects introduced by this pivot character selection strategy, it proves to be highly effective in the context of SSM, achieving the following key results:

- Eliminating false negatives during the search phase by directly handling cases where the pattern contains no pivots;
- Reducing the space required for the sampled text by up to 83.5% compared to the standard CDS approach;
- Improving search efficiency, achieving a speed-up of up to 86.2% over the standard CDS method;
- Defining a systematic procedure that supports an alphabet expansion strategy through condensed characters and the direct computation of the pivot set.

The CDS approach introduced above presents two fundamental challenges. From a search efficiency perspective, an appropriate alphabet sampling strategy, based on

the selection of pivot characters, must prevent worst-case scenarios—cases where the pattern contains no occurrences of pivot characters, rendering the sampled text ineffective. Simultaneously, minimizing the number of pivot character occurrences in the text is crucial to reducing the size of the sampled text.

If we assume that the length of the search pattern is at least r , achieving both objectives requires ensuring that every substring of length $m \geq r$ in the text y contains at least one pivot character. This approach minimizes the occurrence of worst-case scenarios while keeping the sampled text compact. The core idea of this work is based on the observation that this problem is equivalent to the well-known *Set Cover Problem* and can be directly mapped to it.

The *Set Cover Problem* is a classical problem in combinatorial optimization with broad applications in areas such as network design, resource allocation, and computational biology [20, 50]. Formally, given a universe of elements $U = \{e_1, e_2, \dots, e_n\}$ and a collection of subsets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$, where each $S_i \subseteq U$, the objective is to identify the smallest sub-collection $\mathcal{C} \subseteq \mathcal{S}$ such that the union of subsets in $\mathcal{C}$ covers all elements of U :

$$\bigcup_{S \in \mathcal{C}} S = U.$$

The goal is to minimize the size of $\mathcal{C}$, as it represents the minimal selection required for full coverage [51]. The problem is NP-hard [50], making it computationally infeasible to find an exact solution in polynomial time unless P equals NP. Despite its intractability, the Set Cover Problem can be effectively approximated using heuristic algorithms.

A closely related problem relevant to this work is the *Hitting Set Problem* [50], which requires finding the smallest subset of elements $H \subseteq U$ such that every subset in $\mathcal{S}$ contains at least one element from H . The Hitting Set Problem is the dual of the Set Cover Problem, as selecting elements in one corresponds to selecting subsets in the other. For simplicity, we focus on the Set Cover Problem in this paper.

11.1 Mapping Pivot Selection to the Set Cover Problem

We are now ready to demonstrate how the problem of selecting a set of pivot characters from the text can be mapped to the classical Set Cover Problem. Given a string y of length n and a pattern x of length $m \geq r$, the task is to identify the smallest set of pivot characters such that every substring of length at least r contains at least one pivot character. Let U represent the universe of all distinct substrings of length r in y . Formally:

$$U = \{y[i : i + r - 1] \mid 0 \leq i \leq n - r\},$$

where $y[i : i + r - 1]$ denotes the substring of y starting at index i , with length r . Note that if two substrings of y are identical, they are included in U only once. Thus, it is straightforward to observe that the size of U is $O(\min(n, \sigma^r))$. However, since we typically work with relatively large values of σ , as we will see in the following sections, this value can always be reduced to $|U| = O(n)$. For simplicity, we will assume this reduction in the subsequent discussion. Thus, for each character c occurring in y , define a set S_c containing all substrings in U in which c occurs. Formally:

$$S_c = \{s \in U \mid c \text{ occurs in } s\}.$$

The objective of the Set Cover problem in this context is to determine a minimal collection of characters $C \subseteq \Sigma$ such that the union of their corresponding sets covers U . More formally

$$\bigcup_{c \in C} S_c = U.$$

Solving this optimization problem ensures that the sampled text $\bar{y}$ is minimized while maintaining complete coverage for patterns of length $m \geq r$.

It is also straightforward to prove that, the Universe U corresponds to the Substring Complexity [52].

Example 11.1. Consider the string $y = \text{abracadabra}$ with $r = 4$. The universe U consists of all substrings of length 4, i.e. $U = \{\text{abra, brac, raca, acad, cada, adab, dabr}\}$. Each substring in U must contain at least one pivot character to ensure coverage. For each character c , we define the set S_c , representing the set of all substrings of U containing a c . We have

$$\begin{aligned} S_a &= \{\text{abra, acad, adab, brac, cada, dabr, raca}\}, \\ S_b &= \{\text{abra, adab, brac, dabr}\}, & S_r &= \{\text{abra, brac, dabr, raca}\}, \\ S_c &= \{\text{acad, brac, cada, raca}\}, & S_d &= \{\text{acad, adab, cada, dabr}\}. \end{aligned}$$

To find the minimum pivot set C , we analyze the candidate sets S_c for each character occurring in y . Among these, S_a is particularly notable because it includes all substrings in U that contain the character **a**. As S_a alone covers every substring in U (i.e., $S_a = U$), selecting $C = \{\mathbf{a}\}$ is sufficient to ensure full coverage. This means that every substring of length $r = 4$ in y contains the character **a**, and no additional pivot characters are necessary.

However, while in a greedy way, the problem can be solved selecting only $C = \{\mathbf{a}\}$, a more optimal solution can be found by selecting $C = \{\mathbf{b}, \mathbf{c}\}$, which has less occurrences in the text while still respecting the set cover property.

The previous example illustrates a scenario in which a single pivot character is sufficient to cover all substrings in the universe U . However, there are instances

where a single pivot character is not adequate, requiring the selection of multiple pivot characters to ensure complete coverage. Consider the following example:

Example 11.2. Consider the string $y = \text{abcdeabcde}$ with $r = 4$. The universe U consists of all substrings of length 4, i.e. $U = \{\text{abcd}, \text{bcde}, \text{cdea}, \text{deab}, \text{eabc}\}$. Each character in y contributes to covering specific substrings in U , represented by the sets S_c :

$$\begin{aligned} S_a &= \{\text{abcd}, \text{cdea}, \text{deab}, \text{eabc}\}, & S_b &= \{\text{abcd}, \text{bcde}, \text{deab}, \text{eabc}\}, \\ S_c &= \{\text{abcd}, \text{bcde}, \text{cdea}, \text{eabc}\}, & S_d &= \{\text{abcd}, \text{bcde}, \text{cdea}, \text{deab}\}, \\ S_e &= \{\text{bcde}, \text{cdea}, \text{deab}, \text{eabc}\}. \end{aligned}$$

In this case the single pivot character **a** is insufficient to cover all substrings in U . By adding **b** to the pivot set alongside **a**, the substrings covered by S_b , including **bcde**, fill one of the gaps left by S_a . More formally we have $S_a \cup S_b = U$, so that $C = \{\mathbf{a}, \mathbf{b}\}$.

11.2 Optimizing Search Efficiency with Pivot-Free Patterns

An essential property of the pivot set derived from the Set Cover Problem is its ability to guarantee that every substring of the text of length $m \geq r$ contains at least one pivot character. This is particularly useful for efficiently handling patterns that contain no pivot characters. Specifically, if a search pattern p of length $m \geq r$ contains no pivot character, it can be immediately discarded as non-existent in the text.

Lemma 11.3. Let p be a pattern of length $m \geq r$. If $p \cap C = \emptyset$ (i.e., p contains no pivot character), then p does not occur in y .

Proof. Assume p occurs in y at position j , meaning $y[j \dots j + m - 1] = p$. The substring $s = y[j \dots j + r - 1]$ must contain at least one pivot character, contradicting the assumption that $p \cap C = \emptyset$. Therefore, p cannot occur in y . $\square$

This optimization allows immediate exclusion of many patterns during the search process, significantly reducing computational overhead, as confirmed by experimental results in Section 11.7.

11.3 Enhancing Space Efficiency through Distance Constraints

An additional key innovation of this approach is the reinterpretation of the parameter r , which is defined as the minimum size of the pattern being searched. If we

reinterpret this parameter as the maximum distance between two occurrences of a pivot character, we can leverage this value to obtain a more compact and efficient representation of the distances within the character-distance sampled sequence $\bar{y}$.

By setting $r < 256$, it is guaranteed that every distance $d \in \bar{y}$ satisfies $d \leq 256$, allowing these distances to be represented using just one byte per value instead of the standard 32-bit (4-byte) word typically used. Similarly, if we impose $r < 16$, only 4 bits are required, further halving the space used. This significantly reduces memory consumption and eliminates the complexity of managing fake positions, a requirement in prior methods.

Building on the ideas introduced in [14], this approach leverages the bounded nature of r to optimize memory usage while preserving the integrity of the representation. By constraining the maximum distance between consecutive pivots, it achieves efficient memory utilization without sacrificing substring coverage. Additionally, this method introduces a natural trade-off between the granularity of pivot selection and the size of the sampled representation, making it particularly well-suited for large-scale text processing tasks where both memory efficiency and processing speed are critical.

11.4 Extending Set Coverage for Condensed Alphabets

CDS performance can be constrained when applied to texts with small alphabets, where the diversity of pivot sets is inherently limited. As discussed in [13, 12], an effective way to address this limitation is by extending the CDS representation using a condensed alphabet, i.e. q -grams, which are sequences of q consecutive characters of Σ treated as a single symbol. By grouping characters into q -grams, the effective alphabet size increases, enabling a richer set of pivots and providing more flexibility in covering substrings.

To be precise, in [13], it was shown that the use of q -grams in texts with small alphabets, such as DNA sequences, can reduce the size of the text by up to 96% and speed up the search by up to 85% compared to using the text in its original structure.

This q -gram-based extension integrates seamlessly with the new method for identifying the sampled alphabet proposed in this paper. By leveraging an extended alphabet of q -grams, the Set Cover-based pivot selection process can achieve a more efficient and compact coverage.

It is important to recall that, in a q -gram representation of the text, the parameter q specifies the number of characters that form a single symbol in the extended alphabet. This extended alphabet, denoted as Σ^q , is formally defined as the set of all possible strings of length q that can be generated by concatenating characters from the original alphabet Σ . The text converted from Σ to Σ^q , denoted as y^q , is a

string of n q -grams.

A substring $s = y^q[i \dots i + m - 1]$ of length m in the q -gram representation corresponds to the substring $y[i \dots i + m + q - 2]$ in the original text y , since the last q -gram of the string also consists of q characters. To ensure consistency with this representation, $q - 1$ copies of a special character $\$$ are appended to the end of the string, resulting in $y^q = y \cdot \$^{q-1}$. Observe that, in this context, the last symbol of y^q is $y^q[n] = y[n] \cdot \$^{q-1}$.

For any value of the parameter q , the set of pivots that covers all substrings of length r in y^q can be determined by solving the set cover problem, as previously discussed. Alternatively, an incremental approach can be employed to extend the sampled alphabet computed for the string y^q to the sampled alphabet for the string $y^{(q+1)}$. In other words, a systematic process can be employed to transform the set of pivots for the universe of q -grams into the set of pivots for the universe of $(q + 1)$ -grams. This process is based on the following lemma.

Lemma 11.4 (*q -grams Extension of a Coverage Pivot Set*). Let y be a string of length n over an alphabet Σ , and assume $C_q \subseteq \Sigma^q$ is a pivot set that forms a valid set cover for the string y^q . Then, the set C_{q+1} , constructed by appending every character $c \in \Sigma \cup \{\$\}$ to each pivot $p \in C_q$, forms a valid set cover for the string y^{q+1} .

Proof. By assumption, C_q is a valid set cover for y^q , meaning that for every substring s' of length r in y^q , there exists at least one q -gram $p \in C_q$ such that p occurs in s' . To construct C_{q+1} , we extend each q -gram $p \in C_q$ by appending every character $c \in \Sigma \cup \{\$\}$, resulting in: $C_{q+1} = \{p \cdot c \mid p \in C_q \text{ and } c \in \Sigma \cup \{\$\}\}$, where $p \cdot c$ denotes the concatenation of p with c .

Now, consider any substring s of length r in $y^{(q+1)}$, and let it start at position i . This substring s can be expressed as the concatenation of a substring s' of length r in y^q (starting at the same position i) with the character $y[i + m + q - 2]$. By assumption, there exists a q -gram $p \in C_q$ that occurs in s' .

Let p be followed by the character a in y . By construction, C_{q+1} contains the q -gram p concatenated with every character $c \in \Sigma \cup \{\$\}$, including the character a . Hence, the substring s is covered by C_{q+1} . This completes the proof. $\square$

It is easy to observe that the size of the sampled text obtained from y^q using the set C_q is the same as that of the sampled text obtained from $y^{(q+1)}$ using the set C_{q+1} .

However, while this extension produces a pivot set that adequately covers all substrings of $y^{(q+1)}$ of length at least r , it is important to note that this set does not necessarily guarantee optimality, meaning it may not have the minimum cardinality. As a result, some elements of C_{q+1} could be redundant.

It is possible, however, to perform a pass over the text y to identify any elements in C_{q+1} that never occur in y or that never appear in isolation within a substring

of length r . Such elements can be removed from C_{q+1} without compromising the coverage of the text.

As a result, at the end of the extension and redundancy elimination process, the sampled text obtained from $y^{(q+1)}$ has a size that is equal to or smaller than the size of $\bar{y}^q$.

Example 11.5. Consider the string $y = \text{abracadabra}$ with $r = 4$. The universe U of all substrings of length 4 is $U = \{\text{abra}, \text{brac}, \text{raca}, \text{acad}, \text{cada}, \text{adab}, \text{dabr}, \text{abra}\}$. In the 1-gram scenario, a valid set cover might consist of the pivot $\{\text{a}\}$. To generate a 2-gram pivot set, we derive all 2-grams starting with the 1-gram pivot a . For instance, if $\Sigma = \{\text{a}, \text{b}, \text{c}, \text{d}, \text{r}\}$, the 2-grams derived are $\{\text{aa}, \text{ab}, \text{ac}, \text{ad}, \text{ar}\}$. To ensure full coverage near the end of y , where incomplete 2-grams occur, we append a placeholder symbol $\text{\$}$, resulting in $y' = \text{abracadabra}\text{\$}$. The updated set cover becomes $\{\text{aa}, \text{ab}, \text{ac}, \text{ad}, \text{ar}, \text{a}\text{\$}\}$. We observe that not all 2-grams in this set are necessary for full coverage. For example, $\text{acad} \in U$ is covered by ac , rendering ad redundant. Removing ad yields a more compact pivot set: $\{\text{aa}, \text{ab}, \text{ac}, \text{ar}, \text{a}\text{\$}\}$. This final set optimizes the representation by avoiding redundancy while maintaining coverage.

11.4.1 Complexity Issues

This section addresses the computational complexity of the proposed methods, analyzing both theoretical and practical aspects. Subsection 11.5 focuses on the space consumption associated with the exact construction of the sampled alphabet using the set cover formulation. Subsection 11.6, on the other hand, explores efficient approximations and trade-offs that balance coverage quality with computational efficiency. Together, these analyses provide a comprehensive view of the complexity challenges and the scalability of the proposed approach.

11.5 On the Space Complexity

Regarding the space complexity required to store the sampled text, we recall that the size of the sampled text in the CDS representation, which we denote by ρ , is determined by the number of all occurrences in y of any pivot character.

The best-case scenario occurs when each pivot character is able to cover the maximum possible number of substrings of the text, which is r substrings. Specifically, a pivot character located at position i in the text covers the r substrings starting at positions in the range $[i - r + 1, i]$. To achieve complete coverage of all substrings of length r , it follows that there must be a pivot character for every r characters of the text. This ensures that every substring of length r in the text contains at least one pivot character. Consequently, the number of pivot characters required is minimized and given by $\rho = \lceil n/r \rceil$, where the ceiling function accounts for cases

where n is not an exact multiple of r . Such a best-case occurs, for example, for the string $y = [a^{r-1}b]^k$, of length $n = rk$, for which we have $C = \{b\}$.

Conversely, the worst-case scenario for substring coverage arises when, in order to ensure that all substrings of length r are covered, every character of the string y must be selected as a pivot. This results in a maximum value for ρ , equal to the total length of the string, i.e., $\rho = n$. This situation occurs, for example, with the string $y = [a^r b^r]^k$, where $n = 2kr$. In this case, the substring $y[0..r-1] = a^r$ requires the character a to be a pivot, while the substring $y[r..2r-1] = b^r$ requires the character b to be a pivot. Consequently, $C = \{a, b\}$, leading to $\rho = n$.

However, such a case is uncommon, as the experimental results presented in the following Section 11.7 demonstrate that the average case tends to align more closely with the optimal scenario than with the worst-case scenario, resulting in significantly more efficient pivot selection.

11.6 On the Construction of the Sampled Alphabet

Regardless of the optimal composition of the set that achieves coverage of the universe U of substrings of length r in the text, the construction of the sampled alphabet can be performed either using an exact computation of the set (e.g., through a naïve approach) or through a more efficient computation that, however, provides a suboptimal solution.

The exact solution explores all possible combinations of characters in the text to identify the smallest subset that ensures complete coverage of all substrings of length r . Specifically, the algorithm evaluates every subset of characters in the text, verifying whether it satisfies the coverage requirement. However, we remember that, while this method guarantees the optimal solution, it has exponential complexity in the worst case. Due to this computational overhead, the exact algorithm is impractical for large texts. It should also be highlighted that this algorithm is typically used exclusively during a preprocessing phase, executed once on the text prior to performing multiple search queries. Therefore, for texts of moderate length, the computational cost of the exact algorithm can be distributed across subsequent query operations, particularly in scenarios where the text remains static and is queried repeatedly.

More specifically, the computational complexity of the exact algorithm for computing the sampled alphabet is given by $\mathcal{O}(2^\sigma \cdot |U| \cdot r) = \mathcal{O}(2^\sigma nr)$, where σ denotes the size of the alphabet Σ , r represents the length of the text substrings, and U is the set of all distinct substrings of length r . Thus, the term 2^σ accounts for all possible subsets of Σ .

A greedy algorithm exists for approximating the Set Cover Problem in polynomial time [53]. The algorithm operates by selecting sets based on the following

rule: at each step, it chooses the set that covers the largest number of uncovered elements. This approach can be efficiently implemented in linear time with respect to the total size of the input sets by utilizing a bucket queue for set prioritization. The algorithm achieves an approximation ratio of $H(|U|)$ [54, 55]. In other words, the solution produced by the algorithm may be at most $H(|U|)$ times larger than the optimal solution, where $H(n)$ is the n -th harmonic number, defined as $H(n) = \sum_{k=1}^n \frac{1}{k} \leq \ln n + 1$.

Although this approach does not guarantee the minimal number of pivot characters, it is significantly faster than the exact algorithm and often produces results that are near-optimal in practice. The greedy algorithm is particularly suitable for large-scale applications where preprocessing time is critical. However, the trade-off lies in the potential for selecting more pivot characters than strictly necessary, which can increase the size of the sampled text.

If we implement the priority queue using a max-heap, the computational complexity of the greedy algorithm is $\mathcal{O}(\sigma \cdot (\log \sigma + |U|)) = \mathcal{O}(\sigma \cdot (\log \sigma + n))$, where $\log \sigma$ represents the complexity of re-heapifying during the extraction operation, and $|U| = \mathcal{O}(n)$ denotes the size of the universe U , accounting for the set coverage updates after each character extraction. The first term reflects the repeated extraction and re-heapification operations, while the third term captures the cost of updating the covered sets after each extraction.

11.7 Experimental Results

This section presents an experimental evaluation of the variant of the Character Distance Sampling method proposed in this paper, comparing it with its standard version. The goal is to highlight its ability to create a compact text representation while ensuring complete text coverage, as well as to improve the performance of pattern search within the text.

Specifically, we implemented the most efficient version of the CDS available in the literature [14], which employs fake samples, as well as two versions of the CDS variant proposed in this chapter: one that constructs the pivot set using an exact algorithm and another that employs a greedy approach for pivot set construction. Moreover, in line with the approach commonly adopted in the literature, all sampling algorithms were designed to use the Horspool’s algorithm [41] as the underlying search procedure.

Each buffer was reduced to a size of 1 MB to allow the execution of the procedure for the exact construction of the pivot set, which has exponential time complexity. The experiments were conducted for varying values of the parameter r , ranging from 8 to 256. For each of the two text buffers, a search was performed for 1,000 patterns of length r . For each value of r , the 1,000 patterns were randomly selected from the text. The pattern set used for the search was identical across all three methods compared.

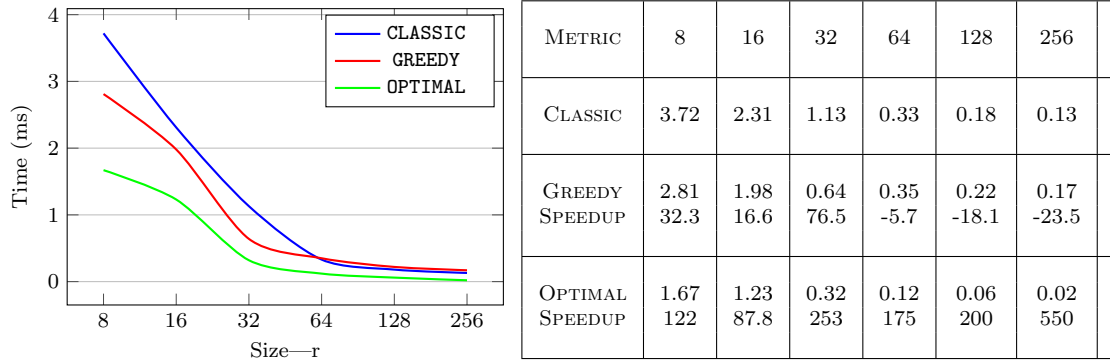


Figure 11.1: Execution times of the algorithms for various input sizes. The x-axis represents both the pattern size and the value of r , while the y-axis shows the execution time in milliseconds. The three curves represent the performance of the three different algorithms.

Experimental results obtained on the natural language text are shown in Figure 11.1. From the plot it turns out that the refined search strategy proves substantial performance improvements. By ensuring that every substring of length r contains at least one pivot character, the CDS method eliminated unnecessary validation checks for patterns lacking pivot characters, Being between 90 and 550 times faster than the baseline approaches. Additionally, the optimal method turns out to be 80 times faster than the greedy approach by reducing redundant candidate occurrences during validation. This demonstrates the efficiency of the Set Cover method, achieving superior performance over both classical and greedy, particularly as r increases.

Table 11.1 provides detailed metrics for the above results, including the percentage of pivots selected, worst-case occurrences and longest distances.

Specifically, the size of the sampled text, which corresponds to the frequency of pivot characters within the text, slightly increases in the approach based on the greedy strategy but decreases significantly in the approach based on the exact algorithm, reaching an extraordinary 1% for particularly long patterns. This is achieved despite the substantial improvement in search times. The dramatic improvement in search times is further justified by the substantial reduction in the percentage of occurrences where the search falls into the worst-case scenario, which, in the standard method, can reach up to 94% for very short patterns.

A significant observation during the experiments was the high frequency of the space character in natural language texts, often exceeding 15% of the total text. While the space character frequently serves as a feasible set cover for the greedy solution when $r \geq 16$, it is suboptimal for CDS representation due to oversampling. With the expected sampling rate for $r = 16$ at approximately $100/r \approx 6.25\%$, relying on the space character as a primary pivot introduces redundancy. Instead, an optimal pivot set incorporates multiple characters with lower frequencies to balance

Table 11.1: Experimental results obtained from 1000 executions over a natural language text.

PIVOT SET COMPUTAT.	r	PIVOT FREQUENCY		WORST-CASE FREQUENCY	SAMPLING SPACE RATIO	LONGEST DISTANCE	SELECTED PIVOT SET
		REAL	BEST				
CLASSIC	8	17.3%	12.5%	94%	17.3%	13	{\s}
	16	12.2%	6.25%	87%	12.2%	121	{t}
	32	12.2%	3.13%	62%	12.2%	121	{t}
	64	8.8%	1.6%	32%	8.8%	382	{a}
	128	7.3%	0.8%	11%	7.3%	484	{o}
	256	6.7%	0.4%	6%	6.7%	556	{h}
GREEDY	8	22.8%	12.5%	0%	22.8%	6	{\s, i}
	16	17.3%	6.25%	0%	17.3%	13	{\s}
	32	17.3%	3.13%	0%	17.3%	13	{\s}
	64	9.49%	1.6%	0%	9.49%	57	{e}
	128	9.49%	0.8%	0%	9.49%	57	{e}
	256	9.49%	0.4%	0%	9.49%	57	{e}
OPTIMAL	8	13.7%	12.5%	0%	13.7%	8	{a, s, p, A, k, v, B, \n}
	16	9.4%	6.25%	0%	9.4%	16	{h, d, g, y, .}
	32	4.1%	3.13%	0%	4.1%	32	{c, m, U, \n}
	64	2.0%	1.6%	0%	2.0%	64	{z, y, f, c}
	128	1.1%	0.8%	0%	1.1%	128	{b, z, k, \n}
	256	1.1%	0.4%	0%	1.1%	128	{b, z, k, \n}

Figure 11.2: Execution times of the algorithms for q -gram ($q = 2$) results. The x-axis represents both the pattern size and the value of r , while the y-axis shows the execution time in milliseconds. The three curves represent the performance of the three different algorithms applied to q -gram data.

substring coverage and reduce sampled text size.

It is also important to note that, unlike previous representations requiring 4 bytes to store each exact position, and the fake representation introduced in [14], which relies on approximating positions to reduce memory usage, the new approach presented here eliminates the need for both exact and fake positional data. This advancement significantly streamlines the representation by focusing solely on the distances between pivots, using 8 bits for $16 < r \leq 256$ and using only 4 bits for $r \leq 16$.

To evaluate the applicability of new presented approach in specialized domains,

Table 11.2: Experimental results obtained from 1000 executions over a Protein Sequence.

PIVOT SET COMPUTAT.	r	PIVOT FREQUENCY		WORST CASE	SPACE RATIO	LONGEST DISTANCE	SELECTED PIVOT SET
		REAL	BEST				
CLASSIC	8	18.2%	12.5%	78%	18.2%	345	{LL, SS, SL, LS, LA, AA, VL}
	16	16.8%	6.25%	65%	16.8%	457	{LL, SS, SL, LS, LA}
	32	13.7%	3.13%	57%	13.7%	398	{LL, SS, SL, LS, LA}
	64	10.0%	1.6%	44%	10.0%	573	{LL, SS, SL, LA}
	128	8.1%	0.8%	36%	8.1%	712	{SS, SL, LA}
	256	8.1%	0.4%	32%	8.1%	712	{SS, SL, LA}
GREEDY	8	22.3%	12.5%	0%	22.3%	7	{LL, SS, SL, LS, LA, AA, AL, AS, LG, VL}
	16	18.4%	6.25%	0%	18.4%	12	{LL, SS, SL, LS, LA, AS, AL}
	32	18.4%	3.13%	0%	18.4%	12	{LL, SS, SL, LS, LA, AS, AL}
	64	9.2%	1.6%	0%	9.2%	49	{LL, SS, SL}
	128	9.2%	0.8%	0%	9.2%	49	{LL, SS, SL}
	256	9.2%	0.4%	0%	9.2%	49	{LL, SS, SL}
OPTIMAL	8	14.1%	12.5%	0%	14.1%	7	{LL, LV, LE, KL, AV, AA, AI, AL}
	16	9.2%	6.25%	0%	9.2%	13	{LL, VL, LV, GL, IL, AV, AA, AI, AL, LE}
	32	4.3%	3.13%	0%	4.3%	32	{VI, GL, IL, LG, IA, LL, AL, LS}
	64	2.4%	1.6%	0%	2.3%	57	{LG, LS, IP, TG, LI, LE}
	128	1.3%	0.8%	0%	1.3%	116	{IP, KI, TG, GL}
	256	1.3%	0.4%	0%	1.3%	116	{IP, KI, TG, GL}

we also conducted experiments on a protein sequence corpus. This dataset presented unique challenges, particularly the small alphabet size (20–25 symbols) and the uneven distribution of character frequencies. Small alphabets, such as those found in protein sequences or genomes (4-character alphabets), are well-known to represent an impractical scenario for sampling methods [9]. In such cases, the high frequency of individual characters in equidistributed datasets means that selecting multiple pivots (e.g., 2 or 3 characters) could result in covering 40–50% or more of the text. This leads to suboptimal sampling and diminishes the benefits of search optimizations. To overcome this limitation, we investigated the use of q -grams (for simplicity we used only $q = 2$) to artificially expand the effective alphabet size. Experimental results shown in Figure 11.2 demonstrate that incorporating q -grams as potential pivots significantly reduces redundancy in character sampling while enhancing coverage efficiency.

From experimental results (see also Table 11.2), it turns out that the execution times for the three algorithms applied to q -gram ($q = 2$) datasets highlight the superior performance of the optimal pivot selection, which consistently achieves the lowest execution times compared to classical and greedy methods. The performance gap widens as the size and r value increase, underscoring the scalability and efficiency of the optimal method for q -gram applications and its potential for handling datasets with challenging characteristics, including small alphabets or high redundancy. It is also reasonable to assume that using a higher value of q could lead to results closer to the optimal ones.

Monotonic Run Length Scaling

The main goal of a text sampling approach is to identify locally distinctive features of the text that, on the one hand, allow easy identification of candidate occurrences of the pattern and, on the other hand, reduce the possibility of false positives as much as possible.

In this section we present an approach of sampled string matching based on the length of the *monotonic runs* identified in the input text. Informally, a monotonic run is a sequence of contiguous characters arranged in non-increasing or non-decreasing order. Compared to the sequence of distances between consecutive positions of the same pivot character, the sequence of the lengths of the monotonic runs in the text represents a more distinctive feature, which keeps track of the contiguity of the characters of the text and the relationship of this contiguity, rather than the frequency of a single character. This approach to text sampling allows to access more information about the text windows and is consequently more effective in the context of sampled string matching.

In order for monotonic runs to be defined in a text, it is therefore necessary that the underlying alphabet is composed of characters for which a total ordering relationship is defined. Below we give the formal definition of a monotonic run.

Definition 12.1 (Monotonic Run). Let y be a text of length n over a finite and totally ordered alphabet Σ of size σ . A Monotonic Increasing Run of y is a not extendable substring w of y whose elements are arranged in increasing order. Formally, if $w = y[i..i + k - 1]$ is a monotonic increasing run of y of length k , we have:

- $w[j - 1] < w[j]$, for each $0 < j < k$;
- $y[i] \leq y[i - 1]$;
- $y[i + k - 1] \geq y[i + k]$.

Symmetrically a Monotonic Non-Increasing Run of y is a not extendable substring w of y whose elements are arranged in non-increasing order. Formally, if $w = y[i..i + k - 1]$ is a monotonic non-increasing run of y of length k , we have:

- $w[j - 1] \geq w[j]$, for each $0 < j < k$;

- $y[i] > y[i - 1]$;
- $y[i + k - 1] < y[i + k]$.

We will indicate with the general term Monotonic Run any substring of y that can be either monotonic increasing and monotonic non-increasing.

By Definition 12.1 two adjacent monotonic substrings of a string have a single overlapping character, i.e. the rightmost character of the first substring is also the leftmost character of the second substring.

Example 12.2. Let $y = \langle 4, 5, 11, 7, 6, 6, 12, 12, 2, 9, 8, 6, 7, 10, 13 \rangle$ be a numeric sequence of length 15. We can identify the following monotonic runs in y :

- $y[0..2] = \langle 4, 5, 11 \rangle$ is a monotonic increasing run;
- $y[2..5] = \langle 11, 7, 6, 6 \rangle$ is a monotonic non-increasing run;
- $y[5..6] = \langle 6, 12 \rangle$ is a monotonic increasing run;
- $y[6..8] = \langle 12, 12, 2 \rangle$ is a monotonic non-increasing run;
- $y[8..9] = \langle 2, 9 \rangle$ is a monotonic increasing run;
- $y[9..11] = \langle 9, 8, 6 \rangle$ is a monotonic non-increasing run;
- finally, $y[11..14] = \langle 6, 7, 10, 13 \rangle$ is a monotonic increasing run.

We give the following definition of *Monotonic Run Length Scaling* (MRLX) of a string y as the process of decomposing the string in a set of adjacent monotonic runs. The resulting sequence, which we call *monotonic run length scaled version* of y , is the numeric sequence of the lengths of the monotonic runs given by the MRLX process.

Definition 12.3 (Monotonic Run Length Scaling). Let y be a text of length n over a finite and totally ordered alphabet Σ of size σ . Let $\langle \rho_0, \rho_1, \dots, \rho_{k-1} \rangle$ the sequence of adjacent monotonic runs of y such that ρ_0 starts at position 0 of y and ρ_{k-1} ends at position $n - 1$ of y . The monotonic run length scaled version of y , indicated by $\tilde{y}$, is a numeric sequence, defined as

$$\tilde{y} = \langle |\rho_0|, |\rho_1|, \dots, |\rho_{k-1}| \rangle. \quad (12.1)$$

It is straightforward to prove that there exists only one unique monotonic run length scaled version of a given string y . In addition we observe that

$$\left[\sum_{i=0}^{k-1} |\rho_i| \right] - k + 1 = n \quad (12.2)$$

Function 1: Monotonic-Run-Length-Scaling(x, m)

Data: a string x of length m
Result: The Scaled version $\tilde{x}$ of x
 $\tilde{x} \leftarrow \langle \rangle$;
 $\mu \leftarrow 2$;
 $d \leftarrow 1$;
if ($x[1] - x[0] \leq 0$) **then** $d \leftarrow -1$;
 $i \leftarrow 2$;
while ($i < n$) **do**
 if ($(d = 1 \text{ and } x[i] - x[i-1] > 0) \text{ or } (d = -1 \text{ and } x[i] - x[i-1] \leq 0)$)
 then
 $i \leftarrow i + 1$;
 $\mu \leftarrow \mu + 1$;
 else
 $\tilde{x} \leftarrow \tilde{x} + \langle \mu \rangle$;
 $\mu \leftarrow 1$;
 $d \leftarrow d \times -1$;
 end
 end
 $\tilde{x} \leftarrow \tilde{x} + \langle \mu \rangle$;
return $\tilde{x}$

Example 12.4. As in Example 12.2, let $y = \langle 4, 5, 11, 7, 6, 6, 12, 12, 2, 9, 8, 6, 7, 10, 13 \rangle$ be a numeric sequence of length 15. Then the scaled version of y is the sequence $\tilde{y} = \langle 3, 4, 2, 3, 2, 3, 4 \rangle$ of length 7. We have therefore that $3+4+2+3+2+3+4-7+1 = 15$.

Function 1 depicts the pseudo-code of the algorithm which computes the Monotonic Run Length Scaling version of an input string x of length m . It constructs the sequence $\tilde{x}$ incrementally by scanning the input string x character by character, proceeding from left to right. During text scanning, Function 1 keeps the length of the current monotonic run within the μ variable. The length of the monotonic run is incremented if the next character maintains the same increasing (or decreasing) trend as the current run. The trend is identified by the variable d , whose value is equal to 1 in the case of an increasing monotonic run, or -1 in the case of a non-increasing monotonic run. It is straightforward to prove that its time complexity is $\mathcal{O}(m)$.

Figure 12.1 shows the average and maximal length of a monotonic run on 500 random texts of 100 MB, over an alphabet of size 2^δ , with $2 \leq \delta \leq 8$, and where the Y-axis shows the length values while the X-axis shows the values of δ .

We point out that, while the length of an increasing monotonic run is always limited by the size of the alphabet, this does not apply to the length of a non-decreasing monotonic run, since the presence of an indefinite number of contiguous

characters of equal value allows to obtain runs with a length not limited above. Despite this point we observe that, in practice, the length of each monotonic run never exceeds ten characters on a random text with an alphabet of size $\sigma \leq 8$. Furthermore, we observe that the average length of the monotonic substrings is much lower being always between 1.10 (for small alphabets) and 1.50 (for large alphabets). This implies that the average length of the monotonic run length scaled version of the text is on average between 66% and 90% of the length of the original sequence, a result not particularly exciting when compared with those obtained by previous sampling techniques such as CDS and OTS that goes from 5% to 28%.

12.1 On the Average Length of a Monotonic Run

In this section, we present a theoretical analysis of the Monotonic Run Length Sampling approach, assuming that the sampled text is generated randomly from a finite and ordered alphabet of size σ . To illustrate our thesis, we model this process as a game involving the rolling of a dice. Specifically, a string of characters of length n from an alphabet of Σ characters is represented by a sequence of n rolls of a σ -sided die.

Our analysis is structured around two primary results: the average length of a non-decreasing monotonic run and the average length of a strictly increasing monotonic run. Additionally, we will conclude our analysis by determining the average length of a monotonic run derived from the decomposition of a text.

For the sake of completeness, we observe that the assumptions of our theoretical analysis might not fully capture the dynamics of all monotonic runs in a sequence. Specifically, while our model assumes a uniform probability distribution for the starting character of each run, this assumption holds strictly for the first run. For subsequent runs, the starting character distribution is influenced by the structure of the preceding run. For example, in the case of binary strings, every increasing run is always of length 2, leading to a deviation from the results presented in this section. To maintain the simplicity of our theoretical framework, we have omitted this dependency in our analysis. Consequently, as highlighted later in the experimental results, slight discrepancies may emerge between the theoretical predictions and the empirical findings.

Monotonic Non-Increasing Runs. From a mathematical perspective, a σ -sided die is represented by a discrete random variable uniformly distributed over the set $\{1, 2, \dots, \sigma\}$, meaning each outcome has an equal probability of $1/\sigma$. Instead of single rolls, our focus is on the sequences of non-increasing outcomes from repeated rolls. This leads us to the following definition. Given any random variable X , we define a corresponding random variable $R(X)$, the run-length variable for X , which

counts the number of rolls until the first decrease occurs, including the final roll.¹ We refer to X as the underlying variable of $R(X)$.

The notation $R(X)$ highlights the dependency on the underlying variable X , but we will often omit X when the context is clear. Specifically, we denote the run-length variable as R_σ when X is a σ -sided die roll.

Our objective is to determine the expected value of R , particularly for a σ -sided die roll. Informally, the expected value of a random variable is the long-term average of its outcomes. By definition, the expected value of R is given by:

$$E[R] = \sum r f(r),$$

where the summation is over all possible outcomes r , and $f(r)$ represents the probability of obtaining the outcome r .

Regardless of the underlying variable, the minimum value for R is 2. However, there is no upper limit to the length of a non-increasing sequence, making the possible outcomes of R the set $\{2, 3, 4, \dots\}$. Therefore, the expected value can be rewritten with explicit summation limits:

$$E[R] = \sum_{r=2}^{\infty} r f(r).$$

Next, we derive an explicit formula for $f(r)$ for a σ -sided die roll. Let $\sigma \geq 2$, and let $a_\sigma(r)$ be the number of non-increasing sequences of length r that can be formed from the set $\{1, 2, \dots, \sigma\}$. This problem, known as selection with repetition, is addressed in combinatorics and is represented by the binomial coefficient:

$$a_\sigma(r) = \binom{\sigma + r - 1}{\sigma - 1}.$$

Using this, the probability that a sequence of length r is non-increasing is $\frac{a_\sigma(r)}{\sigma^r}$. Thus, the probability that a sequence of length r first decreases at the last position is the difference between the probabilities of it being non-increasing for $r - 1$ and r steps:

$$f_\sigma(r) = \frac{a_\sigma(r-1)}{\sigma^{r-1}} - \frac{a_\sigma(r)}{\sigma^r},$$

which simplifies to

$$f_\sigma(r) = \binom{\sigma + r - 2}{r - 1} \cdot \frac{r - 1}{\sigma^r}.$$

¹Since we also take into account the first roll for which the sequence decreases in our average analysis, we should subtract one unit from our expected result.

With the probability mass function established, we can proceed to calculate the expected value as:

$$E[R_\sigma] = \sum_{r=2}^{\infty} r \cdot \binom{\sigma + r - 2}{r - 1} \cdot \frac{r - 1}{\sigma^r}.$$

This sum can be transformed as follows:

$$E[R_\sigma] = \sum_{r=2}^{\infty} r \cdot \frac{(\sigma + r - 2)!}{(r - 1)!(\sigma - 1)!} \cdot \frac{r - 1}{\sigma^r}.$$

Rewriting, we obtain:

$$E[R_\sigma] = \sum_{r=2}^{\infty} \frac{r \cdot (\sigma + r - 2)! \cdot (r - 1)}{(r - 1)! \cdot (\sigma - 1)! \cdot \sigma^r}.$$

Simplifying, we get:

$$E[R_\sigma] = \sum_{r=2}^{\infty} \frac{(\sigma + r - 2)!}{(r - 2)! \cdot (\sigma - 1)! \cdot \sigma^r}.$$

This can be further simplified to:

$$E[R_\sigma] = \frac{1}{(\sigma - 1)!} \sum_{r=2}^{\infty} \frac{(\sigma + r - 2)!}{(r - 2)! \cdot \sigma^r}.$$

Recognizing the series expansion of the binomial theorem, we have:

$$E[R_\sigma] = \frac{\sigma(\sigma - 1)}{\sigma - 1} \sum_{r=0}^{\infty} \binom{\sigma + r - 1}{r} \left(\frac{1}{\sigma}\right)^{r+2}.$$

This series converges to:

$$E[R_\sigma] = \left(\frac{\sigma}{\sigma - 1}\right)^\sigma.$$

We can then conclude that the average length of a non-increasing monotonic run in a text over a finite alphabet of size σ is $\left(\frac{\sigma}{\sigma - 1}\right)^\sigma - 1$.

Monotonic Increasing Runs. Let us consider a variant of the die-rolling game where the sequence must be strictly increasing. For example, if we roll:

1, 3, 5, 5

the game ends with a score of 4, as the sequence does not continue to strictly increase.

We can derive results similar to those obtained for non-increasing sequences for this strictly increasing variant. The calculation of the expected value here is somewhat simpler, as it utilizes the more familiar form of the binomial theorem with a positive integer exponent.

For any random variable X , define another random variable $R_s(X)$ as follows: we sample X until we obtain a result that is not strictly greater than the previous result, then $R_s(X)$ represents the total number of samples taken. If X represents a σ -sided die roll, the probability mass function for $R_s(X)$ is:

$$f_s(r) = \binom{\sigma + 1}{r} \cdot \frac{r - 1}{\sigma^r}.$$

where the method for this derivation is very similar to the non-decreasing case. When X represents an σ -sided die roll, the expected value of $R_s(X)$ can be expressed as:

$$E[R_s(X)] = \left(\frac{\sigma + 1}{\sigma} \right)^\sigma.$$

To verify this, observe that in the strictly increasing game, the possible scores range from 2 to $\sigma + 1$. Thus, the expected value calculation involves a finite sum:

$$E[R_s] = \sum_{r=2}^{\sigma+1} r \cdot \binom{\sigma + 1}{r} \cdot \frac{r - 1}{\sigma^r}.$$

This simplifies to:

$$E[R_s] = \left(\frac{\sigma + 1}{\sigma} \right) \sum_{r=0}^{\sigma-1} \binom{\sigma - 1}{r} \left(\frac{1}{\sigma} \right)^r.$$

By applying the binomial theorem, we find:

$$E[R_s] = \left(\frac{\sigma + 1}{\sigma} \right) \left(1 + \frac{1}{\sigma} \right)^{\sigma-1},$$

which further simplifies to:

$$E[R_s] = \left(\frac{\sigma + 1}{\sigma} \right)^\sigma.$$

We can then conclude that the average length of an increasing monotonic run in a text over a finite alphabet of size σ is $((\sigma + 1)/\sigma)^\sigma - 1$.

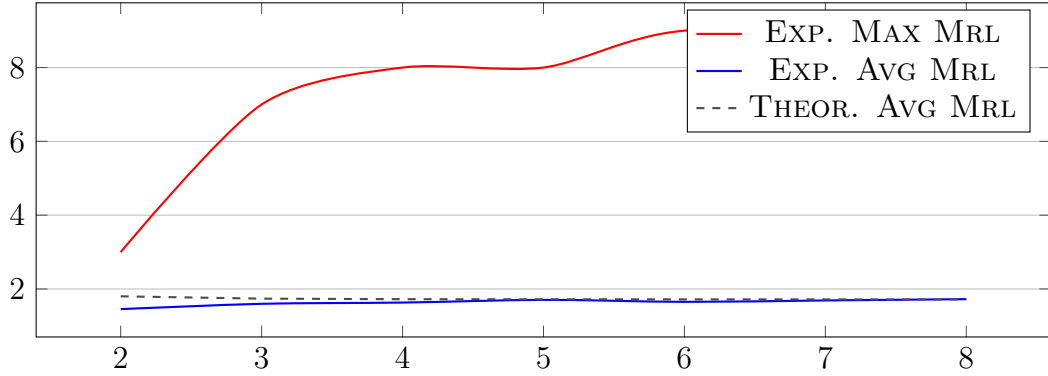


Figure 12.1: Average and maximal length of a monotonic run on a random text over an alphabet of size 2^δ . The Y-axis shows the length values while the X-axis shows the values of δ .

Final Analysis Finally, we are interested in computing the average length of the monotonic runs of the sampled text. Since sampling involves the identification, alternately, of a non-increasing monotonic run and an increasing monotonic run, we can state that the average length of each run is:

$$\frac{\left(\frac{\sigma}{\sigma-1}\right)^\sigma + \left(\frac{\sigma+1}{\sigma}\right)^\sigma}{2} - 1 \quad (12.3)$$

Fig. 12.1 shows the average and maximal length of a monotonic run on a random text over an alphabet of size 2^δ , together with the theoretical values based on function (12.3). In Fig. 12.1, the Y-axis shows the length values while the X-axis shows the values of δ .

12.2 Searching Using Monotonic Run Length Scaling

In this section we discuss the use of monotonic run length scaling as a sampling approach with application to classical exact and non classical exact string matching. For the case of non classical exact string matching, we take the Order Preserving Pattern Matching and Cartesian Tree Pattern Matching problems [16, 25, 56, 57, 58, 26], as a case study.

Specifically, we present a naïve searching procedure designed to use the monotonic scaled version of the text as a partial index in order to speed up the search for the occurrences of a given pattern within the original text. Like any other solution based on text-sampling, the solution proposed in this section requires that the partial index is used in a preliminary filtering phase and that each candidate occurrence

Algorithm 4: Naïve algorithm based on Monotonic Run Length Scaling

Data: a pattern x of length m , a text y of length n and its scaled version $\tilde{y}$ of length k

Result: all positions $0 \leq i < n$ such that x occurs in y starting at position i

$\tilde{x} \leftarrow \text{Monotonic-Run-Length-Scaling}(x, m);$

$h \leftarrow |\tilde{x}|;$

$d \leftarrow \tilde{x}[0];$

$r \leftarrow \tilde{y}[0];$

for $s \leftarrow 1$ **to** $k - h$ **do**

$j \leftarrow 1;$

while $j < h - 1$ **and** $\tilde{x}[j] = \tilde{y}[s + j]$ **do** $j \leftarrow j + 1;$

if ($j = h - 1$ **and** $\text{Verify}(y, n, x, m, r - d)$) **then**

 | Output($r - d$)

end

$r \leftarrow r + \tilde{y}[s] - 1;$

end

identified in this first phase is then checked within the original text using a simple verification procedure.

The choice of using a naïve algorithm for the search phase was mainly dictated by the need to maintain, during the access to a specific position of the sampled sequence, the information related to the corresponding position in the original text, necessary for the verification phase of a possible candidate occurrence.

The pseudo-code of the naïve searching procedure is presented in Algorithm 4. The preprocessing phase of the algorithm computes the monotonic run length scaled version $\tilde{x}$ of the input pattern x . Let k be the length of the sequence $\tilde{y}$ and let h be the length of the sequence $\tilde{x}$. In addition let d be the length of the first monotonic run of x , i.e. $d = \tilde{x}[0]$.

During the searching phase the algorithm naively searches for all occurrences of the substring $\tilde{x}[1..h - 2]$ along the scaled version of the text. We discard the first and the last element of $\tilde{x}$ ($\tilde{x}[0]$ and $\tilde{x}[h - 1]$, respectively) since they are allowed to match any element in the text which is greater than or equal.

The main **for** loop of the algorithm iterates over a shift value s , which is initialized to 1 at the first iteration and is incremented by one when passing from one iteration to the next. During each iteration the current window of the text $\tilde{y}[s..s + h - 3]$ is attempted for a candidate occurrence. An additional variable r is maintained, representing the shift position in the original text y corresponding to the current window $\tilde{y}[s..s + h - 3]$. Thus at the beginning of each iteration of the main **for** loop the following invariant holds:

$$r = \tilde{y}[0] + \sum_{i=1}^{s-1} (\tilde{y}[i] - 1).$$

At each iteration the window of the text $\tilde{y}[s..s+h-3]$ is compared, character by character, against the substring $\tilde{x}[1..h-3]$, proceeding from left to right. If a match is found then a candidate occurrence of the pattern is located at position $r-d$ of the original text and a verification phase is called in order to check if the substring $y[r-d..r-d+m-1]$ corresponds to a full occurrence of the pattern x . In all cases at the end of each attempt the value of r is increased by $\tilde{y}[s]$ and the value of the shift s is increased by one position.

Example 12.5. As in Example 12.2, let $y = \langle 4, 5, 11, 7, 6, 6, 12, 12, 2, 9, 8, 6, 7, 10, 13 \rangle$ be a numeric sequence of length 15 (the text) and let $x = \langle 12, 2, 9, 8, 6, 7, 10 \rangle$ be a pattern of length 7. Then we have $\tilde{y} = \langle 3, 4, 2, 3, 2, 3, 4 \rangle$ and $\tilde{x} = \langle 2, 2, 3, 3 \rangle$, with $d = 2$.

The algorithm naively searches the text $\tilde{y}$ for any occurrence of the substring $\tilde{x}[1..2] = \langle 2, 3 \rangle$, finding two candidate occurrences at position 2 and 4, respectively. We obtain that:

- at the beginning of the third iteration of the **for** main loop, with $s = 2$, we have $r = 3 + 4 - 1 = 6$. Thus the verification procedure is run to compare x against the substring at position $r - d = 4$ in the original text, i.e. $\tilde{y}[4..10] = \langle 6, 6, 12, 12, 2, 9, 8 \rangle$. Unfortunately at position 6 the verification procedure would find neither an exact match nor an order preserving match.
- at the beginning of the fifth iteration of the **for** main loop, with $s = 4$, we have $r = 3 + 4 + 2 + 3 - 3 = 8$. Thus the verification procedure is run to compare x against the substring at position $r - d = 6$ in the original text, i.e. $\tilde{y}[6..12] = \langle 12, 12, 2, 9, 8, 6, 7, 10 \rangle$. Thus at position 6 the verification phase would find both an exact match and an order preserving match.

Regarding the complexity issues it is straightforward to observe that, if the verification phase can be run in $\mathcal{O}(m)$ time, Algorithm 4 achieves a $\mathcal{O}(nm)$ worst case time complexity and requires only $\mathcal{O}(m)$ additional space for maintaining the scaled version of the pattern $\tilde{x}$.

This strategy is applicable to all the problems described before because of the following observations that can be easily checked.

Observation 2. *If two strings u and v are equal then they have the same Monotonic Run Length scaled version: $u = v \Rightarrow \tilde{u} = \tilde{v}$.*

Observation 3. *If two strings u and v are order-equivalent then they have the same Monotonic Run Length scaled version: $u \approx v \Rightarrow \tilde{u} = \tilde{v}$.*

Observation 4. *If two strings u and v have the same Cartesian tree then they have the same Monotonic Run Length scaled version: $\mathcal{CT}(u) = \mathcal{CT}(v) \Rightarrow \tilde{u} = \tilde{v}$.*

Then the same searching strategy can be applied to the three problems and only the verification phase changes. The verification phase is obviously done by direct character comparisons for Exact String Matching. For Order Preserving Pattern Matching in order to establish whether two given sequences of the same length m are order-isomorphic, it is enough to compute their rank and equality functions, and then compare them. For Cartesian Tree Pattern Matching, it is done by computing the Parent-Distance representation of the candidate substring of the text and comparing it with the Parent-Distance representation of the pattern that needs to be computed only once in a preprocessing phase. Thus for the three strategies the verification phase can be performed in linear time.

12.3 Monotonic Run Length Sampling

As observed above in the results of Section 12, the space consumption for representing the partial index obtained by monotonic run length scaling is not particularly satisfactory. This influences also the performance of the search algorithms in practical cases, as we will see in Section 12.4.

In this section we propose the application of a similar approach to that used by Character Distance Sampling sampling in order to obtain a partial index requiring a reduced amount of space, on the one hand, and is able to improve the performance of the search procedure, on the other hand.

The new idea introduced in this section is to identify each monotonic run of the text with its initial position and sample only monotonic runs having a specific length. As in the Character Distance Sampling approach we maintain the distance between two consecutive sampled monotonic runs, allowing to significantly decrease, through sampling, the number of elements represented in the index.

For understanding the remaining of this section we need to introduce some further notions. Given a monotonic run u of w , and assuming $u = w[i..i + k - 1]$, assuming that all the monotonic runs are unique, we use the symbol $\mu(u)$ to indicate its starting position i in the text.

The following definition introduces the *Monotonic Run Length Sampling* (MRLS) process which, given an input string w , constructs a partial index $\tilde{w}^q$ which is the numeric sequence of all (and only) starting positions of any monotonic runs of w with length equal to q , for a given parameter $q > 1$.

Definition 12.6 (Monotonic Run Length Sampling). Let w be a string of length n and let $\tilde{w} = \langle |\rho_0|, |\rho_1|, \dots, |\rho_{k-1}| \rangle$ be the monotonic run length scaled version of w , with $|\tilde{w}| = k$. In addition let ℓ be the maximal length of a monotonic run in w , i.e. $\ell = \max\{|\rho_i| \mid 0 \leq i < k\}$. If q is an integer value, with $2 \leq q \leq \ell$, we define the Monotonic Run Length Sampled version of w , with pivot length q , as the numeric

Function 2: Monotonic-Run-Length-Sampling($\tilde{w}, h, q$)

Data: The Monotonic Run Length Scaled version $\tilde{w}$ of the string w , with length h

Result: The Monotonic Run Length Sampled version $\tilde{w}^q$ of w

$\tilde{w}^q \leftarrow \langle \rangle$;

for $i \leftarrow 0$ **to** $h - 1$ **do**

if $\tilde{w}[i] = q$ **then**

$\tilde{w}^q \leftarrow \tilde{w}^q + \langle \mu(i) \rangle$;

end

end

return $\tilde{w}^q$

sequence $\tilde{w}^q$, defined as

$$\tilde{w}^q = \langle \mu(\rho_{i_0}), \mu(\rho_{i_1}), \dots, \mu(\rho_{i_{h-1}}) \rangle,$$

where

1. $h \leq k$ and $i_{j-1} < i_j$ for each $0 < j < h$;
2. $|\rho_{i_j}| = q$ for each $0 \leq j < h$.
3. $|\rho_t| \neq q$ if $t < i_0$, $t > i_{h-1}$ or $i_{j-1} < t < i_j$ for every $0 \leq j < h$.

Example 12.7. Again, as in Example 12.2, let w be a numeric sequence of length 15, with $w = \langle 4, 5, 11, 7, 6, 6, 12, 12, 2, 9, 8, 6, 7, 10, 13 \rangle$. As already observed $\tilde{w} = \langle 3, 4, 2, 3, 2, 3, 4 \rangle$. Thus we have:

- $\tilde{w}^2 = \langle 5, 8 \rangle$, since $\mu(\langle 6, 12 \rangle) = 5$, $\mu(\langle 2, 9 \rangle) = 8$
- $\tilde{w}^3 = \langle 0, 6, 9 \rangle$, since $\mu(\langle 4, 5, 11 \rangle) = 0$, $\mu(\langle 12, 12, 2 \rangle) = 6$, $\mu(\langle 9, 8, 6 \rangle) = 9$;
- $\tilde{w}^4 = \langle 2, 11 \rangle$, since $\mu(\langle 11, 7, 6, 6 \rangle) = 2$, $\mu(\langle 6, 7, 10, 13 \rangle) = 11$

Function 2 depicts the pseudo-code of the algorithm which computes the MRLS version of an input string w of length m . It gets as input the monotonic run length scaled version $\tilde{w}$ of the string, its length h and the pivot length q . Then it constructs the sequence $\tilde{w}^q$ incrementally by scanning the input sequence $\tilde{w}$ element by element, proceeding from left to right.

It is straightforward to prove that, also in this case, the worst case time complexity of the procedure is $\mathcal{O}(m)$.

Algorithm 5: Naïve algorithm based on Monotonic Run Length Sampling

Data: a pattern x of length m , a text y of length n and its MRLS version $\tilde{y}^q$ of length k

Result: all positions $0 \leq i < n$ such that x occurs in y starting at position i

$\tilde{x} \leftarrow \text{Monotonic-Run-Length-Scaling}(x, m);$
 $\tilde{x}^q \leftarrow \text{Monotonic-Run-Length-Sampling}(\tilde{x}, |\tilde{x}|, q);$
 $h \leftarrow |\tilde{x}^q|;$
 $d \leftarrow \tilde{x}^q[0];$
for $s \leftarrow 1$ **to** $k - h$ **do**
 $i \leftarrow 1$
 while $i < h - 1$ **and** $\tilde{x}^q[i] = \tilde{y}^q[s + i]$ **do**
 $i \leftarrow i + 1$
 end
 $r \leftarrow \tilde{y}^q[s];$
 if $i \geq h - 1$ **and** $\text{Verify}(y, n, x, m, r - d)$ **then**
 Output($r - d$)
 end
end

12.3.1 Searching Using Monotonic Run Length Sampling

In this section we shortly describe a simple naïve procedure to search for all occurrences (in their classical or non classical version) of a pattern x of length m inside a text y of length n . The pseudo-code of such procedure is depicted in Algorithm 5.

The algorithm takes as input both the text y and its MRLS version $\tilde{y}^q$ of length k . During the preprocessing phase the algorithm first computes the scaled version $\tilde{x}$ of the input pattern x and, subsequently, computes its monotonic run length sampled version $\tilde{x}^q$. Let k be the length of the sequence $\tilde{y}$ and let h be the length of the sequence $\tilde{x}^q$. In addition let d be the starting position of the first monotonic run length of x of length q , i.e. $d = \tilde{x}^q[0]$.

The searching phase of the algorithm consists of a main **for** loop which iterates over the sequence $\tilde{y}^q$, proceeding from left to right. For each element $\tilde{y}^q[s]$, for $0 \leq s < h$, the algorithm calls the verification procedure to check an occurrence beginning at position $\tilde{y}^q[s] - d$ in the original text. Roughly speaking, the algorithm aligns the first monotonic run of length q in the pattern with all monotonic runs of length q in the text.

Example 12.8. As in Example 12.2, let $y = \langle 4, 5, 11, 7, 6, 6, 12, 12, 2, 9, 8, 6, 7, 10, 13 \rangle$ be a numeric sequence of length 15 (the text) and let $x = \langle 12, 2, 9, 8, 6, 7, 10 \rangle$ be a pattern of length 7. Then we have $\tilde{y} = \langle 3, 4, 2, 3, 2, 3, 4 \rangle$ and $\tilde{x} = \langle 2, 2, 3, 3 \rangle$. Assuming $q = 3$ we have also $\tilde{y}^q = \langle 0, 6, 9 \rangle$, $\tilde{x}^q = \langle 2, 4 \rangle$ and $d = 2$.

The algorithm considers any position $r \in \tilde{y}^q$ as a candidate occurrence of the pattern and naively checks the whole pattern against the substring $y[r - d..r - d +$

$m - 1]$ of the original text. Thus we have:

- at the first iteration of the main **for** loop we have $s = 0$ and the algorithm would run a verification for the window starting at $\tilde{y}^q[0] - d = 0 - 2 = -2$. However, since $-2 < 0$, such window is skipped.
- at the second iteration of the main **for** loop we have $s = 1$ and the algorithm would run a verification for the window starting at $\tilde{y}^q[1] - d = 6 - 2 = 4$. Thus the pattern x is compared with the substring $y[4..10] = \langle 6, 6, 12, 12, 2, 9, 8 \rangle$. However in this case neither an exact occurrence nor an order preserving occurrence would be found.
- finally, at the third iteration of the main **for** loop we have $s = 2$ and the algorithm would run a verification for the window starting at $\tilde{y}^q[2] - d = 9 - 2 = 7$. Thus the pattern x is compared with the string $y[7..13] = \langle 12, 2, 9, 8, 6, 7, 10 \rangle$. In this case the verification procedure would find both an exact occurrence and an order preserving match.

Regarding the complexity issues it is straightforward to observe that, if the verification phase can be run in $\mathcal{O}(m)$ time, also Algorithm 5 achieves an $\mathcal{O}(nm)$ worst case time complexity and requires only $\mathcal{O}(m)$ additional space for maintaining the sampled version of the pattern $\tilde{x}^q$.

12.4 Experimental Evaluation

In this section, we present experimental results in order to evaluate the performances of the sampling approaches presented in this paper for exact string matching, order preserving pattern matching and Cartesian tree pattern matching.

For both classical and non classical exact string matching, comparisons have been performed in terms of searching times. In the context of ESM and OPM experimental results, we used the following text buffers:

- three real data sequences and specifically: a genome sequence, a protein sequence and a natural language text;
- six RAND- δ sequences of short integer values, where each element of the sequence is an integer in the range $[0;255]$, varying around a fixed mean equal to 100 with a variability of δ , with $\delta \in \{4, 8, 16, 32, 64, 128\}$.
- Seoul Temperature Dataset: consists of a dataset with 658,795 integers referring to the hourly temperatures in Seoul (multiplied by ten) in the years 1907-2019.

All sequences have a size of 100 MB and are provided by the SMART research tool, available online for download. For each sequence in the set, we randomly selected 500 patterns, extracted from the text, and computed the average running time over the 500 runs.

Meanwhile, in the context of CTPM, the experimental results have been conducted using two different alphabets:

- Random Integers: consists of 10,000,000 random integers;
- Random Characters: consists of 10,000,000 random unsigned characters.
- Seoul Temperature Dataset: consists of a dataset with 658,795 integers referring to the hourly temperatures in Seoul (multiplied by ten) in the years 1907-2019.

All sequences are composed by 10,000,000 values [59]. For each sequence in the set, we randomly selected 500 patterns, extracted from the text, and computed the average running time over the 500 runs.

12.4.1 Space Consumption

The first evaluation we discuss in this section relates to the space used to maintain the partial index. This evaluation is independent of the application for which the index is used and has the same value for both classical and non classical string matching.

Figure 12.2 shows the space consumption of the new proposed sampling approaches compared against the CDS and OTS methods. Data are reported as percentage values with respect to the size of the original text on which the index is built. Values are computed on six random texts with a uniform distribution and built on an alphabet of size 2^δ (X-axis) with $2 \leq \delta \leq 8$.

From the data shown in the Figure 12.2 it is possible to observe how the best results in terms of space are obtained by the MRLS and CDS. However, while the latter tends to have good results only for large alphabets, the MRLS approach proves to be more flexible, obtaining good results also for small alphabets. However, it should be noted that these results were obtained on random texts with a uniform distribution of characters, a condition not favorable to the best performances for CDS and OTS.

12.4.2 Running Times for the OPPM problem

For the OPPM problem we took the standard NRq algorithm [25] as a reference point for our evaluation, since it is one of the most effective solution known in literature. Specifically we evaluated the following text-sampling solutions:

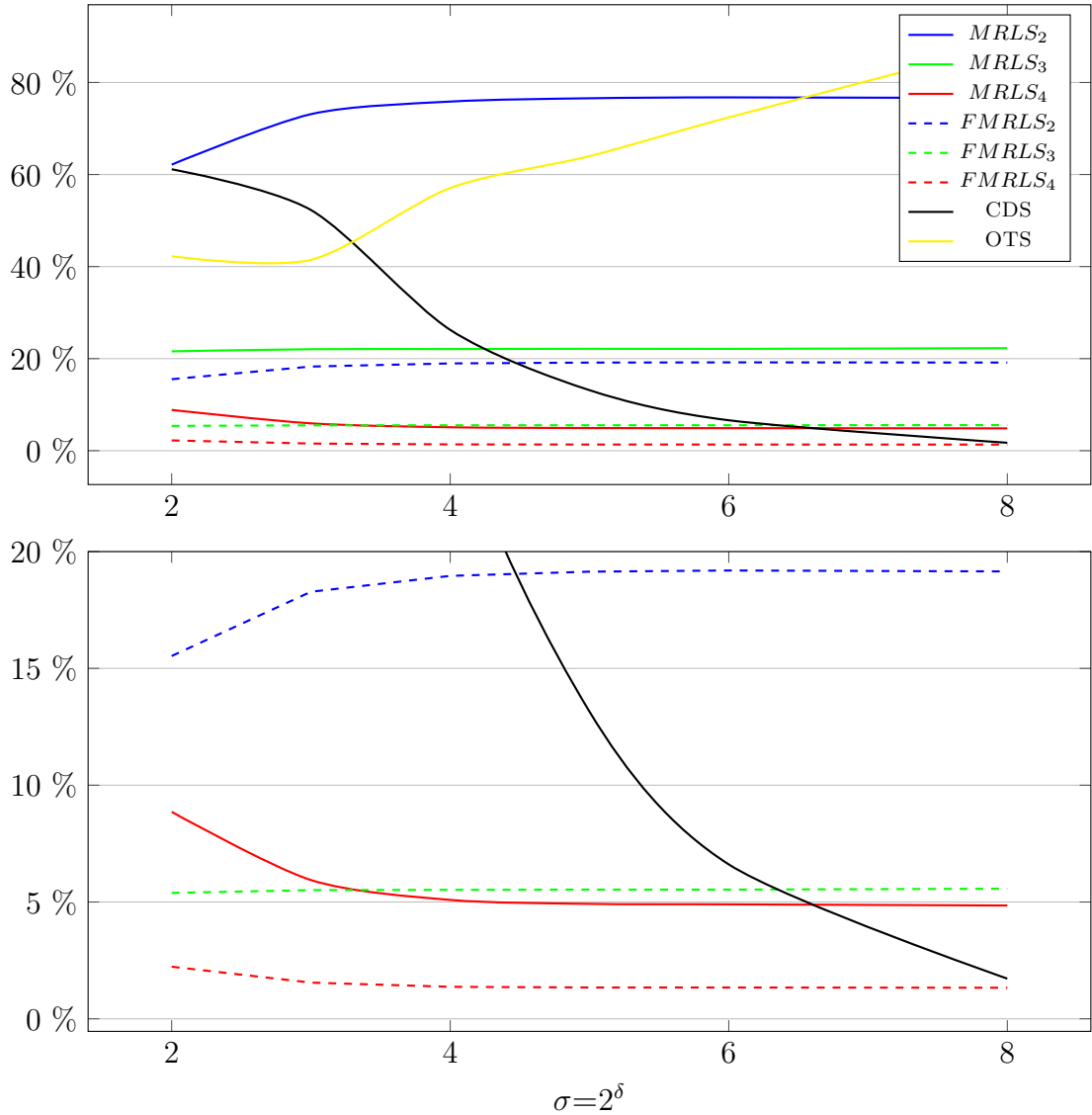


Figure 12.2: Space Consumption of sampling approaches to text searching. Data are reported as percentage values with respect to the size of the original text on which the index is built. Values are computed on six random texts with a uniform distribution and built on an alphabet of size 2^δ (X-axis) with $2 \leq \delta \leq 8$.

- **MRLX₄**: The Monotonic Run Length Scaling approach (Section 12) using a compact representation of the elements (4 bits for each run length) and implemented using the Naïve algorithm.
- **MRLX₈**: The Monotonic Run Length Scaling approach (Section 12) using a relaxed representation of the elements (an 8-bit char for each run length) and

implemented using the Naïve algorithm.

- MRLS_q : The Monotonic Run Length Sampling approach (Section 12.3) using a 32-bit integer value for each text position and implemented by sampling runs of length q , with $2 \leq q \leq 4$.
- FMRLS_q : The Monotonic Run Length Sampling approach using Fake Distance Decomposition using an 8-bit integer value for each text position and implemented by sampling runs of length q , with $2 \leq q \leq 4$.

Table 12.1 shows the experimental evaluation for the OPPM problem on the Rand- δ short integer sequences while Table 12.2 reports the same experimental evaluation using the real Seoul Temperature dataset. Running times for both tables are expressed in milliseconds, and refer to the searching phase.

To better highlight the improvements obtained by the new proposed solutions, in both tables we show the running times only for the reference NR_q algorithm, while we show the speed-up obtained against the latter for all the other tested algorithms. In this context a value greater than 1 indicates a speed-up of the running times proportional to the reported value, while a value less than 1 indicates a slowdown in performance. Best and second best results have been enhanced for a better visualization.

From our experimental results it turns out that the best solutions in almost all the analyzed cases are the MRLS_q and FMRLS_q algorithms, which are almost always at least twice as fast as the NR_q algorithm and reach impressive speed-ups for very long patterns, up to 400 times compared to the reference algorithm. Such algorithms are second only to the MRLX_8 algorithm for short patterns ($m = 8$).

We observe that FMRLS_q turns out to be faster than MRLS_q for short patterns ($8 \leq m \leq 64$), and this is justified by the fact that, by proposing a decomposition of the distances obtained from sampling, FMRLS_q tends to increase the size of the searched pattern, favoring the use of faster search techniques. In contrast, when the pattern length increases, the MRLS_q approach is more effective because it implements a jump-based search strategy, such as that of BM-Horspool. This allows the approach to be up to two times faster than FMRLS_q , which instead uses a naïve searching approach.

The MRLX_8 algorithm allows only marginal speed-ups compared to NR_q , which oscillate between one and a half times and twice as fast as the reference algorithm. In general, its performance is not that impressive, and this is mainly due to the fact that the approach we need to use on this kind of indexes is a naïve approach. The MRLX_4 algorithm performs worse than MRLX_8 and almost never brings improvements over NR_q , probably due to the trade-off introduced by the compact representation of the partial index.

As noted by [10, 9], all sampling methods require a minimum number of sampled characters to function correctly. Specifically, in the case of the MRLS method, a minimum of three sampled distances is necessary to ensure reliable performance. If

δ	m	NR q	MRLX ₄	MRLX ₈	FMRLS q			MRLS q		
					($q = 2$)	($q = 3$)	($q = 4$)	($q = 2$)	($q = 3$)	($q = 4$)
4	8	4.64	0.59	1.02	0.87	0.64	0.62	0.89	0.63	0.62
	16	4.64	0.71	1.15	1.73	0.75	0.62	1.57	0.75	0.62
	32	4.66	0.74	1.18	7.17	1.22	0.68	4.48	1.20	0.68
	64	4.75	0.76	1.20	19.00	3.52	1.04	22.62	3.25	1.03
	128	4.64	0.72	1.16	21.09	25.78	2.23	58.00	19.33	2.19
	256	4.68	0.76	1.18	24.63	66.86	18.00	78.00	93.60	14.62
	512	4.57	0.73	1.14	25.39	76.17	152.33	76.17	228.50	152.33
	1024	4.64	0.76	1.18	27.29	77.33	232.00	154.67	154.67	464.00
8	8	4.79	0.82	1.29	1.08	0.66	0.64	1.13	0.69	0.64
	16	4.37	0.83	1.22	2.47	0.77	0.65	1.93	0.69	0.61
	32	4.39	0.96	1.35	8.78	1.34	0.73	5.29	1.32	0.72
	64	4.32	0.92	1.31	16.00	4.04	0.85	27.00	3.69	0.84
	128	4.34	0.94	1.31	18.08	31.00	1.36	54.25	22.84	1.34
	256	4.44	0.97	1.34	20.18	55.50	5.77	55.50	111.00	5.48
	512	4.41	0.96	1.35	22.05	63.00	147.00	73.50	220.50	88.20
	1024	4.37	0.96	1.34	23.00	72.83	218.50	109.25	145.67	437.00
16	8	4.37	0.82	1.35	1.04	0.68	0.68	1.03	0.67	0.66
	16	4.35	0.97	1.49	2.72	0.83	0.66	2.20	0.81	0.64
	32	4.41	1.03	1.55	9.19	1.31	0.74	5.65	1.29	0.73
	64	4.31	1.03	1.51	15.96	4.35	0.88	25.35	3.88	0.87
	128	4.37	1.04	1.52	17.48	27.31	1.31	54.62	20.81	1.32
	256	4.35	1.04	1.52	18.91	54.37	3.72	54.37	108.75	3.82
	512	4.35	1.03	1.51	20.71	62.14	54.37	72.50	217.50	43.50
	1024	4.32	1.05	1.53	21.60	72.00	216.00	108.00	144.00	432.00
32	8	4.25	0.85	1.42	1.09	0.69	0.67	1.07	0.68	0.66
	16	4.32	1.03	1.62	2.59	0.78	0.65	2.11	0.77	0.65
	32	4.36	1.09	1.67	8.90	1.34	0.69	5.45	1.31	0.68
	64	4.29	1.07	1.65	15.89	4.47	0.85	26.81	4.05	0.85
	128	4.29	1.06	1.65	17.16	26.81	1.39	53.62	22.58	1.38
	256	4.30	1.06	1.63	18.70	53.75	3.64	47.78	107.50	3.55
	512	4.41	1.10	1.69	21.00	63.00	44.10	73.50	220.50	36.75
	1024	4.29	1.05	1.64	21.45	71.50	214.50	107.25	143.00	429.00
64	8	4.40	0.90	1.49	1.14	0.73	0.71	1.10	0.72	0.69
	16	4.35	1.05	1.71	2.54	0.79	0.66	2.12	0.78	0.65
	32	4.42	1.09	1.75	9.40	1.23	0.68	5.74	1.22	0.68
	64	4.31	1.09	1.72	15.96	3.92	0.78	25.35	3.48	0.77
	128	4.31	1.08	1.69	17.24	35.92	1.20	53.87	26.94	1.21
	256	4.37	1.11	1.74	19.00	54.62	3.61	54.62	109.25	3.58
	512	4.30	1.08	1.69	20.48	61.43	47.78	71.67	143.33	35.83
	1024	4.39	1.11	1.71	21.95	73.17	219.50	109.75	146.33	439.00
128	8	4.20	0.88	1.48	1.02	0.71	0.69	1.03	0.70	0.68
	16	4.33	1.12	1.77	1.86	0.82	0.65	1.86	0.82	0.66
	32	4.20	1.08	1.74	2.61	1.11	0.65	5.32	1.14	0.65
	64	4.32	1.15	1.84	2.79	3.82	0.79	27.00	4.24	0.80
	128	4.26	1.03	1.75	2.92	8.52	1.20	47.33	23.67	1.21
	256	4.26	1.13	1.81	3.18	9.47	3.46	53.25	106.50	3.58
	512	4.20	1.14	1.82	3.31	10.00	23.33	84.00	210.00	46.67
	1024	4.26	1.14	1.82	3.46	10.14	213.00	142.00	213.00	426.00

Table 12.1: Experimental results for the OPPM problem on six Rand- δ short integer sequences, for $4 \leq \delta \leq 128$. Running times of the NR q algorithm are expressed in milliseconds. Results for all other algorithms are expressed in terms of speed-up against the reference NR q algorithm.

m	NR q	MRLX ₄	MRLX ₈	FMRLS q			MRLS q		
				($q = 2$)	($q = 3$)	($q = 4$)	($q = 2$)	($q = 3$)	($q = 4$)
8	3.92	1.14	1.22	1.32	1.18	1.94	1.88	1.73	1.65
16	3.78	0.92	1.32	1.85	1.27	2.18	1.89	2.34	2.11
32	3.86	1.15	1.22	2.84	2.31	2.99	5.21	4.32	3.97
64	3.57	1.84	1.92	3.11	3.42	2.28	19.14	15.42	11.58
128	3.56	1.74	1.32	2.88	3.29	3.17	45.21	32.16	73.14
256	3.34	2.74	2.13	3.14	4.11	3.84	58.15	58.11	50.49
512	3.92	1.11	1.18	2.17	3.92	18.14	77.15	119.36	82.41
1024	3.27	2.19	3.15	2.18	8.26	84.13	162.50	196.23	327.00

Table 12.2: Experimental results for the OPPM problem on Seoul Temperature Data. Running times of the NR q algorithm are expressed in milliseconds. Results for all other algorithms are expressed in terms of speed-up against the reference NR q algorithm.

this condition is not met, alternative search procedures or, in the worst case, the original algorithms must be employed [10].

Our experimental results demonstrate that the best performance is achieved for longer patterns, particularly when the pattern length $m > 512$, where the worst-case scenario is effectively avoided. This is because longer patterns naturally satisfy the minimum sampling requirements, thus allowing the sampling methods to operate at their optimal efficiency.

12.4.3 Running Times for the ESM problem

For the ESM problem, in accordance with what has been done in previous publications on the subject, we took the standard BM-Horspool (HOR) algorithm [41] as a reference point for our evaluation. We evaluated the following text-sampling solutions:

- OTS: The Occurrence Text Sampling approach [9] introduced by Claude *et al.* and implemented by removing the first 8 characters of the alphabet, with the exception of the case $\delta = 4$, for which we removed the first 3 characters, and the case $\delta = 8$, for which we removed the first 7 characters.
- CDS: The Character Distance Sampling approach [10] introduced by Faro *et al.*, implemented by selecting the 8th character of the alphabet as a pivot, with the exception of the case $\delta = 4$, for which we selected the 4th character as a pivot.
- MRLS q : The Monotonic Run Length Sampling approach (Section 12.3) using a 32-bit integer value for each text position and implemented using the sampling of monotonic runs of length q , with $2 \leq q \leq 4$.

- FMRLS_q: The Monotonic Run Length Sampling approach with Fake Distance Decomposition using a 8-bit integer value for each text position and implemented using the sampling of monotonic runs of length q , with $2 \leq q \leq 4$.

Table 12.3 shows the experimental evaluation for the ESM problem on the Rand- δ short integer sequences where running times are expressed in milliseconds. Also in this case to better highlight the improvements obtained by the new proposed solutions, in Table 12.3 we show the running times only for the reference HOR algorithm, while we show the speed-up obtained against the latter for all the other tested algorithms.

Our experimental results show that in the case of medium and large-sized alphabets, the MRLS_q and FMRLS_q algorithms do not obtain the impressive performance as in the case of OPPM, since the CDS approach proves to be more effective in most cases.

However, MRLS_q and FMRLS_q algorithms turns out to be very efficient and competitive in the case of small alphabets, a case in which the CDS approach suffers particularly and shows not exciting results.

Moreover, the new approaches achieve the best results even in the case of very long patterns, a condition in which the ratio of text size to pattern size plays a key role, especially in a jump-based searching approach, as is the case in MRLS_q.

The jump-based searching approach is at the root of the performance gap observed for very long patterns between MRLS_q and FMRLS_q. In this case MRLS_q is up to twice as fast as FMRLS_q, which instead uses a naïve searching approach.

It is also interesting to observe how the MRLS_q and FMRLS_q algorithms prove to be competitive in the general case, always obtaining the second best results. The speed-up obtained by comparing them with the reference HOR algorithm reaches a factor of 197, in the case of small alphabets and very long patterns, and a factor of 53, for large alphabets and long patterns.

12.4.4 Running Times for the CTPM problem

For the CTPM problem we took the standard KMPCT algorithm [26] as a reference point for our evaluation, since it is one of the most effective solution known in literature. Moreover we selected the algorithms that has been pointed as the faster in Song *et al.* [59]. Specifically we evaluated the following text-sampling solutions:

- BMHCT: algorithm based on the q -gram Boyer-Moore-Horspool filtration using SIMD instructions, first introduced for OPPM, and then revised for CT matching. [60, 59].
- SKSCT: algorithm based on the q -gram Alpha Skip Search filtration using SIMD instructions [61].

δ	m	HOR	CDS	OTS	FMRLS _q			MRLS _q		
					(q = 2)	(q = 3)	(q = 4)	(q = 2)	(q = 3)	(q = 4)
4	8	2.19	1.09	1.13	1.20	1.16	1.16	1.20	1.16	1.15
	16	1.97	1.11	1.22	1.61	1.19	1.17	1.43	1.19	1.17
	32	1.97	1.12	1.22	2.10	1.42	1.18	2.21	1.41	1.18
	64	1.97	1.12	1.22	7.58	2.70	1.35	9.38	2.35	1.34
	128	1.98	1.12	1.21	8.61	12.38	2.51	22.00	9.00	2.41
	256	1.95	1.11	1.22	8.86	24.38	10.83	32.50	48.75	8.48
	512	1.94	1.12	1.22	9.70	27.71	64.67	38.80	64.67	64.67
	1024	1.97	1.13	1.22	10.37	28.14	65.67	65.67	98.50	197.00
8	8	1.34	1.97	1.28	1.35	1.29	1.30	1.35	1.30	1.30
	16	1.04	2.81	1.49	1.65	1.39	1.37	1.12	1.37	1.37
	32	0.95	3.80	1.56	2.57	1.67	1.40	1.36	1.61	1.40
	64	0.95	6.33	1.56	3.39	2.88	1.46	5.59	2.21	1.46
	128	0.95	8.64	1.56	3.65	7.92	1.83	10.56	5.28	1.79
	256	0.95	11.88	1.56	3.80	11.88	3.96	13.57	19.00	3.52
	512	0.95	13.57	1.53	3.96	13.57	23.75	19.00	31.67	19.00
	1024	0.95	15.83	1.56	4.32	13.57	47.50	31.67	47.50	95.00
16	8	1.02	1.52	0.89	1.38	1.36	1.36	1.42	1.38	1.38
	16	0.76	2.38	1.25	1.58	1.58	1.55	0.97	1.68	1.55
	32	0.65	4.06	1.67	1.91	1.86	1.67	1.02	1.81	1.62
	64	0.63	5.73	1.75	2.25	2.86	1.75	3.50	2.10	1.75
	128	0.62	8.86	1.77	2.30	5.64	2.00	6.89	3.88	2.00
	256	0.62	12.40	1.77	2.48	7.75	3.65	7.75	15.50	3.26
	512	0.62	15.50	1.77	2.58	8.86	20.67	12.40	20.67	12.40
	1024	0.62	20.67	1.82	2.82	8.86	31.00	20.67	31.00	62.00
32	8	0.90	1.53	1.32	1.48	1.45	1.43	1.50	1.45	1.43
	16	0.66	2.06	1.58	1.57	1.69	1.69	0.89	1.78	1.65
	32	0.55	3.67	2.20	1.67	2.12	1.90	0.83	1.96	1.90
	64	0.51	6.38	2.55	1.82	3.00	2.04	3.00	1.96	2.04
	128	0.49	9.80	2.72	1.88	4.90	2.45	5.44	3.77	2.33
	256	0.50	12.50	2.78	2.00	6.25	3.85	7.14	10.00	3.57
	512	0.49	24.50	2.72	2.04	6.12	12.25	9.80	16.33	8.17
	1024	0.50	25.00	2.78	2.17	7.14	25.00	16.67	25.00	50.00
64	8	0.84	1.56	1.45	1.53	1.47	1.45	1.53	1.47	1.47
	16	0.65	2.10	1.97	1.67	1.91	1.81	0.89	1.81	1.81
	32	0.51	3.00	1.97	1.50	2.22	2.04	0.74	1.96	2.04
	64	0.48	6.00	2.82	1.71	3.00	2.29	2.67	1.92	2.40
	128	0.45	11.25	3.21	1.61	4.50	2.81	4.50	2.81	2.81
	256	0.46	15.33	3.54	1.77	5.11	4.60	6.57	9.20	3.83
	512	0.46	23.00	3.54	1.84	5.75	15.33	9.20	15.33	9.20
	1024	0.46	23.00	3.83	1.92	6.57	23.00	15.33	23.00	46.00
128	8	0.84	1.68	1.47	1.56	1.47	1.50	1.58	1.53	1.53
	16	0.62	2.21	2.00	1.55	1.94	1.88	0.83	1.94	1.94
	32	0.52	3.25	2.74	1.58	2.60	2.48	0.72	2.26	2.36
	64	0.51	5.10	3.64	1.42	3.64	3.19	3.19	2.04	3.19
	128	0.50	10.00	4.55	1.85	5.00	3.85	6.25	3.57	3.85
	256	0.49	16.33	5.44	1.88	6.12	5.44	7.00	12.25	4.45
	512	0.49	24.50	5.44	1.96	6.12	16.33	7.00	16.33	9.80
	1024	0.53	26.50	6.62	2.30	7.57	53.00	17.67	26.50	53.00

Table 12.3: Experimental results for the ESM problem on six Rand- δ short integer sequences, for $4 \leq \delta \leq 128$. Running times of the HOR algorithm are expressed in milliseconds. Results for all other algorithms are expressed in terms of speed-up against the reference HOR algorithm. Over a text of 100MB.

δ	m	KMP CT	BMH CT	SKS CT	MRLS _q ($q = 2$) ($q = 3$) ($q = 4$)		
Random Int	128	71.24	13.15	104.50	10.00	46.24	7.70
	256	65.45	13.38	79.78	77.20	46.01	13.86
	512	70.27	16.00	74.70	7.70	51.23	10.00
	1024	65.45	75.93	65.45	8.47	28.28	101.71
	2048	67.49	14.56	87.72	9.24	37.09	92.32
Random Char	128	68.32	14.21	929.32	7.70	10.01	5.60
	256	68.53	15.71	834.87	6.98	941.13	20.49
	512	65.44	7.79	891.14	7.00	913.18	18.14
	1024	69.27	13.68	958.16	12.54	887.57	931.43
	2048	64.68	12.45	911.94	13.21	924.00	938.19
Seoul Temperature Dataset	128	35.98	15.12	88.19	12.48	9.86	15.47
	256	35.43	22.84	75.52	64.11	71.39	75.67
	512	37.84	19.72	84.12	64.84	67.37	61.52
	1024	39.18	31.87	91.63	56.43	72.13	90.84
	2048	40.11	30.48	89.27	92.13	87.63	100.18

Table 12.4: Experimental results for the CTPM problem on random Integers and Characters, and Seoul Temperature. Running times of the KMPCT are expressed in milliseconds. Results for all other algorithms are expressed in terms of speed-up against the reference KMPCT algorithm. Over a text of 10,000,000 different symbols.

- MRLS_q: The Monotonic Run Length Sampling approach (Section 12.3) using a 32-bit integer value for each text position and implemented using the sampling of monotonic runs of length q , with $2 \leq q \leq 4$.

Table 12.4 shows the experimental evaluation for the CTPM problem on the random Integers and Characters sequences where running times are expressed in milliseconds. Also in this case, to highlight the improvements obtained by the new solutions, in Table 12.4 we show the running times only for the reference KMPCT algorithm, while we show the speed-up obtained against the latter for all the other tested algorithms.

Our experimental results show that in the case of large-sized alphabets, the MRLS_q algorithm obtain an impressive speed-up compared with the KMPCT and BMHCT algorithms, while achieving similar results with SKSCT.

Moreover it is important to highlight that the newly introduced methods, as previously defined in their practical implementation, require a verification phase to ensure that the potential matches found in the sampled representation are valid in both the original text and the pattern. It is evident that a certain number of false matches will be identified during the searching phase and subsequently discarded in the following phases. Our experimental data indicate that the percentage of false matches ranges from 33% when searching for short patterns ($m = 128$) to 7% when searching for long patterns ($m > 1024$).

Parameterized String Matching through Sampling

A *parameterized string* (p-string) is a string defined over an alphabet $(\Sigma_s \cup \Sigma_p)$, where Σ_s and Σ_p are disjoint alphabets representing static symbols (s-symbols) and parameter symbols (p-symbols), respectively. Two p-strings x and y are said to *parameterized match* (p-match) if and only if x can be transformed into y by applying a bijective mapping on Σ_p to every occurrence of p-symbols in x .

While *parameterized matching* was initially introduced for use between two strings of equal length, it has since been extensively studied for applications involving strings of different lengths (a text and a pattern), similar to classical matching techniques. In this paper, we focus on this class of solutions.

Example 13.1. Let $\Sigma_s = \{A, B, C\}$ and $\Sigma_p = \{w, x, y, z\}$. The string $P = AxByCx$ p-matches with $T[1, 21] = AyBxCyAwBxCzxyAzBwCz$ at positions 1 and 15. At position 1, the mapping is $x \rightarrow y$ and $y \rightarrow x$, whereas at position 15, the mapping is $x \rightarrow z$ and $y \rightarrow w$. Note that P does not match at position 7, as x would need to correspond to both w and z .

A naive procedure to determine whether a string $x[1..m]$ has a p-match at position s of the text y involves traversing both x and $y[s..s + m - 1]$ from left to right while constructing a mapping table that defines the required transformations for p-matching. This process continues until a mismatch occurs, with mismatches arising in the following cases: (i) one symbol is a parameter and the other a non-parameter; (ii) both symbols are different non-parameters; and (iii) both symbols are parameters but have previously been mapped to different symbols. If no mismatches occur, then x and y perform a p-match. The time complexity of this procedure is $O(m)$, with space complexity $O(|\Sigma_p|)$.

The parameterized string matching problem was initially approached by Baker [62], who applied it to software maintenance for detecting large duplicate segments within software systems. Another early application was plagiarism detection [63]. Baker's algorithm preprocesses both the text and the pattern by transforming them into *predecessor strings*, showed in Def. 13.2. This preprocessing allows the problem to be nearly reduced to conventional exact string matching algorithms [64].

```

CHECK-P-MATCH( $x, m, y, s$ )
1.   $map \leftarrow \{\}$ 
3.  while  $i \leq m$ 
4.    if  $x[i] \in \Sigma_s$ 
4.    if  $x[i] \neq y[s + i]$  then return FALSE
4.    else
4.    if  $x[i] \in map$ 
4.    if  $map[x[i]] \neq y[s + i]$  return FALSE
5.    else
4.     $map[x[i]] = y[s + i]$ 
4.     $i \leftarrow i + 1$ 
7.  return TRUE

```

Figure 13.1: Pseudocode for the CHECK-P-MATCH procedure, which tests for the occurrence of a pattern x of length m in a text y of length n , starting at position s . The procedure also checks if an isomorphism exists, as required for parameterized pattern matching.

Definition 13.2 (Prev). Let $X[1, m]$ be a p-string defined over an alphabet $\Sigma = \Sigma_s \cup \Sigma_p$, where Σ_s and Σ_p are disjoint. The transformation X' of a string X is another string $X'[1, m]$ defined over the alphabet $\Sigma' = \Sigma_s \cup \mathbb{N}$ where, for each $1 \leq i \leq m$, $X'[i]$ is defined as follows:

$$X'[i] = \begin{cases} X[i] & \text{if } X[i] \in \Sigma_s \\ 0 & \text{if } X[i] \in \Sigma_p \text{ and the first occurrence of } X[i] \text{ in } X \text{ is at position } i \\ i - j & \text{otherwise, where } j \text{ is the last occurrence of } X[i] \text{ in } X[1, i - 1]. \end{cases}$$

The time complexity of the computation of X' is $\Theta(m)$, and the space complexity is $O(|\Sigma_p|)$ by means of a table containing the last occurrence position of each parameter. Notice that X' is calculated in such a way that it does not matter what the parameters of X are; what is really relevant is the different occurrences of the same parameter character, which provides valuable information about the structure of the p-string. Thus, two p-strings X and Y are a p-match if and only if $X' = Y'$. For example, given $\Sigma_s = \{b\}$, $\Sigma_p = \{x, y, z\}$, $X[1, 7] = xbyyxbx$ and $Y[1, 7] = zbxzxzbz$, we find that $X' = 0b014b2 = Y'$ and therefore X and Y are a p-match.

In 1997, Amir et al. [65] extended the concept to two-dimensional parameterized matching, which supports a broad range of applications, including programming languages, computational biology, and image processing. This advancement allows for image recognition even when color maps are altered. Subsequent two-dimensional approaches were developed by Hazay et al. [66] and later refined by Cole et al. [67].

An alternative approach by Salmela and Tarhio [64] uses q-grams, leveraging text condensation to solve the problem with sublinear time complexity on average

for moderately repetitive patterns. Other significant developments in parameterized matching include techniques for handling multiple patterns [68], matching with mismatches [69], and approximate parameterized search [70].

In summary, parameterized string matching has seen substantial advancements, leading to the development of efficient algorithms addressing diverse string matching problems with practical applications across fields like bioinformatics, natural language processing, and software engineering.

Baker’s foundational paper [62] introduced an innovative suffix-tree-based algorithm for discovering parameterized matches. In this method, both the text and pattern undergo a preprocessing step that converts them into predecessor strings, allowing the use of exact string matching algorithms [64]. One challenge with this approach is managing text windows where predecessor pointers may refer to positions outside the current window. Baker resolved this by modifying the suffix tree construction algorithm, achieving a time complexity of $O(n \log n)$.

Subsequent improvements in suffix tree construction were made by Kosaraju [71], whose algorithm achieves a time complexity of $O(n(\log \lambda + \log \sigma))$, where σ and λ represent the sizes of the parameterized and fixed alphabets. Cole and Hariharan [72] further refined suffix tree construction, introducing a randomized linear-time algorithm.

In the area of parameterized matching, Baker [73] also developed an algorithm based on the Boyer–Moore method using predecessor strings. This TurboBM algorithm [74] detects parameterized matches with a worst-case time complexity of $O(n \log \min(m, \sigma))$, though the paper did not address its average-case complexity.

Amir et al. [75] contributed an algorithm inspired by the Knuth–Morris–Pratt algorithm for parameterized matching, with a worst-case time complexity of $O(n \log(\sigma))$, optimal for scenarios with an unbounded alphabet.

Further progress was made by Fredriksson and Mozgovoy [63], who created sub-linear algorithms for one-dimensional parameterized matching. These algorithms build on the shift-or technique [76] and backward DAWG matching (BDM) [74]. The shift-or algorithm has a worst-case complexity of $O(n \lceil m/w \rceil)$, while the BDM-based algorithm achieves an average complexity of $O(n \log_\lambda(m)/m)$, where λ is the size of the fixed alphabet. Notably, the BDM-based algorithm can handle multiple pattern searches, though its average-case analysis depends on a high proportion of non-parameterized symbols.

In two-dimensional parameterized matching, Amir et al. [65] initially introduced function matching, pre-processing the text to form a predecessor representation for two-dimensional strings. This approach has a worst-case complexity of $O(n^2 \log^2 m)$. Hazay et al. [66] later proposed an algorithm using the “duel-and-sweep” paradigm, achieving a worst-case complexity of $O(n^2 + m^{2.5} \text{polylog}(m))$. Salmela et al. [64] presented a related two-dimensional algorithm based on Tarhio’s two-dimensional string matching algorithm [77] using q-grams, with a worst-case complexity of $O(n^2 m^2)$.

13.1 A Text Sampling Solution to Parameterized Matching

In this section, we introduce an adaptation of text sampling techniques to another class of SCER-matching: the parameterized string matching (PSM) problem. Specifically, we modify the CDS approach [10], originally designed for exact string matching, to accommodate the unique constraints of parameterized matching.

Let y be the input text of length n , and let x be the input pattern of length m , both defined over an alphabet $\Sigma = \Sigma_s \cup \Sigma_p$ of size $\sigma = \sigma_s + \sigma_p$. We assume that all strings are treated as vectors indexed from position 1, where $x[i]$ denotes the i -th character of string x , for $1 \leq i \leq m$.

Our approach begins by selecting a sub-alphabet $C \subseteq \Sigma_s$, referred to as the set of pivot characters. These pivots serve as anchors to sample the text y by computing the distances between consecutive occurrences of any pivot character $c \in C$. Formally, this sampling methodology is based on position sampling. Given

$$\delta : \{1, \dots, n_c\} \rightarrow \{1, \dots, n\}$$

where $\delta(i)$ denotes the position of the i -th occurrence of a pivot character c in y , the position-sampled version of y , denoted as $\dot{y}$, is a numeric sequence of length n_c : $\dot{y} = \langle \delta(1), \delta(2), \dots, \delta(n_c) \rangle$.

For example, consider the text sequence $y = \text{"AgAACgCAgtAtA"}$ of length 13 over the alphabet $\Sigma_s = \{\mathbf{A}, \mathbf{C}\}$ and $\Sigma_p = \{\mathbf{g}, \mathbf{t}\}$. If we select $C = \{\mathbf{A}\}$ as the set of pivot characters, the position-sampled representation of y is

$$\dot{y} = \langle 1, 3, 4, 8, 11, 13 \rangle.$$

Here, the first occurrence of $\mathbf{A}$ is at position 1 ($y[1] = \text{"A"}$), the second at position 3 ($y[3] = \text{"A"}$), and so forth.

Following this representation, we define the character distance function as

$$\Delta(i) = \delta(i+1) - \delta(i), \quad 1 \leq i \leq n_c - 1.$$

The characters-distance sampled version of y , denoted as $\bar{y}$, is a numeric sequence of length $n_c - 1$:

$$\begin{aligned} \bar{y} &= \langle \Delta(1), \Delta(2), \dots, \Delta(n_c - 1) \rangle \\ &= \langle \delta(2) - \delta(1), \delta(3) - \delta(2), \dots, \delta(n_c) - \delta(n_c - 1) \rangle. \end{aligned}$$

For instance, given the same text $y = \text{"AgAACgCAgtAtA"}$ with $C = \{\mathbf{A}\}$, the character-distance sampled version of y is

$$\bar{y} = \langle 2, 1, 4, 3, 2 \rangle.$$

Specifically, $\bar{y}[1] = \Delta(1) = \delta(2) - \delta(1) = 3 - 1 = 2$ and $\bar{y}[3] = \Delta(3) = \delta(4) - \delta(3) = 8 - 4 = 4$.

In practice, particularly for large alphabets, the set of pivot characters is often restricted to a single character. Consequently, we frequently refer to the pivot character in the singular rather than explicitly mentioning the entire set. By restricting the selection of pivots to characters from the static alphabet Σ_s , the CDS algorithm reduces the text to a simplified representation requiring only exact matching. Given this assumption, the preprocessing and searching phases follow a straightforward pipeline: first, the position-sampled versions of both the text and the pattern are computed, and second, all occurrences of the sampled pattern within the sampled text are identified.

However, like other sampling-based string matching approaches, an additional verification phase is required once a potential match is detected. In the case of parameterized string matching, the verification process consists of two distinct steps. The first step ensures that all characters from Σ_s in the pattern match exactly with those in the corresponding positions of the text. Formally, given a candidate match at position j , we define

$$\text{VERIFY}(x, y, j) = \begin{cases} 1, & \text{if } y[i + j] = x[i], \quad \forall i \mid 1 \leq i \leq m, y[i + j] \in \Sigma_s, x[i] \in \Sigma_s, \\ 0, & \text{otherwise.} \end{cases}$$

An additional step is introduced to handle characters from Σ_p . This second phase ensures that occurrences identified by the algorithm also satisfy the bijective mapping conditions of parameterized string matching, described above. By incorporating this two-phase verification process, our adaptation maintains the efficiency of CDS while ensuring correctness in the parameterized setting. The resulting approach improves computational scalability, making it well-suited for large-scale parameterized string matching tasks. The pseudocode of the proposed searching procedure is presented in Algorithm 6.

The CDS-based sampled string matching approach maintains a partial index, represented by the position-sampled version of y . This index requires $32n_c$ bits in memory, making it lightweight for search operations. To search for a pattern x of length m in y , a preprocessing step computes its sampled version $\bar{x}$. An occurrence of x in y corresponds to an occurrence of $\bar{x}$ in $\bar{y}$, meaning that standard string matching algorithms can locate $\bar{x}$ in $\bar{y}$. However, the reverse does not necessarily hold, requiring a validation step in y .

Since validation requires $O(m)$ time, the total search complexity is $O(mn)$. However, optimizations ensure the search remains linear [8]. Notably, the CDS approach does not explicitly maintain $\bar{y}$, instead keeping $\dot{y}$, which holds the positions. The sequence $\bar{y}$ is computed dynamically during search, with each element given by:

$$\bar{y}(i) = \dot{y}(i + 1) - \dot{y}(i).$$

Algorithm 6: Parameterized Search with Character Distance Sampling

Function pCDS($\bar{x}$, $|\bar{x}|$, $\dot{y}$, $|\dot{y}|$, y , n , x , m , ϕ):

```

    occurrences  $\leftarrow$  0
    shift  $\leftarrow$  0
    hbc  $\leftarrow$  PreprocessHBC( $\bar{x}$ ,  $|\bar{x}|$ )
    while shift  $\leq |\dot{y}| - |\bar{x}|$  do
        i  $\leftarrow$  1
        while i  $< |\bar{x}|$  and  $\bar{x}[i - 1] = \dot{y}[\text{shift} + i] - \dot{y}[\text{shift} + i - 1]$  do
            i  $\leftarrow$  i + 1
        end
        if i =  $|\bar{x}|$  then
            start  $\leftarrow \dot{y}[\text{shift}] - \phi$ 
            i  $\leftarrow$  0
            while i  $< m$  and  $x[i] = y[\text{start} + i]$  do
                i  $\leftarrow$  i + 1
            end
            if i = m and IsoCheck( $x$ ,  $m$ ,  $y$ , start) = 1 then
                occurrences  $\leftarrow$  occurrences + 1
            end
            shift  $\leftarrow$  shift + hbc[ $\bar{x}[0]$ ]
            continue
        end
        k  $\leftarrow \dot{y}[\text{shift} + |\bar{x}|] - \dot{y}[\text{shift} + |\bar{x}| - 1]$ 
        shift  $\leftarrow$  shift + max(HBC( $k$ ),  $|\bar{x}|$ )
    end
    return occurrences

```

Despite these advantages, CDS has limitations. Its performance depends on pivot selection, which must balance index size and execution time. Studies indicate that, in English text, the pivot character ranked 8th in frequency offers optimal results. Additionally, for very short patterns without pivot occurrences, standard string search methods are required. In small-alphabet texts, CDS may not provide space efficiency benefits, though the use of condensed alphabets [13] can mitigate this issue. Recently, further improvements in space and time efficiency have been introduced using the *fake distance representation* [14].

13.2 Experimental Results

This section presents the evaluation of our parameterized string matching approach, Character Distance Sampling (pCDS). We compare pCDS against the parameterized Boyer-Moore (pBM), parameterized Turbo Boyer-Moore (pTBM), and our adapted parameterized Horspool (pHor) algorithm, which treats characters in the parameter

alphabet Σ_p as don't-care symbols and applies a CDS-like verification phase.

We evaluated performance across multiple 5 MB benchmark datasets with varying parameter probabilities (φ), using 1000 distinct patterns per dataset. Metrics include preprocessing time, execution time, and memory usage. Execution time was measured as the average time required to match a pattern, while memory consumption tracked data structure overhead.

Notably, the additional memory for constructing the partial index, essential for sampling-based methods like pCDS, remained below 2% of the text size, highlighting its efficiency.

Experimental results are presented in Table 13.1. They clearly highlight significant differences in the performance of the evaluated algorithms.

Among the tested approaches, pCDS consistently achieves the best execution times. Its performance is significantly superior across all values of φ and pattern lengths m , demonstrating its efficiency in parameterized string matching. In contrast, the pBM and pTBM algorithms exhibit higher execution times, with pTBM generally performing slightly better than pBM, although with some fluctuations. The pHOR algorithm offers intermediate performance, outperforming pBM but remaining less efficient than pCDS.

As φ increases, meaning a larger proportion of parameterized characters in the text, the performance of all algorithms deteriorates. This effect is particularly evident in the case of pHOR, which experiences a notable increase in execution time as φ grows from 10% to 50%. The pBM and pTBM methods show similar trends, with execution times increasing gradually. Although pCDS remains the most efficient solution, it is also affected by the rise in φ , with its execution time nearly doubling when φ increases from 10% to 50%.

The impact of the pattern length m is also significant. As m increases, all algorithms benefit from reduced execution times. However, this improvement is particularly pronounced for pCDS, which scales exceptionally well. For instance, while pBM, pTBM, and pHOR stabilize at relatively high execution times for large m , pCDS achieves a dramatic reduction, dropping from approximately 40 ms for $m = 32$ to only 3.8 ms for $m = 1024$. This confirms the superior scalability of the CDS-based approach in handling long patterns efficiently.

Each of the evaluated algorithms presents distinct strengths and weaknesses. The pBM algorithm is a well-established method with solid theoretical foundations. It performs well for long patterns but suffers from high computational costs when applied to short patterns or when φ is high. Its optimized variant, pTBM, aims to improve upon pBM by introducing more efficient pattern shifts. However, its performance varies across different settings, and it does not consistently surpass pBM in all cases.

The pHOR algorithm provides a balanced trade-off between efficiency and implementation complexity. It generally outperforms pBM and pTBM but exhibits a noticeable performance degradation for longer patterns and higher values of φ . This makes it a reasonable choice when seeking a simple yet effective approach, though

φ	m	pBM	pTBM	pHOR	pCDS
10	32	73.55	66.48	57.46	39.77
	64	55.84	53.50	46.93	23.52
	128	52.25	49.49	43.83	13.20
	256	51.57	58.37	44.25	06.42
	512	50.51	46.98	42.32	04.73
	1024	49.45	48.51	42.79	03.80
20	32	75.86	78.64	63.45	46.01
	64	60.53	60.52	51.40	27.47
	128	54.00	52.18	47.76	13.35
	256	53.50	60.92	45.46	06.76
	512	51.80	51.61	47.05	05.02
	1024	53.81	48.83	47.26	03.92
30	32	82.16	74.93	71.25	54.83
	64	61.56	63.91	53.70	30.18
	128	55.31	54.02	50.47	14.77
	256	55.97	63.56	51.94	06.56
	512	53.24	52.59	50.76	04.85
	1024	53.61	51.03	50.07	03.87
40	32	93.01	92.08	88.81	70.99
	64	68.03	62.68	65.55	37.19
	128	59.75	59.95	59.04	17.51
	256	57.85	67.97	59.62	06.52
	512	53.29	53.82	55.65	04.45
	1024	55.55	55.01	55.49	03.63
50	32	103.83	97.49	112.45	96.46
	64	73.97	79.07	76.10	52.46
	128	61.82	54.65	64.32	23.02
	256	54.32	67.90	63.32	07.42
	512	54.12	55.52	61.80	04.47
	1024	56.11	55.17	62.86	03.43

Table 13.1: Comparative performance of parameterized string matching algorithms across different values of φ and pattern lengths m . Execution time values are in milliseconds. Bold values indicate best performance, while the second-best results are highlighted in gray.

it does not achieve the same level of efficiency as pCDS.

Among all tested methods, pCDS emerges as the fastest and most scalable solution. It significantly reduces execution times across all scenarios and is particularly well-suited for large patterns. However, its efficiency declines more rapidly as φ increases, indicating that its advantages are more pronounced in cases where the

proportion of parameterized characters is moderate.

The results demonstrate that pCDS is the most efficient algorithm for parameterized string matching, achieving the lowest execution times across all tested scenarios. When φ is low, all algorithms perform relatively well, but pCDS consistently maintains an advantage. The benefits of pCDS become even more pronounced for longer patterns, where its execution time is significantly lower than that of the competing methods.

However, for higher values of φ ($\geq 40\%$), pCDS experiences a more noticeable performance drop compared to pBM and pTBM. Despite this, it remains the best-performing algorithm overall.

In conclusion, pCDS represents the most effective solution for parameterized string matching, particularly when the percentage of parameterized characters is moderate. Its combination of speed and scalability makes it highly suitable for real-world applications.

Conclusions

This thesis has investigated the problem of sampled string matching, introducing a set of novel techniques that aim to reconcile the classical trade-off between search efficiency and space consumption. Starting from foundational methods such as Occurrence Text Sampling (OTS) and Character Distance Sampling (CDS), we have proposed several extensions and refinements that significantly enhance the practical applicability of sampled indexes. Among these, Character Context Sampling (CCS), Fake Sampling, and Set Cover-based optimization emerged as particularly effective strategies for reducing memory usage while maintaining or even improving search performance. Furthermore, the exploration of Monotonic Run Length Sampling (MRLS) demonstrated how sampled approaches can be extended beyond traditional exact matching tasks to order-preserving and structural pattern matching problems.

The contributions of this work can be summarized along three main axes:

- **Algorithmic innovation:** New sampling schemes have been designed to reduce the size of the index while increasing its accuracy, providing theoretical and empirical evidence of their effectiveness.
- **Experimental validation:** All proposed methods were implemented and tested under rigorous and reproducible conditions, showing consistent improvements over existing approaches on both real and synthetic datasets.
- **Broader applicability:** By extending sampling techniques to structural properties of strings (periods, covers, monotonic runs), this thesis has demonstrated that the principles of sampled string matching can be generalized to a wider class of problems.

14.1 Future Directions

The work presented in this thesis opens several promising directions for further research.

1. **Adaptive and hybrid sampling.** Current methods rely on fixed strategies for pivot selection or distance encoding. Adaptive schemes that dynamically

adjust to the local structure of the text or to the query distribution could lead to more robust performance across heterogeneous datasets.

2. **Integration with compressed indexes.** A natural step forward is the combination of sampled matching with compressed suffix arrays or FM-indexes, in order to obtain hybrid data structures that simultaneously optimize for space, query speed, and update efficiency.
3. **Application-driven optimizations.** While this thesis has shown the relevance of sampled matching in bioinformatics and large-scale text retrieval, future work should explore domain-specific adaptations. Examples include pangenomic indexing, cybersecurity log analysis, and streaming applications where updates occur in real time.
4. **Theoretical exploration.** Several questions remain open from a combinatorial perspective, particularly regarding the optimality of pivot selection under set cover constraints, and the characterization of worst-case scenarios for context-based sampling.
5. **Emerging paradigms.** Finally, the increasing relevance of parallel computing, GPU acceleration, and quantum algorithms provides opportunities to re-examine sampled string matching in new computational models.

In conclusion, this thesis contributes to bridging the gap between classical indexing and efficient online searching, offering a spectrum of techniques that are both theoretically sound and practically scalable. The approaches developed here are expected to serve as building blocks for future research in text indexing, bioinformatics, and large-scale information retrieval, where the demand for efficient, memory-conscious solutions will continue to grow.

Bibliography

- [1] Simone Faro and Thierry Lecroq. The exact online string matching problem: A review of the most recent results. *ACM Computing Surveys*, 45(2):13:1–13:42, 2013.
- [2] Donald E. Knuth, James H. Morris Jr., and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977.
- [3] R.S. Boyer and J.S. Moore. A fast string searching algorithm. *Communications of the ACM*, 20(10):762–772, 1977.
- [4] P. Weiner. Linear pattern matching algorithms. In *IEEE Symposium on Switching and Automata Theory*, pages 1–11, 1973.
- [5] U. Manber and G. Myers. Suffix arrays: A new method for on-line string searches. *SIAM Journal on Computing*, 22(5):935–948, 1993.
- [6] Paolo Ferragina and Giovanni Manzini. Opportunistic data structures with applications. *Foundations of Computer Science (FOCS)*, pages 390–398, 2000.
- [7] U. Vishkin. Deterministic sampling—a new technique for fast pattern matching. *SIAM Journal on Computing*, 20(1):22–33, 1991.
- [8] Simone Faro, Francesco Pio Marino, and Antonino Andrea Moschetto. Beyond horspool: A comparative analysis in sampled matching. In Jan Holub and Jan Zdárek, editors, *Prague Stringology Conference 2024, Prague, Czech Republic, August 26-27, 2024*, pages 16–26. Czech Technical University in Prague, Faculty of Information Technology, Department of Theoretical Computer Science, 2024.
- [9] Francisco Claude, Gonzalo Navarro, Hannu Peltola, Leena Salmela, and Jorma Tarhio. String matching with alphabet sampling. *J. Discrete Algorithms*, 11:37–50, 2012.
- [10] Simone Faro, Francesco Pio Marino, and Arianna Pavone. Efficient online string matching based on characters distance text sampling. *Algorithmica*, 82(11):3390–3412, 2020.
- [11] Simone Faro and Francesco Pio Marino. Reducing time and space in indexed string matching by characters distance text sampling. In Jan Holub and Jan Zdárek, editors, *Prague Stringology Conference 2020, Prague, Czech Republic, August 31 - September 2, 2020*, pages 148–159. Czech Technical University in Prague, Faculty of Information Technology, Department of Theoretical Computer Science, 2020.

- [12] Simone Faro, Francesco Pio Marino, and Arianna Pavone. Improved characters distance sampling for online and offline text searching. *Theoretical Computer Science*, 946:113684, 2023.
- [13] Simone Faro, Francesco Pio Marino, and Arianna Pavone. Enhancing characters distance text sampling by condensed alphabets. In Claudio Sacerdoti Coen and Ivano Salvo, editors, *Proceedings of the 22nd Italian Conference on Theoretical Computer Science, Bologna, Italy, September 13-15, 2021*, volume 3072 of *CEUR Workshop Proceedings*, pages 1–15. CEUR-WS.org, 2021.
- [14] Simone Faro, Francesco Pio Marino, Antonino Andrea Moschetto, Arianna Pavone, and Antonio Scardace. The great textual hoax: Boosting sampled string matching with fake samples. In Andrei Z. Broder and Tami Tamir, editors, *12th International Conference on Fun with Algorithms, FUN 2024, June 4-8, 2024, Island of La Maddalena, Sardinia, Italy*, volume 291 of *LIPICs*, pages 13:1–13:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [15] Simone Faro, Francesco Pio Marino, Arianna Pavone, and Antonio Scardace. Towards an efficient text sampling approach for exact and approximate matching. In Jan Holub and Jan Zdárek, editors, *Prague Stringology Conference 2021, Prague, Czech Republic, August 30-31, 2021*, pages 75–89. Czech Technical University in Prague, Faculty of Information Technology, Department of Theoretical Computer Science, 2021.
- [16] Jinil Kim, Peter Eades, Rudolf Fleischer, Seok-Hee Hong, Costas S. Iliopoulos, Kunsoo Park, Simon J. Puglisi, and Takeshi Tokuyama. Order-preserving matching. *Theoretical Computer Science*, 525:68–79, 2014.
- [17] Sung Gwan Park, Amihoud Amir, Gad M. Landau, and Kunsoo Park. Cartesian tree matching and indexing. In Nadia Pisanti and Solon P. Pissis, editors, *30th Annual Symposium on Combinatorial Pattern Matching, CPM 2019, June 18-20, 2019, Pisa, Italy*, volume 128 of *LIPICs*, pages 16:1–16:14, 2019.
- [18] Simone Faro, Thierry Lecroq, Francesco Pio Marino, Arianna Pavone, and Stefano Scafiti. Improving sampled matching through character context sampling. In Ugo de'Liguoro, Matteo Palazzo, and Luca Roversi, editors, *Proceedings of the 25th Italian Conference on Theoretical Computer Science, Torino, Italy, September 11-13, 2024*, volume 3811 of *CEUR Workshop Proceedings*, pages 300–312. CEUR-WS.org, 2024.
- [19] Thierry Lecroq and Francesco Pio Marino. Fast computation of the period and of the shortest cover of a string using its character-distance-sampling representation. *CoRR*, abs/2407.18216, 2024.
- [20] Vijay V. Vazirani. *Approximation Algorithms*. Springer, 2001.

- [21] Simone Faro, Thierry Lecroq, and Francesco Pio Marino. Optimal character distance sampling for exact string matching through set cover reformulation. *IEEE Access*, 14:13910–13920, 2026.
- [22] Simone Faro, Thierry Lecroq, Stefano Borzi, Simone Di Mauro, and Alessandro Maggio. The string matching algorithms research tool. In Jan Holub and Jan Zdárek, editors, *Proceedings of the Prague Stringology Conference 2016, Prague, Czech Republic, August 29-31, 2016*, pages 99–111. Department of Theoretical Computer Science, Faculty of Information Technology, Czech Technical University in Prague, 2016.
- [23] Paolo Ferragina and Gonzalo Navarro. *Pizza&Chili*. Available online: `pizzachili.dcc.uchile.cl`, 2005.
- [24] Ge Nong, Sen Zhang, and Wai Hong Chan. Linear suffix array construction by almost pure induced-sorting. *Proceedings of the 2009 Data Compression Conference (DCC)*, pages 193–202, 2009.
- [25] Domenico Cantone, Simone Faro, and M. Oguzhan Külekci. Shape-preserving pattern matching. In *Proceedings of the 21st Italian Conference on Theoretical Computer Science 2020*, volume 2756 of *CEUR Workshop Proceedings*, pages 137–148. CEUR-WS.org, 2020.
- [26] Sung Gwan Park, Magsarjav Bataa, Amihoud Amir, Gad M. Landau, and Kunsoo Park. Finding patterns and periods in Cartesian tree matching. *Theoretical Computer Science*, 845:181–197, 2020.
- [27] Michael Roberts, Wayne Hayes, Brian R Hunt, Stephen M Mount, and James A Yorke. Reducing storage requirements for biological sequence comparison. *Bioinformatics*, 20(18):3363–3369, 2004.
- [28] Omar Y Ahmed, Massimiliano Rossi, Travis Gagie, Christina Boucher, and Ben Langmead. Spumoni 2: improved classification using a pangenome index of minimizer digests. *Genome Biology*, 24(1):122, 2023.
- [29] Saul Schleimer, Daniel S Wilkerson, and Alexander Aiken. Winnowing: Local algorithms for document fingerprinting. In *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, pages 76–85. ACM, 2003.
- [30] Guillaume Marçais, Dan DeBlasio, and Carl Kingsford. Improving the performance of minimizers and syncmers. *Bioinformatics*, 34(13):i77–i85, 2018.
- [31] Robert C Edgar. Syncmers are more sensitive than minimizers for selecting conserved k-mers in biological sequences. *PeerJ*, 9:e10805, 2021.

- [32] Kristoffer Sahlin. Strobemers: an alternative to k-mers for sequence comparison. *bioRxiv*, 2021.
- [33] Ragnar Groot Koerkamp and Giulio Ermanno Pibiri. The mod-minimizer: A simple and efficient sampling algorithm for long k-mers. In *24th International Workshop on Algorithms in Bioinformatics (WABI)*, pages 11:1–11:23. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2024.
- [34] Heng Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018.
- [35] Brian D Ondov, Todd J Treangen, Páll Melsted, Andrew B Mallonee, Nicholas H Bergman, Sergey Koren, and Adam M Phillippy. Mash: fast genome and metagenome distance estimation using minhash. *Genome Biology*, 17(1):132, 2016.
- [36] Derrick E Wood and Steven L Salzberg. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*, 15(3):R46, 2014.
- [37] Juha Kärkkäinen and Esko Ukkonen. Sparse suffix trees. In Jin-yi Cai and C. K. Wong, editors, *Computing and Combinatorics, Second Annual International Conference, COCOON '96, Hong Kong, June 17-19, 1996, Proceedings*, volume 1090 of *Lecture Notes in Computer Science*, pages 219–230. Springer, 1996.
- [38] Szymon Grabowski and Marcin Raniszewski. Sampling the suffix array with minimizers. In Costas S. Iliopoulos, Simon J. Puglisi, and Emine Yilmaz, editors, *String Processing and Information Retrieval - 22nd International Symposium, SPIRE 2015, London, UK, September 1-4, 2015, Proceedings*, volume 9309 of *Lecture Notes in Computer Science*, pages 287–298. Springer, 2015.
- [39] Kimmo Fredriksson. Succinct backward-dawg-matching. *ACM Journal of Experimental Algorithmics*, 13, 2008.
- [40] Petr Procházka and Jan Holub. On-line searching in IUPAC nucleotide sequences. In Elisabetta De Maria, Ana L. N. Fred, and Hugo Gamboa, editors, *Proceedings of the 12th International Joint Conference on Biomedical Engineering Systems and Technologies (BIOSTEC 2019) - Volume 3: BIOINFORMATICS, Prague, Czech Republic, February 22-24, 2019*, pages 66–77. SciTePress, 2019.
- [41] R. Nigel Horspool. Practical fast searching in strings. *Software Practice and Experience*, 10(6):501–506, 1980.
- [42] Daniel Sunday. A very fast substring search algorithm. *Communications of the ACM*, 33(8):132–142, 1990.

- [43] Domenico Cantone, Simone Faro, and Arianna Pavone. Speeding up string matching by weak factor recognition. In Jan Holub and Jan Zdárek, editors, *Proceedings of the Prague Stringology Conference 2017, Prague, Czech Republic, August 28-30, 2017*, pages 42–50. Department of Theoretical Computer Science, Faculty of Information Technology, Czech Technical University in Prague, 2017.
- [44] Frantisek Franek, Christopher G. Jennings, and William F. Smyth. A simple fast hybrid pattern-matching algorithm. *Journal of Discrete Algorithms*, 5(4):682–695, 2007.
- [45] Axel Thue. Uber unendliche zeichenreihen. *Norske Vid Selsk. Skr. I Mat-Nat Kl. (Christiana)*, 7:1–22, 1906.
- [46] Alberto Apostolico and Andrzej Ehrenfeucht. Efficient detection of quasiperiodicities in strings. *Theoretical Computer Science*, 119(2):247–265, 1993.
- [47] Alberto Apostolico, Martin Farach, and Costas S. Iliopoulos. Optimal superprimitivity testing for strings. *Information Processing Letters*, 39(1):17–20, 1991.
- [48] Neerja Mhaskar and W. F. Smyth. String covering: A survey. *Fundamenta Informaticae*, 190(1):17–45, 2022.
- [49] Maxime Crochemore, Christophe Hancart, and Thierry Lecroq. *Algorithms on strings*. Cambridge University Press, 2007.
- [50] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Springer, 1972.
- [51] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.
- [52] Tomasz Kociumaka, Gonzalo Navarro, and Nicola Prezza. Towards a definitive measure of repetitiveness. In *Latin American Symposium on Theoretical Informatics*, pages 207–219. Springer, 2020.
- [53] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009.
- [54] David S. Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3):256–278, 1974.
- [55] L. Lovász. On the ratio of optimal integral and fractional covers. *Discrete Mathematics*, 13(4):383–390, 1975.

- [56] Sukhyeun Cho, Joong Chae Na, Kunsoo Park, and Jeong Seop Sim. Fast order-preserving pattern matching. In *Combinatorial Optimization and Applications - 7th International Conference, COCOA 2013, Proceedings*, volume 8287 of *Lecture Notes in Computer Science*, pages 295–305. Springer, 2013.
- [57] Djamal Belazzougui, Adeline Pierrot, Mathieu Raffinot, and Stéphane Vialette. Single and multiple consecutive permutation motif search. In *Algorithms and Computation - 24th International Symposium, ISAAC 2013, Hong Kong, China, December 16-18, 2013, Proceedings*, volume 8283 of *Lecture Notes in Computer Science*, pages 66–77. Springer, 2013.
- [58] Simone Faro and M. Oguzhan Külekci. Efficient algorithms for the order preserving pattern matching problem. In *Algorithmic Aspects in Information and Management - 11th International Conference, AAIM 2016, Proceedings*, volume 9778 of *Lecture Notes in Computer Science*, pages 185–196. Springer, 2016.
- [59] Siwoo Song, Geonmo Gu, Cheol Ryu, Simone Faro, Thierry Lecroq, and Kunsoo Park. Fast algorithms for single and multiple pattern Cartesian tree matching. *Theoretical Computer Science*, 849:47–63, 2021.
- [60] Tamanna Chhabra, M. Oğuzhan Külekci, and Jorma Tarhio. Alternative algorithms for order-preserving matching. In Jan Holub and Jan Žďárek, editors, *Proceedings of the Prague Stringology Conference 2015*, pages 36–46, Czech Technical University in Prague, Czech Republic, 2015.
- [61] Christian Charras, Thierry Lecroq, and Joseph Daniel Pehoushek. A very fast string matching algorithm for small alphabets and long patterns. *Lecture Notes in Computer Science*, 1448 LNCS:55–64, 1998.
- [62] Brenda S. Baker. A theory of parameterized pattern matching: Algorithms and applications. In *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing, STOC '93*, page 71–80, New York, NY, USA, 1993. Association for Computing Machinery.
- [63] Kimmo Fredriksson and Maxim Mozgovoy. Efficient parameterized string matching. *Information Processing Letters*, 100(3):91–96, 2006.
- [64] Leena Salmela and Jorma Tarhio. Fast parameterized matching with q-grams. *Journal of Discrete Algorithms*, 6(3):408–419, 2008.
- [65] Amihood Amir, Yonatan Aumann, Richard Cole, Moshe Lewenstein, and Ely Porat. Function matching: Algorithms, applications, and a lower bound. In Jos C. M. Baeten, Jan Karel Lenstra, Joachim Parrow, and Gerhard J. Woeginger, editors, *Automata, Languages and Programming, 30th International Colloquium, ICALP 2003, Eindhoven, The Netherlands, June 30 - July 4, 2003*.

- Proceedings*, volume 2719 of *Lecture Notes in Computer Science*, pages 929–942. Springer, 2003.
- [66] Carmit Hazay, Moshe Lewenstein, and Dekel Tsur. Two dimensional parameterized matching. In Alberto Apostolico, Maxime Crochemore, and Kunsoo Park, editors, *Combinatorial Pattern Matching, 16th Annual Symposium, CPM 2005, Jeju Island, Korea, June 19-22, 2005, Proceedings*, volume 3537 of *Lecture Notes in Computer Science*, pages 266–279. Springer, 2005.
- [67] Richard Cole, Carmit Hazay, Moshe Lewenstein, and Dekel Tsur. Two-dimensional parameterized matching. *ACM Transactions on Algorithms*, 11(2):12:1–12:30, 2014.
- [68] Ramana M. Idury and Alejandro A. Schäffer. Multiple matching of parameterized patterns. In Maxime Crochemore and Dan Gusfield, editors, *Combinatorial Pattern Matching, 5th Annual Symposium, CPM 94, Asilomar, California, USA, June 5-8, 1994, Proceedings*, volume 807 of *Lecture Notes in Computer Science*, pages 226–239. Springer, 1994.
- [69] Carmit Hazay, Moshe Lewenstein, and Dina Sokol. Approximate parameterized matching. In Susanne Albers and Tomasz Radzik, editors, *Algorithms - ESA 2004, 12th Annual European Symposium, Bergen, Norway, September 14-17, 2004, Proceedings*, volume 3221 of *Lecture Notes in Computer Science*, pages 414–425. Springer, 2004.
- [70] Brenda S. Baker. Parameterized diff. In Robert Endre Tarjan and Tandy J. Warnow, editors, *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms, 17-19 January 1999, Baltimore, Maryland, USA*, pages 854–855. ACM/SIAM, 1999.
- [71] S. Rao Kosaraju. Faster algorithms for the construction of parameterized suffix trees (preliminary version). In *36th Annual Symposium on Foundations of Computer Science, Milwaukee, Wisconsin, USA, 23-25 October 1995*, pages 631–637. IEEE Computer Society, 1995.
- [72] Richard Cole and Ramesh Hariharan. Faster suffix tree construction with missing suffix links. In F. Frances Yao and Eugene M. Luks, editors, *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing, May 21-23, 2000, Portland, OR, USA*, pages 407–415. ACM, 2000.
- [73] Brenda S. Baker. Parameterized pattern matching by boyer-moore-type algorithms. In Kenneth L. Clarkson, editor, *Proceedings of the Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, 22-24 January 1995. San Francisco, California, USA*, pages 541–550. ACM/SIAM, 1995.

- [74] Maxime Crochemore, Thierry Lecroq, Artur Czumaj, Leszek Gasieniec, Stefan Jarominek, Wojciech Plandowski, and Wojciech Rytter. Speeding up two string-matching algorithms. In Alain Finkel and Matthias Jantzen, editors, *STACS 92, 9th Annual Symposium on Theoretical Aspects of Computer Science, Cachan, France, February 13-15, 1992, Proceedings*, volume 577 of *Lecture Notes in Computer Science*, pages 589–600. Springer, 1992.
- [75] Amihood Amir, Martin Farach, and S. Muthukrishnan. Alphabet dependence in parameterized matching. *Information Processing Letters*, 49(3):111–115, 1994.
- [76] Ricardo A. Baeza-Yates and Gaston H. Gonnet. A new approach to text searching. In Nicholas J. Belkin and C. J. van Rijsbergen, editors, *SIGIR'89, 12th International Conference on Research and Development in Information Retrieval, Cambridge, Massachusetts, USA, June 25-28, 1989, Proceedings*, pages 168–175. ACM, 1989.
- [77] Jorma Tarhio. A sublinear algorithm for two-dimensional string matching. *Pattern Recognition Letters*, 17(8):833–838, 1996.